

УНИВЕРЗИТЕТ У БЕОГРАДУ
МАТЕМАТИЧКИ ФАКУЛТЕТ



Александар Н. Јаковљевић

РАЗВОЈ СКАЛАБИЛНИХ АПЛИКАЦИЈА
ЗАСНОВАНИХ НА ВЕШТАЧКОЈ
ИНТЕЛИГЕНЦИЈИ УПОТРЕБОМ
ТЕХНОЛОГИЈЕ КУБЕРНЕТЕС

мастер рад

Београд, 2026.

Ментор:

др Владимир Филиповић, редован професор
Универзитет у Београду, Математички факултет

Чланови комисије:

др Младен Николић, ванредни професор
Универзитет у Београду, Математички факултет

др Александар Картељ, доцент
Универзитет у Београду, Математички факултет

Датум одбране: тбд

Селени, Браци и родишељима.

Наслов мастер рада: Развој скалабилних апликација заснованих на вештачкој интелигенцији употребом технологије Кубернетес

Резиме: У овом раду представљено је емпиријско поређење два механизма хоризонталног аутоматског скалирања контејнеризованог сервиса за извршавање модела вештачке интелигенције у Кубернетес окружењу: хоризонталног аутоskalера подова (ХАСП), заснованог на искоришћености процесорске јединице, и КЕДА оквира, заснованог на дубини реда порука у *RabbitMQ* распоређивачу порука. Фокус рада није на општем водичу за развој скалабилних апликација, већ на контролисаном емпиријском поређењу два сигнала скалирања под истоветним условима, при чему изградња скалабилног сервиса за извршавање модела вештачке интелигенције служи као реалистична подлога за то поређење. Експериментална поставка је направљена тако да оба механизма скалирају исти циљни ресурс, са идентичним параметрима захтева и ограничења, истим опсегом реплика и истим апликативним кодом, док се мења искључиво сигнал скалирања. Поређење је спроведено над два контролисана плана оптерећења, са наглим скоковима и са постепеним порастом интензитета, при чему је сваки сценарио поновљен више пута. За потребе синтетичке евалуације уведен је композитни индекс ефикасности аутоskalирања који обједињује кумулативни заостатак, прекорачење нивоа услуге и просечан број реплика. Резултати показују да предност ниједног механизма није једнозначна, већ да два механизма заузимају различите тачке у компромису између квалитета услуге и утрошка ресурса. КЕДА уз исту пропусност користи знатно мањи и стабилнији број реплика, остварује бржи одзив и повољнији композитни индекс, док ХАСП одржава нижи интеграл заостатка у реду по цену већег и нестабилнијег утрошка реплика. Главни закључак рада је да у асинхроним системима за извршавање модела избор сигнала скалирања представља компромис, у коме сигнал заснован на реду омогућава ресурсно ефикасније и стабилније скалирање, а сигнал искоришћености процесорске јединице омогућава краћи ред порука.

Кључне речи: аутоматско скалирање, Кубернетес, КЕДА, хоризонтални аутоskalер подова, ХАСП, извршавање модела вештачке интелигенције, ред порука, архитектура заснована на догађајима, контејнеризација, Докер

Садржај

Садржај	vi
1 Увод	1
2 Преглед литературе	4
2.1 Контејнеризација и оркестрација	4
Докер	4
Контејнеризација наспрам виртуелних машина	7
Кубернетес	10
Асинхрона обрада и редови порука	14
2.2 Аутоматско скалирање у Кубернетесу	16
Категорије приступа скалирању	16
Хоризонтално аутоскалирање подова - ХАСП	16
Скалирање засновано на догађајима - КЕДА	18
Проактивни приступи скалирању	19
2.3 Извршавање модела ВИ као радно оптерећење	20
2.4 Метрике и евалуација квалитета услуге	23
2.5 Позиционирање рада	23
3 Методологија	26
3.1 Алати и технологије	26
Програмски језици и апликативни оквири	26
<i>RabbitMQ</i> као распоређивач порука	27
3.2 Преглед системске архитектуре	28
Логичка организација система	29
Ток обраде захтева	31
Податковни и контролни токови	32

3.3	Контејнеризација и поставке Кубернетеса	33
	Кубернетесови манифести и распоређивање	33
	Поставке ресурса и фер услови поређења	35
	Мрежа, доступност и провере стања	36
	Праћење и прикупљање метрике	37
3.4	Стратегије скалирања	38
	Циљ поређења и експериментални фактори	38
	Конфигурација ХАСП-а у експерименту	40
	Конфигурација КЕДА у експерименту	40
	Планови оптерећења	41
3.5	Метрике и критеријуми евалуације	42
	Композитни индекс ефикасности аутоскалирања	46
	Обрада података и излазне датотеке	48
3.6	Експериментални протокол и ограничења	48
	Поновљивост и контрола окружења	49
	Ограничења методологије	50
	Експериментално окружење	51
4	Имплементација	53
4.1	Структура пројекта	53
4.2	Сервис за оркестрацију захтева	57
4.3	Сервис за извршавање модела ВИ	63
4.4	Контејнеризација и аутоматизација изградње	71
4.5	Кубернетес манифести и распоређивање компоненти	73
4.6	Конфигурација механизма аутоматског скалирања	78
4.7	Мониторинг, генератор оптерећења и скрипта за анализу	82
5	Резултати и анализа	85
5.1	Резултати за план са наглим скоковима оптерећења	85
	Динамика заостатка у реду и броја реплика	85
	Метрике реда порука	86
	Метрике латенције и прекорачења нивоа услуге	87
	Долазна и сервисна стопа	89
	Метрике скалирања и индекс ефикасности аутоскалирања	90
	Реактивност и догађаји скалирања	92
	Закључак потпоглавља	92

5.2	Резултати за план са постепеним порастом оптерећења	94
	Динамика заостатка у реду и броја реплика	94
	Метрике реда порука	95
	Метрике латенције и прекорачења нивоа услуге	96
	Долазна и сервисна стопа	97
	Метрике скалирања и индекс ефикасности аутоскалирања	99
	Реактивност и догађаји скалирања	101
	Закључак потпоглавља	101
5.3	Анализа осетљивости циљних прагова	103
	Осетљивост ХАСП прага (60% наспрам 70%)	103
	Осетљивост КЕДА прага реда по реплици (60/80/100)	107
5.4	Преглед свих резултата	109
	Упоредна табела кључних показатеља	109
	Анализа понашања по типу оптерећења	109
	Ефикасност коришћења ресурса	110
	Резиме налаза	111
5.5	Претње валидности и ограничења резултата	112
6	Закључак	115
	Библиографија	118

Глава 1

Увод

Растућа примена модела вештачке интелигенције у продукционим системима поставља нове захтеве пред инфраструктуру задужену за њихово извршавање. Извршавање модела (енгл. *model inference*) вештачке интелигенције, које представља процес генерисања одговора на основу улазних података, је рачунарски интензивно оптерећење са специфичним карактеристикама. Време обраде по захтеву може бити релативно велико и променљиво, а потражња за сервисом подложна наглим или постепеним променама у интензитету [23]. У таквим условима, способност система да динамички прилагођава своје капацитете постаје критична за одржавање квалитета нивоа услуге и оптимално коришћење инфраструктурних ресурса. Контејнеризација и оркестрација контејнера, са Кубернетесом као доминантном платформом, омогућили су стандардизован приступ хоризонталном скалирању, односно динамичком повећавању или смањивању броја инстанци сервиса у складу са оптерећењем [6, 5]. У пракси, Кубернетес нуди хоризонтално аутошкалирање подова (ХАСП) као уграђени механизам који одлуке доноси на основу ресурсних метрика, најчешће искоришћености процесорске јединице [5]. Овај приступ је широко распрострањен, али има ограничења у системима где ресурсне метрике не одражавају верно стварни обим накупљеног посла, посебно у асинхроним архитектурама са редовима порука, где се поруке накупљају пре обраде [16].

Као алтернатива, КЕДА (енгл. *Kubernetes Event-Driven Autoscaling*) омогућава скалирање засновано на спољашњим метрикама, укључујући и дубину реда порука [16]. Основна претпоставка овог приступа је да ред порука представља непосреднији индикатор накупљеног посла него искоришћеност проце-

сорске јединице и да стога може омогућити брже и прецизније прилагођавање капацитета. Међутим, у постојећој литератури ретка су емпиријска поређења ова два механизма на конкретним радним оптерећењима, под контролисаним условима и уз формализоване метрике евалуације.

Постојећа истраживања о аутоматском скалирању контејнеризованих апликација углавном се фокусирају на оптимизацију параметра ХАСП механизма [4, 20] или на проактивне приступе засноване на машинском учењу [31, 14]. Заједничко за већину тих радова је да користе искоришћеност процесорске јединице или меморије као примарни сигнал скалирања и да евалуацију спроводе на оптерећењима са краћим и предвидљивијим временом обраде и да метрике евалуације свODE на просечну латенцију и искоришћеност ресурса, без анализе перцентилних латенција, кумулативног заостатка или времена прекорачења нивоа услуге.

Циљ овог рада је систематско емпиријско поређење ХАСП и КЕДА механизма хоризонталног аутоскалирања у контексту контејнеризованог сервиса за извршавање модела вештачке интелигенције. Поређење је засновано на асинхроној архитектури са *RabbitMQ* редом порука као распоређивачем порука, где се сервис за извршавање скалира или на основу искоришћености процесорске јединице (ХАСП) или на основу дубине реда порука (КЕДА). Експериментална поставка је направљена тако да оба механизма скалирају исти циљни ресурс, са идентичним параметрима захтева и ограничења, истим опсегом реплика и истим апликативним кодом, док се мења искључиво сигнал на основу кога се доноси одлука о скалирању. У складу с тим, тежиште рада није на изградњи општег водича за развој скалабилних апликација, већ на контролисаном емпиријском поређењу два механизма скалирања. Изградња скалабилног сервиса служи као реалистична подлога на којој се то поређење спроводи, а главни допринос су квантитативни налази о понашању два сигнала скалирања под истим условима.

У погледу доприноса овог рада, пројектована је експериментална поставка која омогућава контролисано поређење два механизма скалирања над истом апликативном логиком и инфраструктуром. Затим је уведен композитни индекс ефикасности аутоскалирања (АЕИ) као синтетички показатељ који спаја кумулативни заостатак, прекорачење нивоа услуге и просечан број реплика

у јединствену метрику за упоредну анализу. Сprovedена је и детаљна емпиријска анализа понашања оба механизма, која обухвата анализу осетљивости ХАСП прага и осетљивости КЕДА прага на дужину реда, а формулисани су и закључци засновани на конкретним квантитативним показатељима.

Што се тиче организације рада, у поглављу 2 представљени су теоријски и технолошки концепти на којима почива експериментална поставка. У то спадају контејнеризација, оркестрација, механизми аутоматског скалирања у Кубернетесу, асинхрона обрада и редови порука, извршавање модела вештачке интелигенције као радно оптерећење, као и преглед постојећих истраживања и идентификоване празнине у литератури. Поглавље 3 описује методологију: избор технологија, системску архитектуру, Кубернетес ресурсе и манифесте, стратегије скалирања, планове оптерећења, метрике евалуације и експериментални протокол. У поглављу 4 описана је имплементација система. Поглавље 5 садржи резултате и анализу, као и детаљно поређење ХАСП и КЕДА механизма у оба профила оптерећења, анализу осетљивости параметара и синтетички преглед налаза. Поглавље 6 сумира основне закључке рада и идентификује правце будућег истраживања.

Глава 2

Преглед литературе

У овом поглављу представљени су теоријски и технолошки концепти на којима почива експериментална поставка овог рада, као и преглед постојећих истраживања из области аутоматског скалирања контејнеризованих апликација. Поглавље је организовано тако да најпре уведе основне технологије (контејнеризацију и оркестрацију), затим представи механизме скалирања и постојећа истраживања, а на крају позиционира допринос овог рада у односу на идентификоване празнине у литератури.

2.1 Контејнеризација и оркестрација

Докер

Докер (енгл. *Docker*) је платформа отвореног кода која омогућава паковање, дистрибуирање и покретање апликација у изолованим окружењима која називамо контејнерима (енгл. *container*). Главна идеја Докера је да се апликација и све њене зависности пакују у један преносиви артефакт, Докер слику, који се може покренути на било ком систему који има Докер извршни систем. На тај начин је решен проблем портабилности, јер је окружење потпуно доследно без обзира на платформу. Докер користи клијент-серверску архитектуру, у којој клијент шаље захтеве системском процесу који управља контејнерима. Контејнери се покрећу као изоловани процеси на нивоу оперативног система, без потребе за покретањем целокупног гостујућег оперативног система као код виртуелних машина [25].

Докер се састоји од три главне компоненте:

- **Докер извршни систем** (енгл. *Docker Engine*) - системски сервис који омогућава рад са сликама и контејнерима;
- **Докер клијент** (енгл. *Docker CLI*) - командна линија за управљање;
- **Докер регистар** (енгл. *Docker Hub*) - регистар за складиштење и дистрибуцију слика [25, 12].

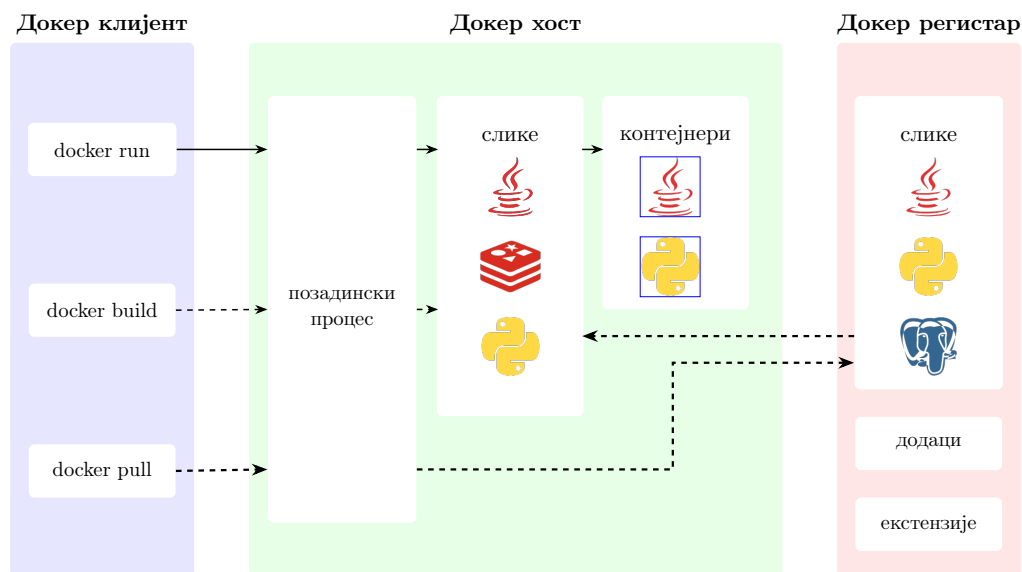
Докер извршни систем. Докер извршни систем представља централну компоненту Докер платформе, одговорну за креирање, управљање и извршавање контејнера. Састоји се из три међусобно повезане компоненте:

- **Докер системски процес** (енгл. *Docker daemon, dockerd*) - дуготрајан процес који слуша Докер АПИ захтеве и обавља операције као што су изградња слика, покретање и брисање контејнера, као и комуникација са другим системским процесима у кластеру;
- **Докер апликациони програмски интерфејс** - интерфејс који дефинише програмски приступ различитим Докер ресурсима, омогућавајући интеграцију са различитим системима и аутоматизацију процеса;
- ***containerd* и *runc*** - подсистеми који имплементирају управљање процесима и покретање контејнера на нивоу оперативног система. Компонента ***runc*** користи Линуксове механизме као што су именски простор и контролне групе за изолацију процеса и контролу ресурса [12, 21].

Докер извршни систем је написан у програмском језику Го, што га чини преносивим и ефикасним [8]. Сlike се креирају из декларативних Докер датотека (енгл. *Dockerfile*) у којима су дефинисани кораци за постављање окружења. Свака инструкција Докер датотеке представља слој слике који се кешира и поново користи, чиме се убрзава процес изградње и смањује потрошња ресурса [25, 21].

Докер клијент, регистар и хост. Поред извршног система, Докер платформу чине још три елемента релевантна за разумевање животног циклуса контејнера. Докер клијент (*CLI*) је интерфејс преко кога се издају команде извршном систему. Докер регистар служи за верзионисање и чување слика.

Изграђене слике се чувају у регистру (нпр. *Docker Hub*), одакле их оркестратор повлачи при распоређивању (енгл. *deployment*) [12]. Докер хост је рачунарски систем на коме је инсталиран извршни систем и на коме се извршавају контејнери, заједно са слојевима слика и волуменима за трајно складиштење.



Слика 2.1: Архитектура Докер система - приказ комуникације између Докер клијента, Докер хоста и Докер регистра.

Докер архитектура. Архитектура Докер платформе је приказана на слици 2.1 и заснива се на клијент-сервер моделу архитектуре. Корисник издаје команде преко Докер клијента које се затим прослеђују системском процесу на Докер хосту. Хост обрађује захтеве, управља контејнерима и сликама и по потреби преузима слике из регистра. На овај начин, Докер клијент, хост и регистар чине интегрисану целину која омогућава изградњу, складиштење и покретање контејнера на поновљив и преносив начин, чиме се обезбеђује конзистентност у свим фазама развоја и продукције [12]. Докер систем подржава и додатке (енгл. *plugins*) и екстензије (енгл. *extensions*), који проширују функционалност основног окружења. Додаци омогућавају интеграцију са спољним системима (нпр. мрежни или складишни управљачи), док екстензије додају алате у оквиру Докер Десктоп окружења, као што су алати за мониторинг и анализу перформанси [12].

Контејнеризација наспрам виртуелних машина

Традиционални модел. Традиционални начин извршавања апликација заснованих на виртуелним машинама захтева припрему целог гостујућег оперативног система за сваки сервис: инсталацију зависности, конфигурацију окружења, компилацију и извршавање апликације (слика 2.2). У сложеним системима ово временом доводи до одступања окружења (енгл. *environment drift*), односно стварања разлика између стања система у различитим фазама (развојна (енгл. *development*), продукција (енгл. *production*)), иако би требало да буду што сличнија. Последице одступања окружења подразумевају веће оперативне трошкове, спорије испоручивање верзија, потешкоће при репродукцији багова. Уобичајени процес испоручивања верзија подразумевао је да програмери достављају скрипту за инсталацију, а оперативни тим је извршава на виртуелним машинама, где се затим скрипте итеративно коригују док окружење не постане стабилно [6].



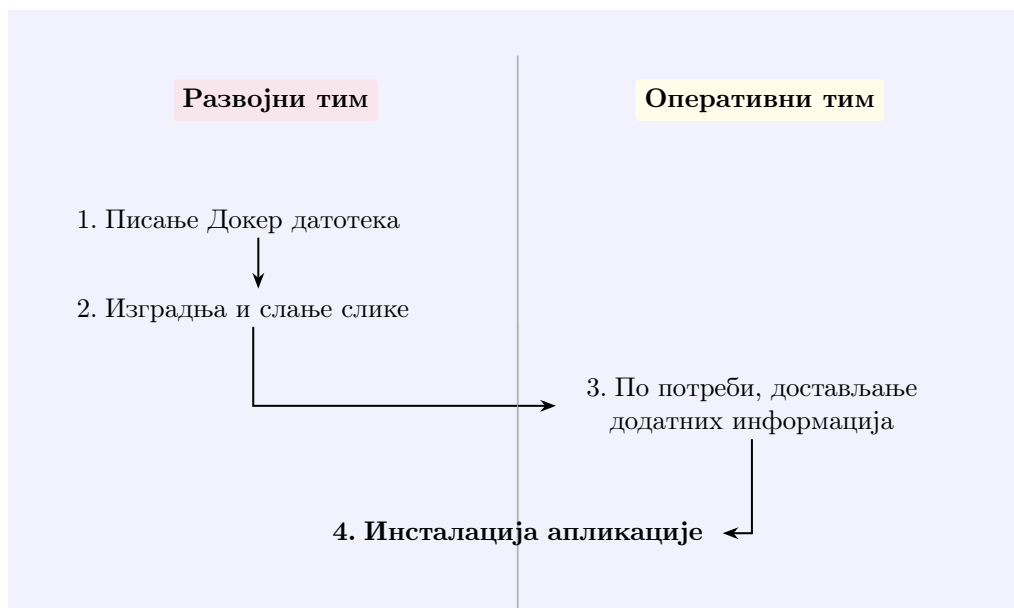
Слика 2.2: Традиционални начин извршавања апликација заснованих на виртуелним машинама

Контејнерски модел. Контејнери се ослањају на две функционалности Линуксовог језгра: именски простор (енгл. *namespaces*) и контролне групе (енгл. *control groups*, *cgroups*). Помоћу ових функционалности постиже се

изоловање процеса на нивоу хоста [17]. Развој апликације у овом моделу подразумева да развојни тим пакује апликацију у слику (енгл. *image*) која садржи бинарне артефакте и све зависности неопходне за извршавање. Сlike су слојевите и неизменљиве и граде се декларативно помоћу Дoкер датотека (енгл. *Dockerfile*). Успешно изграђене слике се чувају у Дoкер регистру. Покретањем слике добија се контејнер као „лаган”, изолован процес на хосту, без покретања целокупног гостујућег оперативног система. Овим се омогућава аутоматизован процес испоруке и елиминише се потреба да оперативни тим ручно извршава скрипте или подешава виртуелне машине. Уместо тога, исти артефакт се покреће у развојном, тестном и продукционом окружењу (слика 2.3). Дакле, овај приступ доноси:

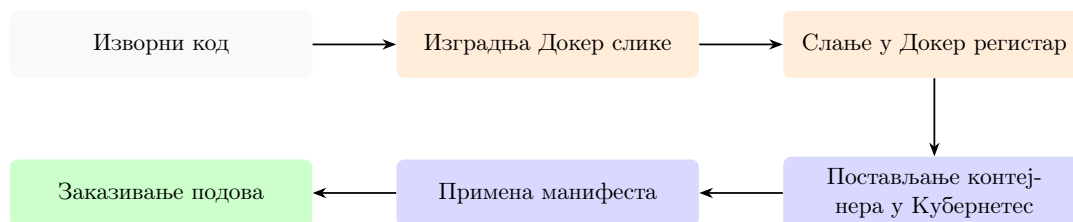
- **Униформност и поновљивост** - идентични контејнери креирају се коришћењем исте Дoкер слике и извршавају се на исти начин у различитим окружењима [17, 21].
- **Портабилност** - контејнери су у стању да се извршавају у свим окружењима, независно од тога у ком је програмском језику апликација написана или за који је оперативни систем апликација намењена [21].
- **Изолованост** - именски простори и контролне групе ограничавају потрошњу хардверских ресурса - радне меморије и процесорског времена [17].
- **Ефикасност** - брже стартовање и боље коришћење ресурса у односу на виртуелне машине, јер се дели језгро хоста [17].
- **Хоризонталну скалабилност** - једноставно умножавање слика и флексибилно прилагођавање оптерећењу [17].

Поред наведених предности, свака Дoкер слика се састоји од више слојева који се кеширају и поново користе, чиме се убрзава изградња и смањује потрошња простора. Иако контејнери обезбеђују изолацију процеса, она није потпуно хардверски одвојена као код виртуелних машина, па се у пракси користе додатни безбедносни механизми (енгл. *seccomp*, Линукс безбедносни модули) [21].



Слика 2.3: Контејнеризовани начин извршавања апликација

Животни циклус контејнера са Кубернетесом у пракси. Развојни тим пише и одржава Докер датотеке, означава их верзијама, а затим их шаље у Докер регистре. Након тога је неопходно описати жељено стање у Кубернетесу кроз манифесте, о којима ће бити реч касније у потпоглављу 2.1. Оперативни тим или систем за непрекидну интеграцију и непрекидну испоруку (енгл. *CI/CD*) примењује написане манифесте, док оркестратор распоређује контејнере на чворове, прати њихово стање и скалира их у складу са дефинисаним правилима [6, 24]. Конфигурације и променљиве вредности се чувају ван Докер слике, чиме се раздвајају инстанциране слике и оперативна подешавања [17].



Слика 2.4: *CI/CD* конвејер

Интеграција *CI/CD* система (нпр. *GitLab CI*, *GitHub Actions*, *Jenkins*)

омогућава аутоматизацију процеса изградње, тестирања и постављања слика, чиме се обезбеђује континуирана испорука и убрзано развијање нових верзија апликација (слика 2.4).

Кубернетес као оркестратор управља животним циклусом контејнера: надгледа њихово стање, рестартује неуспеле подове, скалира ресурсе и омогућава декларативно управљање инфраструктуром на нивоу кластера [6].

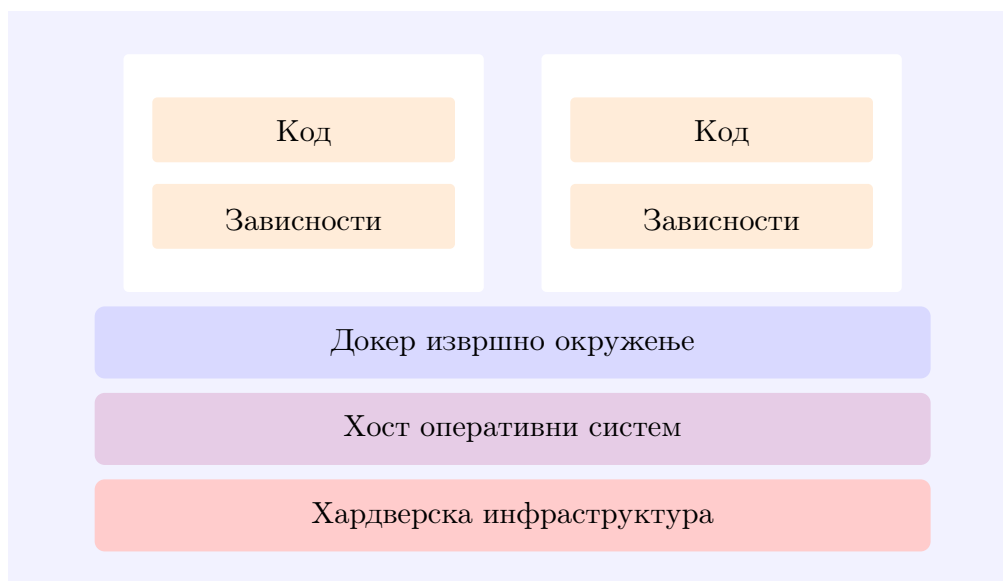
Поређење виртуелне машине и контејнера. Виртуелна машина обезбеђује јачу изолацију (целокупан гостујући оперативни систем) уз већу латенцију покретања и већи трошак ресурса. Контејнер је лакши, брже се покреће и ефикасније користи ресурсе, али дели језгро хоста и ослања се на политике изолације на нивоу језгра (слика 2.5). У облаку се ова два модела често комбинују, при чему се више контејнера извршава унутар једне виртуелне машине ради постизања бољег баланса између безбедности и економичности [21].

Кубернетес

Кубернетес је отворена платформа за оркестрацију контејнера која омогућава аутоматско постављање, скалирање и управљање контејнеризованим апликацијама. Коришћењем декларативних конфигурација и аутоматизација, корисник описује жељено стање (нпр. број реплика, верзију слике), а кластер остварује описано стање распоређивањем, управљањем и надоградњом компоненти [5].

Жељено стање се уписује у Кубернетесов АПИ (типично *YAML* манифести за *Deployment*, *Service*, итд.) након чега контролна равна, коју чине АПИ сервер, планер и контролори, распоређује подове на радне чворове и непрекидно усаглашава стварно стање са спецификацијом. **Под** (енгл. *Pod*) је најмања јединица извршавања, односно скуп једног или више контејнера који деле мрежни контекст и по потреби складиште. На сваком чвору *kubelet* преко интерфејса извршног окружења за контејнере (енгл. *Container Runtime Interface*) комуницира са окружењем које повлачи слике и покреће контејнере у подовима. Комуникација се остварује помоћу *containerd* подсистема који користи *runsc*, који директно комуницира са Линуксовим језгром ради изолације процеса [5].

Овим се остварује неколико практичних добити и особина:



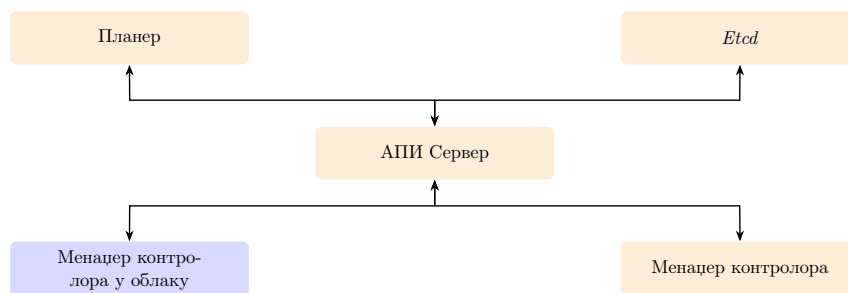
Слика 2.5: Архитектура контејнеризације

- **Аутоматизованост** (енгл. *automation*) - заказивање (енгл. *scheduling*), самопоправљање, контролисане надоградње, аутоскалирање.
- **Декларативност** (енгл. *declarative*) - манифести описују жељено стање, док АПИ чува историју и омогућава поновљивост.
- **Скалабилност** (енгл. *scalability*) - повећање и смањење реплика уз стабилан сервисни слој и ДНС резолуцију.
- **Самопоправљање** (енгл. *self-healing*) - контролери враћају систем у жељено стање при кваровима чворова и подова.

Контролна раван непрекидно тежи ка описаном стању (самопоправљање и контролисане надоградње) у манифесту, планер оптимално распоређује подове у складу са захтевима и лимитима ресурса, а стабилан сервисни слој (ДНС и сервисна апстракција) омогућава транспарентно хоризонтално скалирање без утицаја на клијенте [5, 6].

Архитектура Кубернетеса. Архитектура Кубернетеса се логички дели на контролну раван (енгл. *Control Plane*) и радне чворове (енгл. *Worker nodes*), који заједно чине функционални кластер. Контролна раван управља целокупним кластером, доноси одлуке о распоређивању, прати стање компоненти и реагује на промене, док радни чворови покрећу контејнере. Радни чворови такође чине целину коју називамо раван података.

Контролна раван. Контролна раван представља скуп компоненти које управљају целокупним понашањем кластера на основу декларативно задатог стања. Када корисник примени манифест преко `kubectl` алата, контролна раван прихвата захтев, одлучује на које чворове ће распоредити подове, не престано надгледа њихово стање и аутоматски рестартује подове који су пали или покреће нове уколико број реплика не одговара жељеном. Њена основна улога јесте доношење кластерских одлука и реакција на релевантне догађаје. Компоненте контролне равни могу бити покренуте на било којој машини у кластеру, али је подразумевано понашање да се све компоненте налазе на истој машини. Машина на којој су покренуте компоненте контролне равни логички се разликује од оне на којој се извршавају подови и контејнери. Оваква организација омогућава јасно раздвајање одговорности између слоја управљања и слоја извршавања, што поједностављује управљање, повећава предвидљивост система и олакшава изградњу високодоступних конфигурација [6, 5].



Слика 2.6: Контролна раван

Контролну раван чине следеће компоненте:

- **АПИ сервер** - централна тачка комуникације унутар кластера. Све компоненте, као и спољашњи корисници приступају кластеру преко Кубернетесовог апликативног програмског интерфејса. Поред комуникације, АПИ сервер ауторизује и валидира захтеве.
- **Планер** (енгл. *scheduler*) - одлучује на који радни чвор ће бити распоређен нови под.
- **Менаџер контролора** (енгл. *controller manager*) - извршава више процеса које називамо контролорима. Примери контролора су: контролор

чворова који прати стање чворова и реагује уколико је чвор недоступан; контролор послова који посматра послове и креира подове за извршавање послова; и многи други.

- ***etcd*** - кључ-вредност складиште података у које се чувају сви објекти и метаподаци кластера.
- **Менаџер контролора у облаку** (енгл. *cloud controller manager*) - компонента која омогућава специфичну логику за одговарајућу инфраструктуру у облаку. Омогућава повезивање кластера са АПИ-јем провајдера и раздваја компоненте које комуницирају са платформом од компоненти које комуницирају само са самим кластером [5].

Радни чворови. Радни чворови (енгл. *worker nodes*) су машине које извршавају контејнеризоване апликације у оквиру подова и обезбеђују ресурсе (процесорске јединице, меморију, мрежу и складиште). Задаци покретања, посматрања и одржавања сервиса обављају се помоћу следећих компоненти радног чвора:

- ***kubelet*** - комуницира са АПИ сервером и управља контејнерима на чвору.
- **Окружење за извршавање контејнера** (енгл. *Container Runtime*) - у пракси је то најчешће *containerd* (преко *CRI* интерфејса), док је Докер у овом раду коришћен за изградњу контејнерских слика.
- ***kube-proxy*** - балансира оптерећење мрежног саобраћаја које настаје приликом комуникације компоненти апликације и управља правилима мреже сваког радног чвора.

Оваква подела улога омогућава да контролна равна декларативно описује шта треба покренути, док радни чворови поуздано извршавају те спецификације и одржавају сервис доступним током промена (скалирања, надоградњи и опоравка) [17, 5].

Кубернетес клијент. Кубернетес клијент (енгл. *kubectl, Kubernetes Command Line Tool*) је стандардни командно-линијски клијент за рад са Кубернетес кластером. Клијент комуницира са АПИ сервером и омогућава увид у стање ресурса и извршавање администраторских и оперативних радњи у

складу са политиком приступа. Увид у стање ресурса се постиже командама `get` (листинг и резиме), `describe` (детаљан приказ и догађаји), `logs` (увид у дневнике контејнера), `exec` (интерактивно извршавање унутар контејнера). Управљање ресурсима се постиже на два начина: императивно управљање ресурсима (креирање и брисање) и декларативно управљање ресурсима применом манифеста командом `apply`. Оваква организација омогућава безбедан и репродуцибилан рад у различитим окружењима (локалном, тестном, продукционом) [5, 6].

Кубернетес манифест. Кубернетес манифест је спецификација Кубернетес објекта у *YAML* или *JSON* формату која декларативно описује жељено стање система. Уместо да се прописује низ оперативних корака, манифест дефинише исход који треба постићи: идентификацију и метаподатке објекта, структуру и параметре извршења, политике распоређивања, ограничења ресурса, сигурносне и мрежне поставке. Тако постављен, манифест постаје извор истине за конкретан ресурс, док контролна равна кластера преузима одговорност да стварно стање доведе у сагласност са описаним. При примени манифеста, која се постиже извршавањем команде `kubectl apply`, спецификација се уписује у Кубернетес АПИ и чува у складишту стања. Контролори посматрају промене и теже ка жељном стању. Организација подова и свих других Кубернетес објеката врши се помоћу етикета. Етикете (енгл. *labels*) представљају структуру типа кључ-вредност које се додају објектима ради логичког груписања и селекције, док селектори етикета (енгл. *label selectors*) дефинишу формалне услове за избор тих објеката [17]. На овом механизму заснивају се и кључни Кубернетес ресурси који се користе у пракси (нпр. *Deployment*, *Service*, *ScaledObject*, *HPA*).

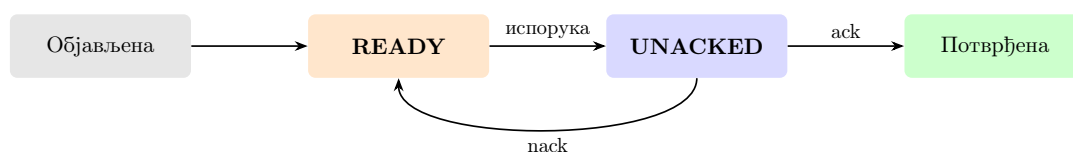
Асинхрона обрада и редови порука

Асинхрона обрада заснована на редовима порука представља уобичајен архитектурни образац за системе са променљивим временом обраде. Ред порука функционише као бафер који раздваја брзину пријема захтева од брзине њихове обраде, чиме се смањује директна спрегнутост између компоненти система [2, 32].

RabbitMQ је широко распрострањен распоређивач порука (енгл. *message broker*) отвореног кода [28]. *RabbitMQ* омогућава редове порука, поуздану

испоруку и раздвајање брзине пријема захтева од брзине њихове обраде [2, 32]. У контексту скалирања, стање реда порука (нпр. број порука спремних за обраду) представља потенцијални сигнал за аутоскалирање, јер директно одражава количину накупљеног посла у систему [16].

Овакво схватање потврђују и емпиријска истраживања аутоскалирања система за извршавање модела, која показују да ресурсне метрике саме по себи нису увек поуздан сигнал оптерећења и да избор метрике значајно утиче на квалитет услуге [19, 15]. Ова запажања детаљније се разматрају у одељку 2.5.



Слика 2.7: Животни циклус поруке у *RabbitMQ*-у.

Да би се разумео сигнал на основу кога КЕДА доноси одлуку о скалирању, неопходно је најпре описати животни циклус поруке у распоређивачу порука *RabbitMQ* (слика 2.7). Када произвођач (енгл. *publisher*) објави поруку, она се смешта у ред и прелази у стање спремности за испоруку (*READY*), у коме чека на потрошача (енгл. *consumer*) и представља накупљени, још не обрађени посао у систему. Када је потрошач преузме, порука прелази у стање обраде, односно у стање у коме је испоручена, али још није потврђена (*UNACKED*). Након успешне обраде, потрошач шаље потврду (*ack*), на основу које *RabbitMQ* трајно уклања поруку из реда. Уколико потрошач откаже или поруку одбије пре потврде (*nack*), порука се враћа у стање *READY* и поново постаје доступна за испоруку, чиме се обезбеђује поуздана испорука.

Број порука које потрошач може истовремено да држи у стању *UNACKED* ограничен је параметром `prefetch_count`, који одређује колико унапред порука испоручује *RabbitMQ* сваком потрошачу пре него што добије потврду претходних. Тиме се регулише степен паралелизма и спречава да један потрошач преузме несразмерно велик део заостатка. Овај параметар, међутим, има дубљу улогу, јер директно одређује који део укупног заостатка остаје видљив механизму скалирања.

Управо у томе лежи кључна веза са КЕДА сигналом. КЕДА у овом раду реагује на дубину реда дефинисану као број порука у стању *READY*, а не на

поруке које су већ преузете и налазе се у обради (*UNACKED*). Поруке које су потрошачи већ повукли тиме постају невидљиве за сигнал скалирања, иако и даље представљају накупљени посао. Параметар `prefetch_count` стога посредно обликује КЕДА сигнал: ниска вредност задржава готово цео заостатак у стању *READY* и видљивим за сигнал, док би висока вредност преместила знатан део заостатка у стање *UNACKED* и тако га сакрила од механизма скалирања. Из тог разлога је у експерименталној поставци вредност параметра `prefetch_count` намерно ограничена, како би дубина реда верно одражавала стварни заостатак.

2.2 Аутоматско скалирање у Кубернетесу

Категорије приступа скалирању

Аутоматско скалирање (енгл. *autoscaling*) је механизам динамичког прилагођавања рачунарских ресурса променљивом оптерећењу. У литератури се разликују три основне категорије приступа: реактивно, проактивно и хибридно скалирање [27].

Реактивно скалирање доноси одлуке на основу тренутних или недавних вредности метрика. Оног тренутка када посматрана метрика пређе дефинисани праг, систем иницира промену броја реплика. Овај приступ је једноставан за имплементацију, али може каснити у односу на нагле промене оптерећења [27]. **Проактивно скалирање** користи предиктивне моделе (нпр. временске серије, моделе машинског учења) за предвиђање будућег оптерећења и превентивно прилагођава ресурсе [31, 34]. **Хибридни приступи** комбинују реактивне и проактивне приступе.

Хоризонтално аутоскалирање подова - ХАСП

ХАСП (Хоризонтално аутоскалирање подова) је реализован као Кубернетес АПИ ресурс и контролер који периодично прилагођава жељени број реплика циљног ресурса (нпр. распоређивач порука) тако да усагласи уочене метрике са дефинисаним циљем [5]. ХАСП представља стандардни механизам скалирања у Кубернетесу за ресурсне сигнале, при чему се метрике читају са сервера за метрике (енгл. *metrics server*) или преко механизма за екстерне метрике. Контролна петља није непрекидна; подразумевано се извршава у

интервалима контролисаним параметром `horizontal-pod-autoscaler-sync-period`, чија је подразумевана вредност 15 секунди [5]. Ова вредност одређује најкраће време за које ХАСП може да примети промену метрике и као таква је задржана у експерименту, док КЕДА свој извор метрике испитује чешће (параметром `pollingInterval`).

Искоришћеност процесорске јединице као сигнал. Када се ХАСП конфигурише да користи просечну искоришћеност (`averageUtilization`), искоришћеност процесорске јединице се тумачи као проценат вредности броја захтева (`requests`) дефинисане у спецификацији контејнера. Практично, циљ од 60% значи да ХАСП тежи да одржи просечну потрошњу процесора на нивоу $\approx 0.6 \cdot \text{request}$ по поду. Ово имплицира да је подешавање `request` методолошки критично за репродукцибилност, јер ХАСП у том случају скалира релативно у односу на захтеване ресурсе [5]. Ако релевантна метрика броја захтева није дефинисана, онда није могуће израчунати искоришћеност на основу те метрике и ХАСП неће доносити одлуке о скалирању користећи исту [5].

Алгоритам израчунавања реплика. ХАСП контролер користи однос тренутне и циљне вредности метрике како би израчунао препоручени број реплика. У поједностављеној форми, принцип израчунавања може се представити као [5]:

$$N_{\text{new}} = \left\lceil N_{\text{current}} \cdot \frac{M_{\text{current}}}{M_{\text{target}}} \right\rceil \quad (2.1)$$

где су:

- N_{current} - тренутни број реплика,
- N_{new} - израчунати жељени број реплика,
- M_{current} - тренутна вредност посматране метрике (нпр. просечна искоришћеност процесорске јединице),
- M_{target} - циљна вредност метрике.

У практичним имплементацијама, ХАСП примењује и додатне механизме стабилитације (нпр. прозоре стабилизације при смањењу броја реплика) како би се избегло пречесто мењање броја реплика при краткотрајним осцилацијама

сигнала. Повећање циљног прага по правилу смањује број реплика, јер систем дуже толерише веће оптерећење по реплици, али може повећати латенцију и време задржавања порука у реду. Због тога се у раду параметар прага посматра као параметар осетљивости, а не као независна стратегија скалирања.

Скалирање засновано на догађајима - КЕДА

У **КЕДА** сценарију, сервис за извршавање модела се скалира на основу спољашње метрике која описује стварни обим посла у систему, попут дужине реда порука у распоређивачу порука (*RabbitMQ*) [16]. КЕДА уводи *event-driven* приступ скалирању кроз Кубернетес ресурсе, који повезује циљни ресурс (нпр. *Deployment*) са окидачем дефинисаним над спољашњом метриком. У пракси, КЕДА периодично читава спољашњу метрику, примењује дефинисана правила и иницира промену броја реплика унутар задатих граница [16].

За овај сценарио се дефинише скалирајући објекат (*ScaledObject*) који садржи:

- циљни ресурс који се скалира,
- минималан и максималан број реплика,
- параметре скалирања,
- опис окидача и циљну вредност реда по реплици.

Спољашња метрика као сигнал. Основна интуиција КЕДА приступа је да ред порука представља непосреднији индикатор накупљеног посла него метрика процесорских јединица, посебно у асинхроним системима и у сценаријима са променљивим временом обраде. Ако број порука у реду расте, КЕДА може раније иницирати скалирање, чак и пре него што ресурсне метрике достигну праг који би активирао ХАСП, што је нарочито релевантно у сценаријима наглих скокова и код извршавања модела са променљивим временом обраде [16].

Што се тиче алгорита за израчунавање броја реплика, он је исти као код ХАСП стратегије (једначина 2.1). То није случајност, већ последица интерне архитектуре КЕДА. КЕДА не имплементира сопствену петљу скалирања, већ

за сваки `ScaledObject` аутоматски креира и одржава Кубернетесов ХАСП ресурс, у који се циљна вредност реда по реплици уписује као циљ спољашње (екстерне) метрике [16]. КЕДА при том делује као сервер за метрике који ХАСП-у испоручује тренутну дубину реда, док саму одлуку о броју реплика доноси ХАСП по истој формули 2.1. Захваљујући томе, два механизма која се у овом раду пореде користе идентичан алгоритам израчунавања броја реплика, а разликују се искључиво у сигналу који улази у формулу (искоришћеност процесорске јединице наспрам дужине реда), што је и предуслов за фер поређење.

Проактивни приступи скалирању

Проактивни приступи аутоматском скалирању заснивају се на предикцији будућег оптерећења, уместо на реаговању на тренутно стање метрика. Оба механизма која се пореде у овом раду припадају реактивној категорији, па се проактивни приступи овде не разматрају експериментално, већ се наводе ради потпуности класификације и као могући правац будућих истраживања. Основна мотивација је уклањање структурног закашњења реактивних механизма: ХАСП покреће скалирање тек пошто метрика пређе праг, а до тренутка када нова реплика постане спремна за обраду, укључујући покретање пода и иницијализацију апликације, систем је већ одређено време у стању прекорачења капацитета [27]. Проактивни приступ предвиђа тај тренутак и покреће скалирање унапред, чиме се латенција покретања пода покрива пре него што оптерећење достигне критичну вредност.

Предиктивна компонента типично ради над историјским временским серијама метрика (нпр. број захтева у јединици времена, искоришћеност процесорске јединице, дубина реда порука, број корисника) и производи прогнозу за наредни временски интервал [34]. За оптерећења са очигледном периодичношћу, попут дневних образаца, статистички модели показују добре резултате [31].

У погледу интеграције са Кубернетесом, постоје два уобичајена начина имплементације. Први је да предиктивна компонента израчунава жељени број реплика, заобилази ХАСП контролер и уписује број реплика директно у `spec.replicas`. Други приступ такође користи ХАСП, али му уместо тренутно измерене вредности метрике подмеће прогнозирану тако што предиктор своје резултате излаже као прилагођену метрику кроз Кубернетес АПИ.

ХАСП тада скалира на основу те предвиђене вредности, а не стварно измерене, при чему цео остатак механизма (нпр. минималан и максималан број реплика) остаје непромењен [5]. Предиктор се у том случају имплементира као засебан сервис или периодични посао (енгл. *CronJob*), обучава се на историјским временским серијама прикупљеним из Прометеја и периодично излаже свежу прогнозу кроз прилагођени адаптер метрика.

Главна ограничење свих проактивних приступа је претпоставка да је будуће оптерећење у довољној мери одређено прошлим, што је услов који важи за оптерећења са правилним обрасцима, али не и за нагле и непредвидиве скокове. Поред тога, сваки такав систем захтева периодично тренирање предиктора и додатну инфраструктуру за излагање прогнозе Кубернетес АПИ-ју, што повећава оперативну сложеност у односу на реактивне механизме попут ХАСП-а или КЕДА-е.

2.3 Извршавање модела ВИ као радно оптерећење

Извршавање модела вештачке интелигенције у продукционим системима представља радно оптерећење са карактеристикама које га суштински разликују од типичних веб сервиса. У овом потпоглављу описане су те карактеристике и њихове импликације за скалирање, чиме се мотивише избор сигнала скалирања који је анализиран у овом раду.

Карактеристике оптерећења при извршавању модела

За разлику од типичних веб сервиса, код којих је време обраде по захтеву кратко и релативно предвидљиво, оптерећења при извршавању модела имају неколико специфичних својстава:

- **Дуже и променљиво време обраде.** Време извршавања модела зависи од сложености и величине улазних података, величине модела и расположивих рачунарских ресурса, па није познато унапред. У систему који се у овом раду анализира, реч је о екстрактивном одговарању на питања, где варијабилност времена обраде потиче од дужине и сложености улазног контекста, броја кандидата за претрагу и тренутне оптерећености процесорске јединице. У ширем спектру оптерећења при изврша-

вању модела, варијабилност може бити још израженија. На пример, код великих језичких модела са ауторегресивним генерисањем укупно трајање захтева зависи и од дужине излазног низа токена, који се генеришу секвенцијално [1]. Заједничка последица у оба случаја јесте да време обраде по захтеву није константно нити поуздано предвидиво, што отежава скалирање засновано на простим ресурсним метрикама.

- **Рачунарска интензивност.** Извршавање модела захтева значајне рачунарске ресурсе (процесорско време), а у сложенијим системима и графичке процесоре. Однос између утрошених ресурса и броја обрађених захтева не мора бити линеаран. Код великих језичких модела, различите фазе обраде оптерећују различите ресурсе, што указује да праћење искоришћености процесора као јединог сигнала скалирања није довољно поуздано [1]. И у систему који се овде анализира, иако је реч о екстрактивном моделу, искоришћеност процесорске јединице не одражава директно количину посла накупљеног у реду, што је управо мотивација за поређење са сигналом заснованим на дужини реда.
- **Потребе за асинхронном обрадом.** Због дужег времена обраде, синхрони модел обраде захтева постаје непрактичан при већем оптерећењу. Захтеви се стога типично сакупљају у ред порука и обрађују асинхронно, чиме се раздваја брзина пријема захтева од брзине њихове обраде [32]. Овај образац је широко примењен у системима са дугим временом обраде, као што су генерисање слика или сумирање текста.

Латенција, пропусност и сервирање модела

У литератури о системима који извршавају моделе често се разматра однос између латенције и пропусности. Пропусност означава број захтева које систем може да обради у јединици времена, док латенција означава време које протекне од пријема захтева до добијања резултата. Код система за сервирање модела (енгл. *model serving* или *inference serving*), овај однос је посебно значајан јер захтеви могу имати различито време обраде, а систем мора да одржи прихватљив квалитет услуге и при променљивом оптерећењу.

Један од механизма који се у таквим системима користи за повећање пропусности јесте груписање више захтева у једну обраду, односно *batching*. Овај приступ може побољшати искоришћеност ресурса и повећати број об-

рађених захтева, али истовремено може утицати на латенцију појединачних захтева, јер захтев може чекати да се формира група за обраду. Радови из области сервирања модела показују да стратегије груписања, заказивања и управљања захтевима могу значајно утицати на латенцију, пропусност и искоришћеност ресурса [9]. Треба нагласити да систем који се у овом раду анализира не примењује груписање. У раду сваки потрошач обрађује поруке појединачно (са параметром `prefetch_count` који ограничава број истовремено преузетих порука и потврдом обраде по свакој поруци), па је груписање овде наведено само ради потпуности прегледа механизма пропусности, а није фактор у експерименталном поређењу.

Овај проблем је додатно изражен код система за сервирање великих језичких модела, где се често посматра размена између високе пропусности и ниске латенције. Због тога се, поред просечне латенције, у литератури и пракси често анализирају и латенције високих перцентила, као што су $p95$ и $p99$, јер оне боље приказују понашање система код захтева који најдуже чекају или се најспорије обрађују [1]. Ове метрике су посебно важне у сценаријима у којима је потребно проценити не само колико захтева систем може да обради, већ и да ли систем одржава прихватљиво време одзива при повећаном оптерећењу.

Импликације за скалирање

Описане карактеристике система за извршавање модела имају значајне импликације за избор сигнала скалирања. У класичним веб сервисима тренутно оптерећење се често може приближно проценити на основу ресурсних метрика, као што су искоришћеност процесора или меморије. Међутим, код асинхроних архитектура са редом порука део оптерећења није видљив само кроз активну обраду на подовима, већ и кроз број захтева који чекају у реду.

Због тога искоришћеност процесора не мора увек у потпуности да одражава стварни обим посла у систему. Она може бити висока када се обрађује мали број рачунарски захтевних захтева, али сама по себи не показује колико захтева још чека на обраду. Насупрот томе, дужина реда порука непосредније описује заостатак необрађеног посла и може бити релевантан сигнал у системима у којима се захтеви баферују пре извршавања модела.

У литератури се истиче да избор метрике скалирања може значајно утицати на понашање аутоskalера и да различите врсте апликација могу захтевати различите сигнале скалирања [23, 13]. У том контексту, поређење

ресурсних метрика и спољних сигнала представља важан аспект анализе аутоматског скалирања. За овај рад је посебно релевантна разлика између искоришћености процесорске јединице као интерне ресурсне метрике и дужине реда порука као екстерног сигнала који одражава акумулирани заостатак. На основу ових запажања, истраживачко питање рада гласи: *у којој мери дужина реда порука, у поређењу са искоришћеношћу процесора, представља погодан сигнал скалирања за посматрано асинхроно оптерећење при извршавању модела?*

2.4 Метрике и евалуација квалитета услуге

У системима са редовима порука и променљивим временом обраде, просечна вредност латенције није довољна за описивање квалитета услуге, јер мали проценат захтева може трпети несразмерно велика кашњења. Из тог разлога, у пракси и литератури се користе перцентили латенције (нпр. p_{95} , p_{99}), који описују понашање горњег репа расподеле [10, 7]. Анализа система са редовима показује да укупно време одговора обухвата и време чекања у реду и време саме обраде [7].

Формализација метрика нивоа услуге (енгл. *Service Level Agreement, SLA*) представља важан аспект дизајна и евалуације система у облаку. У оквиру стандардизованих приступа дефинишу се метрике доступности и перформанси, укључујући концепт преступних интервала (енгл. *violated intervals*), временских периода у којима посматрана метрика премашује задати праг [18].

2.5 Позиционирање рада

Преглед постојеће литературе показује да је аутоматско скалирање контејнеризованих апликација широко истраживано подручје, посебно у контексту Кубернетес ХАСП механизма [4, 20], проактивних приступа заснованих на машинском учењу [31, 14] и скалирања вођеног догађајима (енгл. *event-driven scaling*) [16]. У овим радовима се као сигнали скалирања најчешће користе ресурсне метрике, попут искоришћености процесора или меморије, док поједини приступи уводе предиктивне моделе или прилагођене метрике како би се побољшала реакција система на промене у оптерећењу.

У контексту извршавања модела вештачке интелигенције већ постоје радови који разматрају различите сигнале скалирања, али се по правилу ослањају на синхрону комуникацију и не мере кумулативни заостатак у реду. Митра [19] пореди скалирање засновано на искоришћености меморије графичке картице са скалирањем заснованим на броју захтева у обради (енгл. *inflight requests*) у Кубернетес окружењу са *KFServing/Knative* платформом. Главни налаз је постојање компромиса: приступ заснован на броју захтева у обради остварује ниже време одзива, али по цену ангажовања већег броја графичких процесора, док приступ заснован на искоришћености графичке меморије користи ресурсе конзервативније (једна графичка картица), уз спорије време одзива. Аутор такође примећује да снижавање прага искоришћености зближава понашање два приступа. Шредер и сарадници [30] пореде технике скалирања на различитим нивоима (на апликативном нивоу са *TorchServe* и *RayServe*, и на контејнерском нивоу са *K3s*, *AWS ECS* и *EKS*) у локалном окружењу и у облаку, при чему је контејнерско скалирање конфигурисано на основу искоришћености процесорске јединице (праг 50%), а апликативно на основу циљног броја истовремених захтева. Њихов главни налаз је да се локална окружења брже прилагођавају оптерећењу јер већ располажу потребним ресурсима, док сервиси у облаку касне због покретања нових виртуелних машина и грануларности прикупљања метрика. Џојс и Себастијан [15] испитују Кубернетес ХАСП заснован на искоришћености процесорске јединице за различите типове ВИ апликација и налазе да за просте моделе (линеарна и логистичка регресија) ХАСП остаје на минималном броју реплика, али да за апликацију засновану на моделу *DistilBERT* за одговарање на питања систем скалира до осам реплика уз изразит пораст времена одзива. Закључују да искоришћеност процесорске јединице сама по себи није довољан сигнал за сложеније моделе и да су потребне прилагођене, апликативно специфичне метрике. Заједнички закључак ових радова јесте да избор сигнала скалирања значајно утиче на квалитет услуге код система за извршавање модела и да ресурсне метрике нису увек најпоузданији сигнал.

Овај рад се надовезује на тај правац истраживања, али посматра ужи и конкретнији сценарио: *асинхрону* архитектуру за извршавање модела у којој се захтеви прво смештају у *RabbitMQ* ред, а затим их обрађује посебан сервис за извршавање модела. У таквом окружењу искоришћеност процесорске јединице и дужина реда порука могу дати различиту слику о стању

система. Искоришћеност процесорске јединице показује тренутно оптерећење активних подова, док дужина реда указује на количину посла који још није обрађен. Због тога је корисно испитати како се ова два сигнала понашају у истом систему и под истим плановима оптерећења.

У односу на најближе сродне радове, овај рад се разликује пре свега по комбинацији коришћене архитектуре и механизма скалирања. Уместо синхроног ХТТП-а, *gRPC*-а или модела сервирања на захтевима, комуникација се одвија асинхроно, преко реда порука. Поређење је постављено између ХАСП механизма заснованог на искоришћености процесорске јединице и КЕДА механизма заснованог на дужини *RabbitMQ* реда, при чему се оба приступа испитују над истом апликацијом, истим моделом, истим Кубернетес окружењем и истим плановима оптерећења.

Циљ рада није да покаже универзалну предност једног механизма над другим, већ да емпиријски испита у којим условима одређени сигнал скалирања пружа стабилније понашање система. Због тога се анализа не заснива само на броју реплика или просечној латенцији, већ обухвата и *p95* и *p99* латенцију, прекорачења *SLA* прага, динамику заостатка у реду, време опоравка система и композитни индекс ефикасности аутоскалирања.

На тај начин рад доприноси постојећој литератури контролисаним поређењем два практично релевантна приступа скалирању у конкретном асинхронном систему за извршавање модела вештачке интелигенције. Добијени резултати могу помоћи у разумевању предности и ограничења скалирања заснованог на сигналу искоришћености процесорске јединице и скалирања заснованог на дужини реда порука, посебно у системима у којима се захтеви накупљају у реду пре саме обраде.

Глава 3

Методологија

У претходном поглављу представљени су основни концепти и технологије (Докер, Кубернетес, *RabbitMQ*) који чине основу за испоруку и извршавање система, као и постојећа истраживања о аутоматском скалирању. У овом поглављу описује се конкретна примена тих концепата у експерименталној поставци: избор технологија и подела система, архитектура, начин паковања сервиса у контејнерске слике, Кубернетес ресурси дефинисани манифестима, подешавања аутоматског скалирања, метрике евалуације и експериментални протокол.

3.1 Алати и технологије

Програмски језици и апликативни оквири

У оквиру рада су реализоване две апликативне компоненте, при чему је архитектура система намерно подељена тако да се логички и извршно раздвоје оркестрација захтева и извршавање модела:

- **Сервис за оркестрацију захтева**, реализован у програмском језику Јава, уз употребу *Spring Boot* оквира.
- **Сервис за извршавање модела ВИ**, реализован у програмском језику Пајтон, уз употребу *FastAPI* оквира.

Сервис за оркестрацију захтева. Избор програмског језика Јава и *Spring Boot* оквира за сервис оркестрације заснива се на њиховој широкој

употреби у развоју серверских и микросервисних апликација, као и на до-број подршци за продукционо окружење и богатом екосистему библиотека [22, 33]. *Spring Boot* посебно олакшава развој сервиса, јер нуди конвенције које убрзавају развој, као и бројне нефункционалне могућности (нпр. конфигурацију, статусне провере и интеграције), што га чини погодним избором за компоненту која координише ток захтева у систему. Поред тога, овај оквир је погодан за интеграцију са инфраструктурним компонентама које су типичне у микросервисном окружењу, укључујући механизме размене порука и спољашњу конфигурацију сервиса [33].

Сервис за извршавање модела ВИ. Сервис за извршавање модела ВИ реализован је у програмском језику Пајтон због библиотека за машинско учење које су једноставне за употребу, библиотека за обраду података и имплементацију извршавања модела, као и због брзине развоја прототипа и експерименталних поставки [26]. За апликативни оквир је изабран *FastAPI*, који представља оквир за развој АПИ сервиса у Пајтону са фокусом на високе перформансе и једноставну декларацију интерфејса [29]. Ове карактеристике су посебно значајне у контексту сервиса за извршавање модела, где је потребно брзо експериментисање са интерфејсом сервиса, јасна валидација улазних параметара и једноставна интеграција са оркестрационим слојем.

Практичне импликације за овај рад. Оваква подела система прати принципе микросервисне архитектуре, у којој се функционалности раздвајају на мање, независне сервисе. У конкретном случају, раздвајање оркестрације и извршавања модела омогућава да се у експериментима прецизније посматра утицај скалирања на компоненту која представља стварни носилац рачунарског оптерећења, односно сервис за извршавање модела. Истовремено, овакав приступ олакшава независно подешавање, тестирање и хоризонтално скалирање појединачних компоненти, што је у складу са праксама развоја контејнеризованих апликација у савременим оркестрационим платформама [6, 5, 24].

***RabbitMQ* као распоређивач порука**

У експерименталној поставци *RabbitMQ* се користи за асинхрону комуникацију између сервиса за оркестрацију захтева и сервиса за извршавање модела. Општи концепти редова порука и поуздане испоруке описани су у поглављу 2. Овде је методолошки релевантна једна последица, а то је да је

ред порука истовремено и оперативни бафер између пријема и обраде и извор сигнала оптерећења на основу кога се доноси одлука о скалирању. Свака јединица посла у систему одговара једној поруци у реду, па се обим накупљеног посла директно читава из дужине реда.

Управо та двострука улога реда чини поставку погодном за поређење две стратегије хоризонталног скалирања сервиса за извршавање модела:

- **скалирање на основу искоришћености процесорске јединице**, где сигнал потиче из ресурсних метрика пода, што је уобичајен приступ у Кубернетес окружењима;
- **скалирање на основу дужине реда**, где сигнал потиче из броја порука у реду, што представља приступ заснован на догађајима и реализује се одговарајућим скалером (нпр. КЕДА).

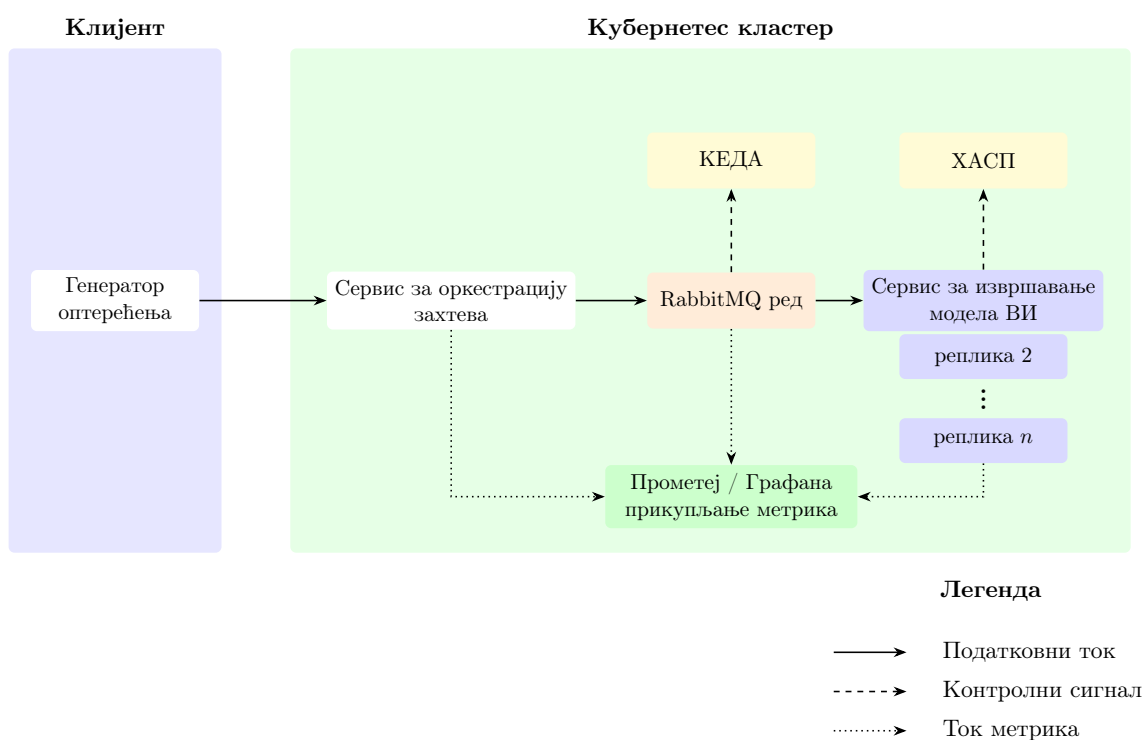
У асинхронном режиму са променљивим временом извршавања модела, ресурсне метрике могу каснити у односу на настанак заостатка или га недовољно верно описивати, док дужина реда непосредно одражава количину накупљеног посла. Због тога поређење ова два сигнала под истим условима представља релевантан експериментални оквир за анализу понашања сервиса за извршавање модела под оптерећењем.

3.2 Преглед системске архитектуре

Након представљања коришћених алата и технологија, у овом потпоглављу описује се начин на који су оне интегрисане у јединствену експерименталну поставку. Архитектура система је пројектована тако да омогући контролисано посматрање понашања сервиса за извршавање модела ВИ под променљивим оптерећењем и анализу реакције различитих механизма аутоматског скалирања.

Основна идеја архитектуре система заснива се на раздвајању улазног оптерећења од капацитета обраде. На тај начин се омогућава да интензитет пристиглих захтева буде независан од тренутног капацитета сервиса за извршавање модела ВИ. Уместо директне синхроне обраде, систем користи асинхрони модел заснован на реду порука, чиме се ствара погодан експериментални оквир за анализу заостајања, промене броја реплика и реакције механизма скалирања.

Кључна методолошка последица овакве организације је да се механизми скалирања могу поредити над истом апликативном логиком, уз промену кључиво сигнала на основу којег се доносе одлуке о скалирању. У наставку су приказане компоненте система, ток обраде и раздвајање податног и контролног тока, што представља основу за експерименталне сценарије и анализу у наредним одељцима.



Слика 3.1: Логичка архитектура експерименталног система

Слика 3.1 приказује логичку архитектуру система. Посебно је важно уочити да је **сервис за извршавање модела ВИ** једина компонента која је предмет хоризонталног скалирања у експериментима, док остале компоненте (генератор оптерећења, сервис за оркестрацију и распоређивач порука) имају фиксну улогу у остваривању стабилне и репродукцибилне поставке.

Логичка организација система

Систем је организован у неколико логички раздвојених компоненти, при чему свака има јасно дефинисану улогу у експерименталној поставци:

1. **Генератор оптерећења** производи контролисане HTTP захтеве према систему и омогућава репродуцибилно тестирање у више профила оптерећења. Његова улога није само да генерише велики број захтева, већ и да их распоређује у фазама које одговарају дефинисаним плановима извршавања (нпр. стабилан раст, нагли скокови).
2. **Сервис за оркестрацију захтева** прихвата улазне захтеве и формира поруке које се уписују у ред. Ова компонента служи као улазна тачка система и као посредни слој који раздваја пријем захтева од обраде, без директног извршавања модела ВИ.
3. **Паспоређивач порука *RabbitMQ*** функционише као асинхрони бафер између пријема и обраде захтева. Ред порука омогућава накупљање посла када интензитет захтева премашује тренутни капацитет обраде, а број порука у реду представља индикатор оптерећења и сигнал за скалирање.
4. **Сервис за извршавање модела ВИ** преузима поруке из реда и извршава модел вештачке интелигенције. Ова компонента представља примарни носилац радног оптерећења и уједно циљни ресурс над којим се примењују механизми хоризонталног аутоскалирања.
5. **Кубернетес окружење** омогућава распоређивање, управљање животним циклусом подова и примену механизма аутоматског скалирања. У овом раду Кубернетес представља извршни и контролни оквир у којем се пореде ХАСП и КЕДА стратегије.
6. **Инфраструктура за праћење и прикупљање метрика** прикупља временске серије потребне за анализу ресурса, реда порука, латенције и броја реплика. У оперативном смислу, ово омогућава корелацију између заостатка у реду, искоришћености ресурса, динамике скалирања и постигнутих перформанси.

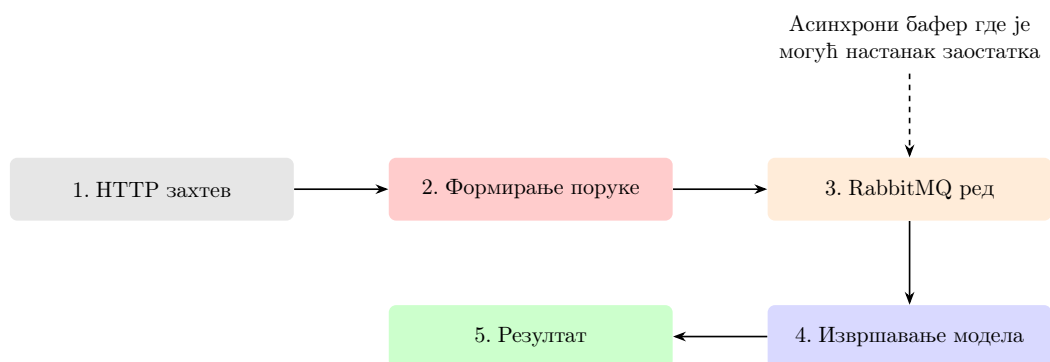
Интерфејси и уговори између компоненти. Комуникација у систему је организована тако да свака компонента има јасно дефинисан улазни и излазни интерфејс. Генератор оптерећења шаље HTTP захтев ка сервису за оркестрацију захтева, који преводи захтев у поруку погодну за асинхрону обраду и уписује је у ред порука. Сервис за извршавање модела ВИ преузима

појединачне поруке из реда, извршава модел и евидентира исход обраде. На овај начин, *јединица посла* у систему одговара једној поруци у реду, што омогућава да се обим накупљеног посла посматра кроз величину реда.

Ток обраде захтева

Обрада захтева у систему реализује се асинхронно и може се описати следећим корацима:

1. генератор оптерећења шаље HTTP захтев ка сервису за оркестрацију захтева;
2. сервис за оркестрацију захтева прихвата захтев и формира поруку;
3. формирана порука се уписује у ред *RabbitMQ*;
4. сервис за извршавање модела ВИ преузима поруку из реда, покреће модел над улазним подацима и добијени резултат прослеђује на даљу обраду;
5. резултат обраде се евидентира или враћа, док се истовремено прикупљају релевантне метрике.



Слика 3.2: Асинхрони ток обраде захтева у систему

Оваква организација омогућава раздвајање брзине долазних захтева од брзине њихове обраде. Уколико је интензитет захтева већи од тренутног капацитета сервиса за извршавање модела, поруке се накупљају у реду, што омогућава јасно посматрање динамике заостатка у систему.

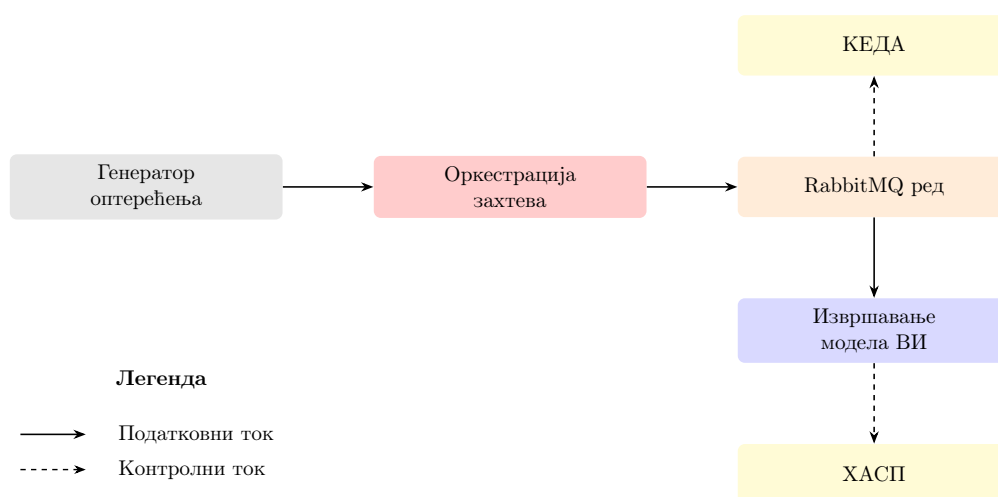
Дефиниција резултата и латенције. У контексту овог рада, резултат представља исход извршавања модела који сервис за извршавање модела генерише над примљеним захтевом (поруком). Латенција која се користи у анализи дефинисана је као време потребно да захтев буде успешно обрађен у сервису за извршавање модела, укључујући потенцијално време чекања у реду порука и време самог извршавања модела. Оваква дефиниција је погодна за асинхрони модел обраде, јер директно повезује накупљање заостатка у реду са смањењем перформанси (нпр. $p95/p99$) у условима повећаног оптерећења.

Податковни и контролни токови

У архитектури се могу разликовати две врсте токова: податковни ток и контролни ток.

Податковни ток представља стварни ток захтева и порука кроз систем. Он почиње на генератору оптерећења, наставља се преко сервиса за оркестрацију захтева, затим кроз RabbitMQ ред порука и завршава се на сервису за извршавање модела ВИ, где се захтеви обрађују.

Контролни ток се односи на механизме који на основу посматраних метрика доносе одлуке о промени броја реплика сервиса за извршавање модела ВИ. У ХАСП сценарију контролни ток се заснива на ресурсним метрикама, а у КЕДА сценарију на спољашњем сигналу.



Слика 3.3: Податковни и контролни токови у архитектури система

Слика 3.3 илуструје раздвајање податковног и контролног тока. Методо-

лошки значај оваквог раздвајања је у томе што омогућава поређење ХАСП и КЕДА механизма над истом апликативном логиком, без измене начина на који се захтеви обрађују: предмет поређења је сигнал и логика скалирања, а не функционалност самог сервиса. Праћење стања система реализовано је кроз инфраструктуру за прикупљање временских серија (Прометеј) и визуелизацију (Графана), о чему ће бити више речено у наставку поглавља.

3.3 Контејнеризација и поставке Кубернетеса

У овом одељку описује се конкретна експериментална поставка. Фокус није на општим дефиницијама компоненти Докера и Кубернетеса (које су представљене у поглављу 2), већ на конкретним артефактима (Докер датотеке и слике) и Кубернетес ресурсима и параметрима који су коришћени да би поређење ХАСП и КЕДА било поновљиво и фер.

Контејнери омогућавају да се апликације извршавају у више примерака, без обзира на окружење, што представља основу за хоризонтално скалирање и поновљива мерења. То значи да се оркестратор и сервис модела испоручују као стандардизоване слике, а Кубернетес их покреће као подове на чворовима кластера. Самим тим се омогућава поређење ХАСП и КЕДА. У анализи ХАСП/КЕДА, контејнеризација омогућава да се:

- **конзистентно** покрећу оркестратор (*Spring Boot*) и модел (*FastAPI*) са истим зависностима у свим срединама;
- **лако** умножавају реплике модела ради хоризонталног скалирања;
- **мере перформансе и изолује потрошња ресурса** по поду, што омогућава фер поређење при различитим профилима оптерећења [6, 17].

Кубернетесови манифести и распоређивање

На основу декларативног модела описаног у одељку 3.1, експериментална поставка је дефинисана скупом Кубернетес манифеста који описују сервисе, њихову међусобну комуникацију и механизме аутоматског скалирања. На тај начин се обезбеђује репродуцибилност (исти артефакти и исти ресурси у свим покретањима) и фер поређење (исти `requests` и `limits` параметри и исти опсег `minReplicas` и `maxReplicas` вредности, уз промену искључиво механизма

и сигнала скалирања) [17, 6]. У Кубернетес окружењу радни чворови представљају извршне машине на којима се подови покрећу преко конфигурисаног *извршног окружења контејнера* (нпр. *containerd* преко *CRI*), док се Докер у оквиру овог рада користи као алат за изградњу и дистрибуцију контејнерских слика.

Поред набројаног, у раду су употребљени следећи типови ресурса, који заједно чине репродуцибилну и контролисану експерименталну поставку:

- ***Deployment*** се користи за распоређивање оркестраторског сервиса и сервиса за извршавање модела. Омогућава управљање животним циклусом подова (креирање, ажурирање, самопоправљање), као и контролисане надоградње (ажурирања) и повратак на претходну верзију. За потребе експеримента, посебно је значајно што *Deployment* представља циљни ресурс над којим се мења број реплика приликом аутоматског скалирања [6];
- ***Service*** (*сервис*) представља мрежну апстракцију која обезбеђује стабилно адресирање и приступ скупу подова који се динамички мења током скалирања. Ово је важно јер клијенти и друге компоненте система не морају бити свесни промена у броју реплика, јер се комуникација остварује преко једне стабилне сервисне адресе, док Кубернетес врши расподелу саобраћаја ка тренутно расположивим подовима [6];
- ***ConfigMap*** (*конфигурационе мапе*) и ***Secret*** (*тајне*) се користе за конфигурационе параметре и осетљиве податке. Конфигурационе мапе садрже неосетљиве конфигурације (нпр. адресе сервиса, називе редова, параметре рада), док тајне садрже креденцијале и поверљиве вредности (нпр. корисничко име и лозинку за *RabbitMQ*). Овај приступ повећава безбедност и олакшава промену конфигурације без промене апликативног кода, што је важно за поновљивост експеримената [6];
- ***Horizontal Pod Autoscaler*** (**HPA**) (*хоризонтални ауто скалер подова, ХАСП*) дефинише правила аутоматског скалирања у сценарију заснованом на праћењу оптерећења процесорске јединице. ХАСП периодично посматра задате метрике и на основу циљне вредности израчунава жељени број реплика за контролисано распоређивање. Овим се реализује скалирање засновано на оптерећењу ресурса [6];

- **ScaledObject** (скалирајући објекти) ресурс се користи у сценарију скалирања заснованог на догађајима. Скалирајући објекат повезује циљни *Deployment* са спољашњим сигналом оптерећења, дефинишући минималан и максималан број реплика и услове под којима се иницира скалирање. У овом раду метрика реда порука у распоређивачу порука се узима као сигнал оптерећења, који је дефинисан у овом ресурсу. На овај начин се реализује скалирање засновано на обиму посла, где број реплика прати количину посла накупљеног у реду [16].

Декларативни приступ распоређивању има методолошки значај у овом раду јер омогућава јасно дефинисање жељеног стања и поновљиво постављање окружења, што је предуслов за контролисано поређење различитих експерименталних конфигурација. Конкретно, коришћењем истих манифеста за апликативне компоненте и истих параметара ресурса као што су лимити, минимални и максимални број реплика, а мењањем искључиво механизма скалирања, обезбеђује се да уочене разлике у резултатима буду последица различитог сигнала и логике скалирања, а не различите инфраструктуре система или верзије апликације [6, 24].

Поставке ресурса и фер услови поређења

Да би поређење механизма ХАСП и КЕДА било фер, у свим експерименталним сценаријима задржани су идентични услови за сервис који извршава модел, при чему се разликовао само сигнал скалирања. Конкретно, контролисани су следећи параметри:

- исти Докер артефакти и иста верзија апликације;
- исте вредности **requests** (захтевани ресурси) и **limits** (ограничења) за процесорску јединицу и меморију;
- исти опсег броја реплика, дефинисан параметрима **minReplicas** и **maxReplicas**, који одређују минимални и максимални број реплика;
- упоредиви параметри скалирања (нпр. **cooldown** и **stabilizationWindow**);
- исти профили оптерећења (исти планови извршавања);
- исти временски интервали за прикупљање и обраду метрика.

Ова контрола је посебно важна зато што је ХАСП осетљив на величину захтеваних ресурса на процесорској јединици, пошто се сигнал искоришћености ресурса посматра у односу на ову величину. Последица је да различите вредности обезбеђених ресурса могу значајно изменити понашање скалирања [5]. Уколико би минимална величина захтеваних ресурса била постављена прениско или превисоко, проценат искоришћености процесорске јединице не би адекватно одражавао стварно оптерећење, што би довело до прераног или закаснелог скалирања. Са друге стране, у сценарију заснованом на дужини реда порука, одлука о скалирању не зависи директно од ресурсних метрика пода, већ од екстерног сигнала који одражава накупљен посао у систему. Због тога је неопходно да се задрже исти лимити ресурса и исти опсег броја реплика, како би се елиминисала могућност да разлике у перформансама потичу од различитих инфраструктурних ограничења, а не од саме логике скалирања. Поред тога, за оба механизма скалирања примењени су упоредиви временски параметри стабилизације и кашњења реакције, чиме се смањује утицај краткотрајних осцилација у оптерећењу (енгл. *thrashing*). На овај начин се обезбеђује да разлике у динамици скалирања произилазе из природе сигнала, а не из различитих конфигурационих поставки. Овим је омогућено да оба механизма буду тестирана у идентичном извршном окружењу, над истом апликативном логиком и под истим профилима оптерећења, што представља предуслов за методолошки валидно и репродуцибилно поређење.

Мрежа, доступност и провере стања

Да би мерења била стабилна и репродуцибилна, за апликативне компоненте је пожељно дефинисати основне оперативне механизме:

- **провере спремности** (енгл. *readiness probe*), како би сервис био укључен у обраду тек након успешног покретања;
- **провере живости** (енгл. *liveness probe*), како би Кубернетес могао аутоматски да рестартује неисправну инстанцу;
- **стабилно сервисно адресирање** преко сервисног ресурса.

Провера спремности. Провера спремности обезбеђује да се нова реплика сервиса не укључи у балансер оптерећења пре него што апликација

успешно иницијализује све зависности (нпр. конекцију ка распоређивачу порука или моделу). Ово је посебно важно у сценаријима скалирања, где се нови подови динамички додају у кластер, јер се спречава слање захтева инстанцама које још увек нису у потпуности спремне за обраду.

Провера живости. Провера живости омогућава детекцију стања у коме апликација формално ради, али је функционално неисправна (нпр. блокирана нит, заглављена обрада). У том случају, Кубернетес аутоматски рестартује под, чиме се минимизује трајање деградираног стања и обезбеђује континуитет експеримента без ручне интервенције.

Стабилно сервисно адресирање. Стабилно сервисно адресирање преко сервисног ресурса обезбеђује да клијенти и друге компоненте комуницирају преко логичке адресе, независно од тренутног броја реплика и њихових динамичких ИП адреса. Ово је кључно у условима хоризонталног скалирања, јер се број подова мења током извршавања тестова, док приступ сервису остаје конзистентан.

Ови механизми не мењају логику аутоскалирања, али имају шум у мерењима који настаје због успутних проблема приликом покретања и рада подова. Конкретно, елиминишу ситуације у којима би привремена недоступност нове реплике или краткотрајни застој у раду довели до вештачког повећања латенције или накупљања порука у реду, што би нарушило валидност поређења различитих стратегија скалирања. На тај начин се обезбеђује да измерене разлике у перформансама одражавају стварно понашање механизма аутоскалирања, а не нестабилности извршног окружења.

Праћење и прикупљање метрике

Метрике коришћене у анализи се прикупљају из више извора:

- **Кубернетесове метрике** - ресурсне метрике (искоришћеност процесорске јединице), број реплика;
- ***RabbitMQ* метрике** - број спремних и прихваћених порука у реду и укупан заостатак;
- **апликативне метрике** - латенција (p95, p99), пропусност, број обрађених захтева.

Прометеј. За централизовано прикупљање и складиштење временских серија коришћен је Прометеј (енгл. *Prometheus*), који периодично преузима метрике са дефинисаних крајњих тачака (енгл. *scrape targets*). Ресурсне метрике Кубернетеса прикупљене су путем одговарајућег механизма метрика унутар кластера, док су апликативне метрике експониране преко ХТТП интерфејса у складу са форматом Прометеја. Метрике распоређивача порука прикупљене су путем *RabbitMQ* експортера, а посматране преко званичног интерфејса за управљање редовима.

Графана. За визуелизацију и оперативно праћење временских серија, коришћена је Графана, која је повезана за Прометејом као извором података. Графана је коришћена за праћење динамике скалирања, корелације између оптерећења процесорске јединице, дужине реда и броја реплика, као и за иницијалну валидацију конзистентности података и утврђивање важности метрика пре њихове даље обраде.

Прикупљене временске серије се усклађују и дохватају периодично, а затим агрегирају у *CSV* датотеке и сумарне извештаје који се користе у поглављу резултата и анализе (поглавље 5). Оваква обрада омогућава упоредиву анализу различитих сигнала скалирања и смањује утицај нерегуларних узорковања или кашњења у прикупљању података.

3.4 Стратегије скалирања

Циљ поређења и експериментални фактори

Основни циљ експерименталног дела рада је поређење две стратегије хоризонталног аутоматског скалирања сервиса за извршавање модела у идентичним условима, уз контролисан улазни профил оптерећења и конзистентну инфраструктурну поставку:

1. **ХАСП** - скалирање на основу искоришћености процесорске јединице као ресурсне метрике [5];
2. **КЕДА** - скалирање на основу метрике реда порука у *RabbitMQ* систему, као спољашњег (заснованог на догађају) сигнала [16].

Поређење је организовано тако да се у оба случаја скалира исти циљни ресурс (*Deployment* ресурс сервиса за извршавање модела), док су апликативна

логика и инфраструктурни услови фиксирани. На тај начин се уочене разлике у перформансама и динамици скалирања могу преписати избору сигнала и механизма скалирања, а не варијацијама у имплементацији или окружењу [16, 5].

Експериментални фактори (независне променљиве). Експериментални фактори који се мењају су:

- **механизам скалирања (ХАСП/КЕДА);**
- **план оптерећења;**
 - *план са најлим скоковима оптерећења;*
 - *план са постепеним порастом оптерећења.*

На тај начин се добијају четири основне експерименталне комбинације:

1. КЕДА и план са наглим скоком;
2. ХАСП и план са наглим скоком;
3. КЕДА и план са постепеним порастом;
4. ХАСП и план са постепеним порастом.

Константне променљиве. Да би поређење било методолошки исправно, у свим комбинацијама се задржавају идентични услови који директно утичу на понашање аутоскалера: (1) исти контејнерски артефакти и верзије компоненти (сервис за извршавање модела, оркестрацију и RabbitMQ), (2) исти ресурси (процесор, меморија) додељени сервису за извршавање модела кроз број захтева и лимит, (3) исти минимални и максимални број реплика, (4) иста конфигурација реда порука и обраде (јединица посла = једна порука). Овим се обезбеђује да је оптерећење по реплици упоредиво, па је и динамика скалирања интерпретабилна у истом контексту [16, 5].

Зависне променљиве. Иако се детаљан опис метрика и протокола даје у наредним потпоглављима, у оквиру овог поређења се посматрају: (1) динамика броја реплика током теста (брзина подизања и спуштања инстанци), (2) метрике реда порука (кретање броја порука које су у реду), (3) перформансе обраде (латенција $p95/p99$ и пропусност) и (4) ресурсне метрике (процесорска јединица) као сигнал у ХАСП случају [16, 5].

Конфигурација ХАСП-а у експерименту

У експерименталној поставци полазна циљна искоришћеност процесорске јединице за ХАСП постављена је на 60%, при чему се сама вредност прага не третира као фиксни избор, већ као параметар чији се утицај испитује. Полазна вредност од 60% представља уобичајену конзервативну поставку у продукционим Кубернетес окружењима, где се праг између 50% и 70% препоручује као баланс између реактивности и стабилности [5].

Како би се проверило да закључци поређења не зависе од конкретне вредности прага, поред полазног прага од 60% испитан је и праг од 70%. Та два нивоа чине анализу осетљивости ХАСП прага, чији се резултати приказују у поглављу 5 (потпоглавље 5.3), и омогућавају да се ефекат врсте сигнала (искоришћеност процесорске јединице) разликује од ефекта његове конкретне граничне вредности.

Конфигурација КЕДА у експерименту

У КЕДА сценарију дефинисан је скалирајући објекат (енгл. *ScaledObject*) са следећим параметрима: циљна вредност реда по реплици од 100 порука, интервал дохватања од 5 секунди и време смиривања од 30 секунди.

Циљна вредност од 100 порука по реплици изабрана је на основу пилот тестова у којима је посматрано понашање система при различитим вредностима. Вредност од 100 порука показала се као баланс између реактивности (довољно рано покретање скалирања) и стабилности (избегавање пречестих осцилација броја реплика). Како се сама вредност прага не третира као фиксни избор, њен утицај систематски је испитан кроз анализу осетљивости КЕДА прага (вредности 60, 80 и 100 порука по реплици), чији се резултати приказују у поглављу 5 (потпоглавље 5.3).

Интервал дохватања (енгл. *polling interval*) од 5 секунди одговара интервалу прикупљања метрика у целом систему, чиме се обезбеђује конзистентност временске резолуције мерења. Време смиривања (енгл. *cooldown period*) од 30 секунди представља умерену вредност која спречава превремено смањивање броја реплика након краткотрајних падова оптерећења, а истовремено је упоредива са прозором стабилизације конфигурисаним за ХАСП, чиме се обезбеђује фер поређење динамике обе стратегије.

Планови оптерећења

За процену понашања аутоскалера, дефинисана су два плана оптерећења (плана извршавања), сваки план моделован као низ фаза, при чему је свака фаза одређена следећим параметрима:

1. бројем порука које се шаљу,
2. трајањем фазе,
3. ефективном фреквенцијом слања (поруке у секунди)

План са наглим оптерећењем. Овај план моделује сценарио у коме систем трпи изненадне промене интензитета долазних захтева. Карактерише га смена фаза ниског и високог оптерећења, са релативно општрим прелазима између фаза. Циљ овог плана је испитивање реактивности механизма скалирања, односно брзина спуштања и подизања инстанци, појаве заостатка и прекорачења дефинисаног нивоа услуге (енгл. *service-level agreement, SLA*).

План са постепеним оптерећењем. Овај план моделује стабилнији сценарио у коме оптерећење расте и опада постепено. Циљ је процена понашања аутоскалера у условима без изражених скокова, односно мерење да ли механизам одржава стабилнији однос између латенције, броја реплика и утрешка ресурса.

Формална спецификација плана. Тачне фазе и параметри (број порука и трајање фазе) дати су у JSON датотекама плана извршавања:

- `plan_steady.json` - план са постепеним порастом оптерећења.
- `plan_spike.json` - план са наглим скоковима оптерећења;

Ради прегледности, фазе оба плана су сумиране у табелама 3.1 и 3.2, док је на слици 3.4 дат њихов кумулативни приказ у времену.

На основу табела 3.1 и 3.2, као и слике 3.4, може се уочити да два плана оптерећења представљају два суштински различита профила рада система. План са постепеним оптерећењем има дужи временски период извршавања и већи укупан број захтева, при чему је пораст интензитета распоређен кроз више узастопних фаза са задржавањем на средњим и високим нивоима оптерећења. Насупрот томе, план са наглим оптерећењем краћег је трајања и садржи изражене, краткотрајне скокове у броју послатих порука, са краћим

Фаза	Бр. порука	$t(s)$
warmup	400	300
baseline_1	400	300
baseline_2	600	300
ramp_to_mid	720	240
hold_mid	900	360
ramp_to_high	900	180
hold_high	1200	420
push_to_max	1200	180
hold_max	1600	600
recover_1	500	420
mini_spike_1	500	90
steady_1	400	300
mini_spike_2	500	90
cooldown	190	360
idle_final	0	600
Σ	10100	4740

Табела 3.1: Фазе плана са постепеним порастом оптерећења

Фаза	Бр. порука	$t(s)$
warmup	400	300
baseline_1	400	300
baseline_2	600	300
burst_low	600	120
recover_1	300	300
burst_mid	1000	120
recover_2	300	360
burst_high	1500	120
hold_peak	800	600
mini_spike_1	600	30
steady_1	400	300
mini_spike_2	600	30
cooldown	300	360
idle_final	0	600
Σ	7800	3840

Табела 3.2: Фазе плана са наглим скоковима оптерећења

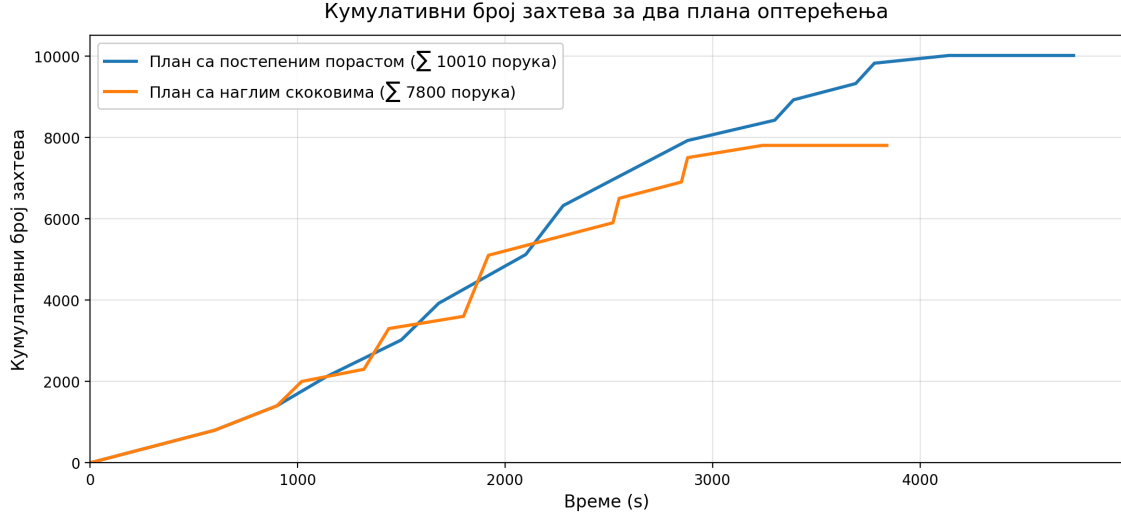
периодима опоравка између њих. Таква разлика је видљива и на графику кумулативног броја захтева: код постепеног плана крива расте уједначеније и континуираније, док код плана са наглим оптерећењем постоје стрмији сегменти који одговарају фазама интензивног прилива захтева. На тај начин, први план је погодан за анализу стабилности и ефикасности аутоскалера у условима постепене промене оптерећења, док други план омогућава испитивање реактивности механизма скалирања, односно његове способности да у кратком времену одговори на нагло повећање обима посла.

Овако дефинисани планови оптерећења представљају основу за све експерименталне комбинације разматране у раду и омогућавају да се понашање различитих механизма аутоматског скалирања анализира у условима и постепене и нагле промене обима посла.

3.5 Метрике и критеријуми евалуације

Поређење механизма ХАСП и КЕДА заснива се на више група метрика које описују различите аспекте понашања система.

Метрике реда порука. Прва посматрана метрика је метрика реда по-



Слика 3.4: Поређење кумулативног броја захтева за постепени план и план са наглим скоковима

рука која посматра нагомилани заостатак у реду. Нека је:

- $Q_{ready}(t)$ - број порука спремних за обраду,
- $Q_{unacked}(t)$ - број преузетих порука које још увек нису потврђене.

Укупни заостатак дефинишемо на следећи начин:

$$Q(t) = Q_{ready}(t) + Q_{unacked}(t) \quad (3.1)$$

Посматране метрике су:

- просечан заостатак $\bar{Q}$;
- максимални заостатак Q_{max} ;
- интеграл заостатка AUC_Q

$$AUC_Q = \int_{t_0}^{t_1} Q(t) dt. \quad (3.2)$$

Интеграл заостатка квантификује кумулативно оптерећење реда током експеримента, па је погодан за упоредну оцену укупне ефикасности аутоskalера, јер агрегира ефекат и трајање накупљања посла у реду [16]. Мања

вредност AUC_Q указује да је систем провео мање времена са великим заостатком, односно да је брже враћен у стање уравнотеженог протока.

Пропусност. Поред метрика реда порука, у анализи се прати и пропусност система, коју означавамо са $\mu_{\text{total}}(t)$. Пропусност се дефинише као број успешно обрађених порука у јединици времена, мерен на излазу сервиса за извршавање модела ВИ, односно стопа по којој систем успешно завршава обраду захтева из реда. Посматрају се и просечна вредност и максимална вредност пропусности током трајања експеримента.

Долазна и сервисна стопа. Ради анализе тренутног односа понуде и тражње у систему, уводе се две изведене стопе.

- *Долазна стопа* $\lambda(t)$ представља тренутни интензитет пристизања порука у систем, мерен у порукама по секунди.
- *Сервисна стопа* $\mu_{\text{total}}(t)$ представља тренутни интензитет обрађених порука свих активних реплика заједно.

Обе стопе израчунавају се као временски прозорски просек над усклађеним временским серијама метрика.

Однос ових стопа одређује тренутно стање реда:

- ако је $\lambda(t) > \mu_{\text{total}}(t)$, систем прима поруке брже него их обрађује, односно ред расте;
- ако је $\mu_{\text{total}}(t) \geq \lambda(t)$, систем ради у уравнотеженом режиму, односно ред се смањује или остаје стабилан.

Ефикасан аутоskalер треба да минимизује укупно трајање режима $\lambda > \mu$, јер управо у тим периодима заостатак расте, а латенција деградира. Разлика $\lambda(t) - \mu_{\text{total}}(t)$ приказана је графички у поглављу анализе резултата (поглавље 5) као непосредан визуелни показатељ тренутног стања система.

Метрике латенције и нивоа услуге. У раду се посматрају временски перцентили латенције ($p95$ и $p99$), који описују понашање горњег репа расподеле латенције и омогућавају анализу екстремнијих кашњења. Ове метрике су посебно значајне у системима са редовима и променљивим временом обраде, јер просечна вредност сама по себи није довољна да опише деградацију квалитета услуге у условима оптерећења.

Дефиниција латенције. У овом раду латенција се оперативно дефинише као време од пријема захтева у систему до завршетка његове обраде и

евидентирања резултата. Оваква дефиниција је у складу са општим појмом *времена одговора* (енгл. *response time*) у системима са редовима, који обухвата време проведено у систему, односно време чекања и време услуге [7]. У конкретної експерименталној поставци, то значи да латенција обухвата и потенцијално задржавање захтева у реду порука и време самог извршавања модела.

За сваки експеримент израчунавају се:

- просечна $p95$ и $p99$ вредност;
- максимална $p95$ и $p99$ вредност;
- време прекорачења дефинисаног нивоа услуге (енгл. *service-level agreement breach time*), нпр. за услов $p95 > 2s$ и $p99 > 2s$.

Време прекорачења нивоа услуге. У овом раду, прекорачење дефинисаног нивоа услуге дефинише се као укупно време током којег посматрани перцентил латенције премашује задати праг τ унутар посматраног интеграла. Овај концепт је сродан приступима заснованим на преступним интервалима (енгл. *violated intervals*), који се користе у формализацији метрика нивоа услуга [18]. Формално, време прекорачења може се записати као:

$$T_{SLA}(\tau) = \int_{t_0}^{t_1} \mathbb{I}\{p95(t) > \tau\} dt, \quad (3.3)$$

при чему индикаторска функција $\mathbb{I}\{\cdot\}$ има вредност 1 када је услов испуњен, а 0 у супротном. Аналогна дефиниција може се применити и на $p99$ латенцију.

Избор прага. За дефиницију прекорачења нивоа услуге коришћен је праг $\tau = 2s$. Овај избор мотивисан је карактеристикама конкретного радног оптерећења: у експерименталној поставци просечно време извршавања модела по захтеву креће се од неколико стотина милисекунди до неколико секунди, у зависности од тренутног оптерећења. Праг од $2s$ стога раздваја прихватљиву латенцију, у којој систем обрађује захтеве без значајне акумулације у реду, од деградираног режима рада, у коме заостатак у реду почиње да доминира укупном латенцијом. Такав начин дефинисања нивоа услуге конзистентан је са праксом у којој се он изражава преко перцентиала латенције уз задати праг, при чему конкретна вредност прага зависи од природе апликације и очекивања корисника [18, 11].

Важно је нагласити да конкретна вредност прага не утиче на квалитативне закључке поређења. Промена прага мења апсолутне вредности T_{SLA} за оба механизма, али задржава релативни однос између њих, јер КЕДА конзистентно остварује мање латенције у свим фазама оптерећења. Мања вредност времена прекорачења указује да је систем краће боравио у стању деградираног квалитета услуге.

Ове метрике су кључне јер директно описују кориснички доживљај услуге, а посебно деградацију у случајевима када мали проценат захтева трпи не-сразмерно велика кашњења. Управо зато се, поред просечних вредности, у анализи посматрају и крајњи перцентили и укупно време проведено изнад задатог прага.

Метрика реактивности и стабилизације. Поред стандардних метрика перформанси и искоришћености ресурса, у анализи се користе и метрике одзива система, које описују брзину реакције механизма аутоматског скалирања и прелазак система у стабилно стање. У ове метрике спадају:

- **време до ефективног повећања капацитета**, односно време од иницирања скалирања до тренутка када систем остварује одржив пораст ефективног капацитета обраде;
- **време опоравка**, односно време потребно да се систем, након настанка заостатка, врати испод унапред дефинисаног прага заостатка;
- **време смиривања**, односно време потребно да се систем након пика оптерећења стабилизује без значајнијих осцилација у величини реда или броју реплика.

Ове метрике су посебно информативне у сценаријима са наглим скоковима оптерећења, јер омогућавају да се, поред вредности латенције, пропусности и броја реплика, анализира и сама динамика прилагодљивости система.

Композитни индекс ефикасности аутоскалирања

Ради синтетичког поређења стратегија скалирања, у раду је уведен **ком-позитни индекс ефикасности аутоскалирања** (енгл. *Autoscaling Efficiency Index, AEI*). Овај индекс није стандардизована метрика из литературе, већ је дефинисан синтетички показатељ уведен за потребе овог рада, са циљем да у једној величини обједини кључне аспекте понашања аутоскалера.

Индекс комбинује три аспекта:

1. кумулативни заостатак,
2. време прекорачења нивоа услуге,
3. просечан број реплика.

У облику који је коришћен у анализи, индекс је дефинисан као:

$$AEI = \frac{1}{(AUC_Q + 1) \cdot (T_{SLA} + 1) \cdot \bar{N}} \quad (3.4)$$

где су:

- AUC_Q - интеграл заостатка (3.2),
- T_{SLA} - укупно време прекорачења услова изабраног нивоа услуге (3.3),
- $\bar{N}$ - просечан број реплика током експеримента.

Додавање константе +1 спречава дељење нулом у случају идеалног понашања. Већа вредност индекса указује на повољнији компромис између квалитета услуге и утrophка ресурса. Избор производа у имениоцу има интуитивно тумачење: индекс је највећи само у случајевима када су истовремено мали кумулативни заостатак, време нарушавања нивоа услуге и просечан број реплика. На тај начин се избегава да добар резултат у једној димензији прикрије изразито лош резултат у другој. Пошто индекс не представља универзалну метрику применљиву ван контекста овог експеримента, он се у раду користи искључиво као допунски индикатор уз упоредну анализу сценарија унутар истог експерименталног оквира, уз појединачно тумачење сваке компоненте од које је састављен.

Идеја сажимања више аспеката квалитета услуге у један нормализован показатељ није нова. У индустрији је широко прихваћен индекс перформанси апликације (*Apdex*), који време одзива у односу на задати праг своди на јединствену вредност у опсегу од 0 до 1.¹ Композитни индекс ефикасности аутошкалирања је инспирисан управо тим приступом, али се од *Apdex*-а разликује по томе што не сажима само латенцију, већ обједињује квалитет услуге и утrophак ресурса (кумулативни заостатак, прекорачење нивоа услуге

¹Apdex Alliance, *Apdex Technical Specification*, верзија 1.1, 2007. Доступно на: <https://www.apdex.org>. Видети и [3].

и просечан број реплика) у јединствену меру погодну за упоредну анализу стратегија скалирања.

Обрада података и излазне датотеке

Након прикупљања података, временске серије се временски усклађују и свде на јединствен интервал узорковања. Овај корак је неопходан јер се метрике прикупљају из више извора (Кубернетес, распоређивач порука, апликативне метрике) и не морају бити иницијално узорковане у истим временским тренуцима. Захваљујући овом приступу, омогућава се упоређивање различитих метрика у истом временском оквиру и да се на поуздан начин израчунавају агрегатне и изведене величине.

Из тако припремљених података формирају се следећи излазни артефакти:

- **merged_*.csv** - усклађене временске серије свих релевантних метрика, које представљају основу за анализу динамике система током експеримента;
- **per_phase_*.csv** - агрегати израчунати по појединачним фазама плана оптерећења, што омогућава фазно поређење понашања система;
- **scaling_efficiency_*.csv** - показатељи ефикасности скалирања по нивоу оптерећења и броју реплика;
- **summary_*.md** - текстуални сумарни извештај са кључним метрикама и индикаторима добијеним за посматрани сценарио.

Оваква организација омогућава јасну поделу између сирових и полуобрађених временских серија, агрегираних података по фазама и крајњих аналитичких показатеља. На тај начин се поједностављује репродукција резултата, а уједно се обезбеђује да табеле, графици и закључци у поглављу анализе резултата буду изведени из конзистентно припремљених података.

3.6 Експериментални протокол и ограничења

Да би поређење механизма аутоматског скалирања било методолошки исправно и поновљиво, сви експерименти су извођени по унапред дефинисаном протоколу. Основна јединица експеримента представља једно изврша-

вање задатог плана оптерећења над конкретном стратегијом скалирања, при чему се током целог извршавања прикупљају ресурсне, редне и апликативне метрике.

Сваки експеримент започиње фазом почетног загревања система (енгл. *warmup*), чија је улога да се компоненте система доведу у радно стање пре почетка главног оптерећења. Ова фаза је део самог плана оптерећења и омогућава да мерења у каснијим фазама не буду доминантно условљена иницијалним покретањем апликације, успостављањем конекција или иницијалним попуњавањем реда порука.

Након фазе стабилизације, извршава се главни део сценарија, који може одговарати плану са постепеним оптерећењем или плану са наглим скоковима оптерећења. Током овог периода систем је изложен контролисаном приливу захтева, док се истовремено бележе промене у броју реплика, величини реда порука, латенцији, пропусности и искоришћености ресурса.

По завршетку главних фаза оптерећења, планови садрже и фазе смиривања и опоравка, које су значајне за посматрање механизма скалирања при смањењу оптерећења. Овај део експеримента омогућава анализу времена опоравка, времена смиривања и понашања механизма при смањивању броја реплика.

Прикупљене временске серије се након извршавања временски усклађују и свде на јединствени фиксни интервал узорковања. Над тако припремљеним подацима израчунавају се агрегатне и изведене метрике које се користе у поглављу резултата и анализе (поглавље 5). На овај начин се обезбеђује да све експерименталне комбинације буду анализиране истим поступком обраде података.

Основне експерименталне комбинације добијају се укрштањем два механизма скалирања (КЕДА, ХАСП) и два плана оптерећења (план са постепеним порастом и план са наглим скоковима). За сваку комбинацију примењују се исти услови окружења, исти ресурси по реплици и исти поступак мерења, чиме се обезбеђује фер поређење добијених резултата.

Поновљивост и контрола окружења

Да би резултати били поновљиви, пре сваког покретања експеримента Кубернетес кластер је доведен у идентично почетно стање: све раније постављени ресурси (подови, распоређивачи, аутоskalери) су обрисани, ред

порука је испражњен, а инфраструктурне компоненте (Прометеј, Графана, RabbitMQ) су рестартоване. На тај начин је елиминисан утицај претходних покретања на мерења.

Поновљивост резултата обезбеђена је контролисаним условима експеримента: детерминистички планови оптерећења са фиксним фазама и интензитетима, идентични параметри окружења за све сценарије и стабилност локалног једночворног кластера ограничавају изворе стохастичности на интерне факторе оперативног система и Кубернетес планера. Као додатна верификација конзистентности уочених образаца, анализа осетљивости прагова ХАСП-а и КЕДА-а (потпоглавље 5.3) потврђује да квалитативни закључци остају важећи при промени параметара.

Ограничења методологије

Иако је експериментална поставка осмишљена тако да омогући контролисано и фер поређење различитих механизма аутоматског скалирања, потребно је имати у виду и неколико ограничења методологије.

Експерименти су засновани на синтетички дефинисаним плановима оптерећења. Иако су ти планови конструисани тако да репрезентују постепене и нагле промене обима посла, они не могу у потпуности обухватити сву сложеност оптерећења која се јављају у реалним продукционим окружењима. Због тога добијене резултате треба тумачити као резултате у контролисаним експерименталним условима, а не као универзалне вредности применљиве у свим окружењима.

ХАСП механизам је у овом раду анализиран пре свега на основу искоришћености процесорске јединице као сигнала скалирања. Иако је такав избор у складу са праксом у Кубернетес окружењима, он не мора да најверније одражава стварни обим накупљеног посла у систему, посебно у асинхроним архитектурама са редом порука. Слично томе, ни дужина реда порука као сигнал у КЕДА сценарију не представља потпун опис понашања система, већ један аспект оптерећења.

Добијене метрике зависе од интервала прикупљања и накнадне обраде временских серија. Иако су сви сценарији обрађени на идентичан начин, сам избор интервала узорковања и поступка агрегације утиче на исход посматране динамике скалирања и може ублажити или нагласити краткотрајне промене.

Композитни индекс ефикасности аутоскалирања је уведен као синтетички показатељ за потребе овог рада и не представља стандардизовану метрику из литературе. Његова улога је допунска, односно да омогући сажето упоредно тумачење из више аспеката понашања аутоскалера унутар истог експерименталног оквира. Из тог разлога, индекс не треба тумачити изоловано, већ заједно са појединачним метрикама од којих је изведен.

На крају, резултати добијени у овом раду односе се на конкретну апликативну архитектуру, конкретан сервис за извршавање модела и конкретну конфигурацију окружења. Иако методологија омогућава релевантно поређење унутар ове поставке, преношење закључака на друге системе захтева додатну експерименталну потврду.

Експериментално окружење

Сви експерименти у овом раду изведени су у контролисаном локалном Кубернетес окружењу, постављеном употребом алата Миникјуб (енгл. *Minikube*). Пошто перформансе и динамика аутоматског скалирања зависе од карактеристика извршног окружења, у табели 3.3 дати су кључни параметри хардверске и софтверске поставке, као и основни конфигурациони параметри сервиса за извршавање модела и механизма скалирања. На тај начин се обезбеђује прецизнија репродуцибилност експеримената и јаснији контекст за тумачење добијених резултата.

Параметар	Вредност
Оперативни систем	macOS 15.4.1
Процесор	Apple M4 Pro
Радна меморија	48 GB
Верзија Докера	24.0.6
Верзија Миникјуба	1.35.0
Број Миникјуб чворова	1
Број процесора додељен Миникјубу	10
Меморија додељена Миникјубу	24 GB
Сервис за извршавање модела - <code>cru:requests</code>	1000m
Сервис за извршавање модела - <code>cru:limit</code>	2000m
Минимални број реплика	1
Максимални број реплика	8
ХАСП - циљна искоришћеност проц. јед.	60%
КЕДА - праг реда по реплици	100
КЕДА - интервал дохватања порука	5s
КЕДА - време смиривања	30s
Интервал прикупљања метрика	5s
Интервал поновног узорковања	5s
Број понављања по сценарију	5
<code>prefetch_count</code> потрошача	5

Табела 3.3: Параметри експерименталне поставке

Глава 4

Имплементација

У претходном поглављу описана је методологија рада, избор алата и технологије, архитектура система, стратегије скалирања и критеријуми евалуације. У овом поглављу представљена је конкретна реализација система, односно начин на који су методолошке одлуке спроведене у изворном коду, конфигурацијама и инфраструктурним артефактима. Фокус није на поновном објашњавању концепата, већ на имплементационим детаљима који су потребни за разумевање и репродукцију експерименталне поставке.

Изворни код је јавно доступан у *GitHub* репозиторијуму¹ и организован је као моно-репозиторијум, чиме је омогућено заједничко верзионисање свих компоненти система.

4.1 Структура пројекта

Репозиторијум је организован тако да одржава логичку поделу система описану у одељку 3.2. Свака апликативна компонента, скуп инфраструктурних манифеста и помоћне скрипте за анализу налазе се у засебним директоријумима на коренском нивоу пројекта. Организација је следећа:

```
master-rad/  
|-- backend/                # Servis za orkestraciju  
|   |-- src/main/java/      # Izvorni kod  
|   |-- src/main/resources/ # Konfiguracije  
|   |-- build.gradle         # Gradle datoteka
```

¹<https://github.com/AlexJakovljevic/master-rad>

```
|  '-- Dockerfile                # Docker datoteka
|-- ai-model-service-fast-api/    # Servis za inferenciju
|  |-- app/                      # Izvorni kod
|  |-- requirements.txt          # Biblioteke
|  '-- Dockerfile                # Docker Datoteka
|-- k8s-manifests/               # Kubernetes manifesti
|  |-- hpa/                      # Rasporedjivanje i HASP
|  |-- keda/                     # RabbitMQ i KEDA
|  '-- monitoring/               # Prometej i Grafana
|-- test/                        # Testovi opterecenja i analiza
|  '-- analysis/                 # Skripte za analizu metrika
'-- skaffold.yaml                # Automatizacija rasporedjivanja
```

Структура 4.1: Репозиторијум

Оваква организација одражава принципе микросервисне архитектуре, јер сваки сервис има сопствени директоријум са изворним кодом, конфигурацијом, дефиницијом зависности и Докер датотеком. Кубернетес манифести су издвојени у засебан директоријум (`k8s-manifests/`) са поддиректоријумима који одговарају различитим сценаријима скалирања, чиме се олакшава управљање конфигурацијама и прелазак између експерименталних поставки. Скрипте за анализу резултата налазе се у `test/analysis/` директоријуму и нису повезане са апликативном логиком, али су саставни део експерименталног тока рада описаног у потпоглављу 3.6.

Сервис за оркестрацију захтева. Директоријум `backend/` садржи *Spring Boot* апликацију написану у програмском језику Јава (верзија 21). Структура директоријума прати конвенције *Spring Boot* оквира:

```
backend/src/main/java/master/backend/
|-- controller/
|  |-- AssistantController.java      # IF ulaznog kontrolera
|  '-- impl/
|     '-- AssistantControllerImpl.java # Impl. ulaznog kontrolera
|-- dto/
|  |-- QuestionAsyncResponse.java    # Pitanje
|  '-- AsyncResultRequest.java       # Povratni rezultat
|-- model/
|  |-- ModelRequest.java             # Entitet
```

```
|  '-- ModelRequestStatus.java          # Statusi zahteva
|-- repository/
|  '-- ModelRequestRepository.java      # Repozitorijum
|-- service/
|  |-- AssistantService.java           # IF za ulazni servis
|  |-- AsyncRequestService.java        # IF za servis za asinh. odg
|  .
|  '-- impl/                           # Implementacije interfejsa
|      |-- AssistantServiceImpl.java
|      '-- AsyncRequestServiceImpl.java
'-- metrics/
    |-- AsyncMetrics.java              # Prometejeve metrike
    '-- AsyncRequestTimingStore.java   # Pomocna komponente
```

Структура 4.2: Сервис за оркестрацију

Апликација је организована по слојевитој архитектури тако да контролер прима ХТТП захтеве и делегира их сервисном слоју, сервисни слој координише пословну логику (објављивање порука у ред, управљање статусима захтева), док слој приступа подацима (репозиторијум) обезбеђује персистенцију у бази података. Пакет `metrics/` садржи класе одговорне за излагање Прометејевих метрика које се користе у анализи резултата. Објекти за пренос података (енгл. *DTO*) су реализовани као Јава `record` типови, што обезбеђује имутабилност и сажету дефиницију.

Сервис за извршавање модела ВИ. Директоријум `ai-model-service-fast-api/` садржи FastAPI апликацију написану у програмском језику Пајтон (верзија 3.12). Апликација је модуларно организована:

```
ai-model-service-fast-api/app/
|-- main.py          # FastAPI ulazna tacka
|-- config.py        # Konfiguracioni fajl
|-- models.py        # Modeli podataka
|-- inference.py     # Inferencija
|-- queue_worker.py  # RabbitMQ postrosac
|-- backend.py       # HTTP klijent
|-- chunks.py        # Kreator baze znanja
|-- metrics.py       # Prometejeve metrike
'-- data/
```

```
'-- document_chunks.json # Podaci
```

Структура 4.3: Сервис за извршавање модела

За разлику од монолитног приступа у којем би целокупна логика била садржана у једној датотеци, модуларна организација омогућава јасно раздвајање одговорности тако да је конфигурација издвојена у `config.py`, логика извршавања модела у `inference.py`, комуникација са распоређивачем порука у `queue_worker.py`, а повратни позив ка сервису за оркестрацију у `backend.py`. Датотека `main.py` служи искључиво за иницијализацију апликације, регистрацију крајњих тачака и управљање животним циклусом позадинских процеса. Ова организација олакшава независно тестирање и одржавање појединачних компоненти.

Кубернетес манифести. Директоријум `k8s-manifests/` садржи декларативне спецификације Кубернетесових ресурса, груписане по функционалности:

```
k8s-manifests/
|-- hpa/
|   |-- backend-service-deployment.yml      # Deployment i Service
|   |-- ai-model-service-deployment.yml      # Deployment i Service
|   '-- ai-model-service-hpa.yml             # HASP manifest
|-- keda/
|   |-- rabbitmq.yml                         # RabbitMQ StatefulSet
|   '-- rabbitmq-keda-scaledobject.yml        # KEDA ScaledObject
'-- monitoring/
    |-- namespace.yml                       # Monitoring namespace
    |-- prometheus-rbac.yml                  # Prometej RBAC
    |-- prometheus-config.yml                # Prometej konfiguracija
    |-- prometheus.yml                       # Prometej Deployment
    '-- grafana.yml                          # Grafana Deployment
```

Структура 4.4: Кубернетес манифести

Раздвајање манифеста у `hpa/` и `keda/` поддиректоријуме одговара потреби за лаким прелазом између два експериментална сценарија. Манифести за апликативне компоненте (*Deployment* и *Service*) налазе се у `hpa/` директоријуму и заједнички су за оба сценарија, док се разлика остварује приме-

ном додатних манифеста за ХАСП (`ai-model-service-hpa.yml`) или КЕДА (`rabbitmq-keda-scaledobject.yml`). Овакав приступ одговара принципу описаном у потпоглављу 3.3, где је наглашено да се мења искључиво механизам и сигнал скалирања, док сви остали параметри остају непромењени.

Аутоматизација изградње. Датотека `skaffold.yaml` у коренском директоријуму дефинише конфигурацију за алат *Skaffold*, који аутоматизује изградњу Докер слика и примену Кубернетес манифеста. Конфигурација садржи два профила, `hpa` и `keda`, који одговарају различитим експерименталним сценаријима. Ови профили се разликују само по скупу манифеста који се примењују, чиме се обезбеђује једноставан и поновљив прелазак између поставки.

Скрипте за анализу. Директоријум `test/analysis/` садржи Пајтон скрипту `prom_analysis.py` за повлачење временских серија из Прометеја и генерисање излазних артефаката описаних у потпоглављу 3.5, то јест усклађених временских серија, агрегата по фазама, показатеља ефикасности скалирања и интерактивних графикана. Скрипта прима параметре сценарија, временског прозора и излазног директоријума, чиме се обезбеђује конзистентна обрада за све експерименталне комбинације.

4.2 Сервис за оркестрацију захтева

Сервис за оркестрацију захтева реализован је као *Spring Boot* апликација (верзија 3.5.10) у програмском језику Јава 21, чија је улога да прихвати корисничке захтеве, координише асинхрону обраду путем реда порука и одржава стање захтева у бази података. У овом потпоглављу описана је имплементација кључних компоненти: крајње тачке АПИ-ја, механизма објављивања порука, модела података, инструментације метрика и конфигурације зависности.

АПИ и крајње тачке. Контролер `AssistantControllerImpl` представља централну улазну тачку сервиса и излаже три крајње тачке под путањом `/api/v1/assistant`. Реализован је као *Spring REST* контролер са аноматацијом `@RestController`, при чему је интерфејс `AssistantController` издвојен у засебну датотеку ради јасног раздвајања уговора од имплементације.

Крајња тачка `POST /askAsync` представља примарни улаз за асинхрону обраду, који се користи у свим експерименталним сценаријима. При пријему

захтева сервис извршава два корака: (1) објављује поруку у ред порука путем класе `AsyncQuestionPublisher` и (2) креира запис са статусом `PENDING` у бази података. Клијенту се одмах враћа јединствени идентификатор захтева (`requestId`), генерисан као `UUID`, без чекања на завршетак обраде. Секвенца корака у контролеру изгледа на следећи начин:

```
@PostMapping("/askAsync")
public ResponseEntity<QuestionAsyncResponse> askAsync(
    @RequestBody QuestionRequest request) {
    QuestionAsyncResponse response = assistantService.
        askAsync(request);
    asyncRequestService.createPendingRequest(
        response.requestId(), request.question());
    return ResponseEntity.ok(response);
}
```

Код 4.5: Крајња тачка `/askAsync`

Крајња тачка `GET /async/{requestId}` омогућава клијенту да преузме резултат обраде. Контролер врши претрагу записа у бази података на основу идентификатора захтева и враћа одговарајући HTTP статус: `200` са одговором уколико запис постоји; `404` уколико запис није пронађен.

Крајња тачка `POST /async/result` служи као повратни интерфејс (енгл. *callback*) који позива сервис за извршавање модела по завршетку обраде поруке. Овај интерфејс представља унутрашњу комуникациону тачку система, а не јавни АПИ. Приликом пријема повратног позива, контролер ажурира запис у бази на статус `COMPLETED`, инкрементира бројач завршених захтева и, уколико постоји забележено време објављивања, израчунава и бележи *end-to-end* латенцију. Мерење латенције је реализовано на следећи начин:

```
asyncMetrics.asyncCompleted.increment();
Instant publishedAt = timingStore.removePublishedAt(
    request.requestId());
if (publishedAt != null) {
    Duration d = Duration.between(publishedAt, Instant.now());
    if (!d.isNegative()) {
        asyncMetrics.asyncEndToEnd.record(d);
    }
}
```

```
}  
}
```

Код 4.6: Мерење латенције након доласка одговора на `/async/result`

Овај механизам омогућава мерење укупног времена обраде од тренутка објављивања поруке до пријема резултата, што представља кључну метрику у анализи резултата (потпоглавље 3.5).

Објављивање порука у ред. Интеграција са *RabbitMQ* распоређивачем порука реализована је кроз три класе које чине слој за објављивање: `RabbitConfig`, `QueuePublisher` и `AsyncQuestionPublisher`. Класа `RabbitConfig` декларативно дефинише топологију реда. Ред `ai-requests` је трајан како би поруке остале при рестартовању распоређивача, размену (енгл. *exchange*) типа `TopicExchange` и везу (енгл. *binding*) са кључем `ai.requests`. Ова конфигурација се примењује при покретању апликације, чиме се обезбеђује да ред постоји пре него што се прва порука објави.

Класа `QueuePublisher` (код 4.7) серијализује поруке у *JSON* формат и шаље их путем `RabbitTemplate`. Свака порука садржи три поља:

1. `requestId` (јединствени идентификатор захтева);
2. `question` (текст питања);
3. `enqueuedAt` (временска ознака објављивања поруке у ред).

При објављивању сваке поруке класа такође бележи временску ознаку у `AsyncRequestTimingStore` и инкрементира Прометејев бројач.

```
...  
@Service  
@RequiredArgsConstructor  
public class QueuePublisher {  
  
    private final RabbitTemplate rabbitTemplate;  
    private final RabbitConfig rabbitConfig;  
    private final AsyncMetrics asyncMetrics;  
    private final AsyncRequestTimingStore timingStore;  
    private final ObjectMapper objectMapper = new  
        ObjectMapper();  
}
```

```
public void publishQuestionWithRequestId(String
    requestId, String question) {
    Instant now = Instant.now();
    timingStore.markPublished(requestId, now);
    asyncMetrics.asyncPublished.increment();

    double enqueuedAt = now.toEpochMilli() / 1000.0; //
        epoch seconds (with millis)

    Map<String, Object> payload = Map.of(
        "requestId", requestId,
        "question", question,
        "enqueuedAt", enqueuedAt
    );

    try {
        String json = objectMapper.writeValueAsString(
            payload);
        rabbitTemplate.convertAndSend(rabbitConfig.
            aiRequestsQueue().getName(), json);
    } catch (JsonProcessingException e) {
        throw new IllegalStateException("Failed to
            serialize message to JSON", e);
    }
}
```

Код 4.7: Класа QueuePublisher

Класа `AsyncQuestionPublisher` (код 4.8) обавија позив класе `QueuePublisher` анотацијом `@Async`, чиме се објављивање извршава у засебном извршном скупу нити (енгл. *executor thread pool*) (`qaAsyncExecutor`). Ова одлука је донета како објављивање порука не би блокирало ХТТП нит контролера, посебно у условима високог оптерећења када се велики број захтева прима у кратком временском интервалу.

...

```
@Service
@RequiredArgsConstructor
@Slf4j
public class AsyncQuestionPublisher {

    private final QueuePublisher publisher;

    @Async("qaAsyncExecutor")
    public void publish(String requestId, String question)
    {
        try {
            publisher.publishQuestionWithRequestId(
                requestId, question);
        } catch (Exception e) {
            log.error("Failed to publish async question.
                requestId={}", requestId, e);
        }
    }
}
```

Код 4.8: Класа AsyncQuestionPublisher

Модел података и перзистенција. Стање асинхроног захтева чува се у релационој бази података преко ЈПА ентитета `ModelRequest` (код 4.9), мапираног на табелу `model_request`. Ентитет садржи следећа поља:

```
@Entity
@Table(name = "model_request")
public class ModelRequest {
    @Id
    private String id;
    private String requestId;
    private String question;
    private String answer;
    private String context;
    private Float score;
    @Enumerated(EnumType.STRING)
    private ModelRequestStatus status;
```

```
private Instant createdAt;  
private Instant completedAt;  
}
```

Код 4.9: Класа `ModelRequest`

Статус захтева је моделиран енумерацијом `ModelRequestStatus` са две могуће вредности: (1) `PENDING` (захтев је објављен) и (2) `COMPLETED` (резултат је примљен од сервиса за извршавање модела). Захтев прелази из стања `PENDING` у стање `COMPLETED` при пријему повратног позива.

За приступ подацима коришћен је Спрингов ЈПА репозиторијум, а у развојном окружењу коришћена је H2 база у меморији, чијом се шемом управља декларативно путем *Liquibase* алата.

Инструментација метрика. У сервис су уграђене три Прометејеве метрике, реализоване помоћу *Micrometer* библиотеке и класе `AsyncMetrics`:

- `backend_async_published_total`, бројач укупног броја порука објављених у ред. Инкрементира се при сваком успешном објављивању у класи `QueuePublisher`. Скрипта за анализу (потпоглавље 4.7) користи ову метрику за израчунавање долазне стопе λ путем *PromQL* упита `sum(rate(backend_async_published_total[1m]))`.
- `backend_async_completed_total`, бројач укупног броја примљених повратних позива од сервиса за извршавање модела. Инкрементира се при сваком успешном ажурирању статуса на `COMPLETED`. У скрипти за анализу се користи за израчунавање пропусности система (сервисне стопе μ).
- `backend_async_end_to_end_seconds`, хистограм *end-to-end* латенције, односно времена од објављивања поруке до пријема резултата. Конфигуриран је са перцентилним хистограмом (`publishPercentileHistogram`), чиме се омогућава израчунавање перцентилних вредности (p95, p99) на страни Прометеја. Ова метрика представља основни извор података за перцентилне латенције у анализи резултата (поглавље 5).

Све три метрике представљају кључне изворе података за анализу. Скрипта `prom_analysis.py` их повлачи из Прометеја и на основу њих израчунава долазну стопу, пропусност, перцентилне латенције и време прекорачења нивоа

услуге. Све ове вредности затим се уписују у излазне *CSV* датотеке. Поред улоге у анализи, метрике су коришћене и за праћење рада система у реалном времену преко Графана контролне табле.

4.3 Сервис за извршавање модела ВИ

Сервис за извршавање модела ВИ реализован је као *FastAPI* апликација у програмском језику Пајтон (верзија 3.12). Овај сервис представља примарни носилац рачунарског оптерећења у систему и циљну компоненту над којом се примењује хоризонтално скалирање. У овом одељку описана је имплементација четири кључне целине: (1) животног циклуса апликације и крајњих тачака, (2) логике извршавања модела, (3) потрошача порука из реда и (4) повратног позива ка сервису за оркестрацију.

Животни циклус апликације и крајње тачке. Иницијализација апликације реализована је путем *FastAPI lifespan* контекстног менаџера (код 4.10) у датотеци `main.py`. При покретању апликација извршава следеће кораке: (1) учитава претходно припремљене делове документа из *JSON* датотеке, (2) иницијализује *QA* модел и уграђивач (енгл. *embedder*), (3) креира *HTTP* клијент за повратне позиве и (4) покреће позадинску нит за конзумирање порука из реда. При гашењу апликације контекстни менаџер обезбеђује уредно заустављање потрошача и затварање *HTTP* конекција:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    chunks = load_chunks()

    backend = BackendClient()

    qa = QAService.build(chunks)

    app.state.qa = qa
    app.state.backend = backend

    worker = None
    print(f"[QUEUE] Startup event. ENABLE_QUEUE_CONSUMER={
        settings.enable_queue_consumer}")
```

```
if settings.enable_queue_consumer:
    ai_worker_active.set(1)
    print("[QUEUE] Starting RabbitMQ consumer thread...")
    worker = RabbitWorker(qa=qa, backend=backend)
    worker.start()
else:
    ai_worker_active.set(0)
    print("[QUEUE] Queue consumer disabled; skipping
        consumer thread.")

app.state.worker = worker

yield

if worker:
    worker.stop(timeout=10)
backend.close()
```

Код 4.10: Асинхрони контекстни менаџер који иницијализује апликацију.

Апликација излаже три крајње тачке. Крајња тачка `POST /qa` прима питање, извршава модел над улазним питањем и враћа одговор; она се не користи у експериментима, али омогућава директно тестирање логике извршавања модела ван асинхроног тока. Крајња тачка `GET /health` враћа статус апликације и користи се за провере спремности и живости у Кубернетесовом окружењу. Крајња тачка `GET /metrics` излаже Прометејеве метрике.

Централна конфигурација. Сви конфигурациони параметри апликације обједињени су у класи `Settings` (датотека `config.py`). Класа при иницијализацији учитава вредности из окружењских променљивих помоћу функције `os.getenv`, при чему свака променљива има дефинисану подразумевану вредност која се користи у њеном одсуству. На тај начин је конфигурација сабрана на једном месту, уместо да буде расута кроз изворни код. Конфигурација обухвата параметре за повезивање са распоређивачем порука (адреса, назив реда, `rabbitmq_prefetch_count`), адресе сервиса за оркестрацију (крајња тачка повратног позива и провера стања сервиса), параметре поновног покушаја (`backend_result_max_attempts`, `backend_result_timeout_sec`) и називе

модела за извршавање.

Подразумевање вредности односе се на Кубернетесово окружење (адресе преко интерних DNS имена сервиса), али се могу заменити постављањем одговарајућих окружењских променљивих (нпр. путем `.env` датотеке) за локални развој. На тај начин је исти изворни код употребљив у обе средине без измена.

Логика извршавања модела. Логика извршавања модела је енкапсулирана у класи `QAService` (датотека `inference.py`), која представља централну компоненту за одговарање на питања. Класа користи два модела: `all-MiniLM-L6-v2` за генерисање семантичких уграђивања (енгл. `embeddings`) и `distilbert-base-uncased-distilled-squad` за екстрактивно одговарање на питања.

Иницијализација је реализована преко методе `_load_models` (код 4.11), која учитава оба модела и унапред израчунава уграђивања за све делове документа:

```
def _load_models(self) -> Tuple[Any, SentenceTransformer,
    Any]:
    if self._models is not None:
        return self._models
    try:
        qa_pipeline = pipeline(
            task="question-answering",
            model=settings.qa_model,
            tokenizer=settings.qa_model,
        )
        embedder = SentenceTransformer(settings.embed_model)
        embeddings = embedder.encode(
            self.chunks,
            convert_to_tensor=True,
            normalize_embeddings=True,
        )
        models = (qa_pipeline, embedder, embeddings)
        self._models = models
    return models
```

```
except Exception as e:
    raise RuntimeError(f"Failed to initialize QA models
                        : {e}")
```

Код 4.11: Иницијализација модела и уграђивача.

Претходно израчуната уграђивања омогућавају да се при обради сваког питања избегне поновно кодирање целокупног корпуса, чиме се значајно смањује латенција по захтеву. Метода `answer` (код 4.12) извршава модел у два корака:

1. **Семантичка претрага:** Питање се кодира у вектор уграђивања, а затим се израчунава косинусна сличност са унапред израчунатим уграђивањима свих делова документа. Део документа са највећом сличношћу се бира као контекст.
2. **Екстрактивно одговарање:** Изабрани контекст и оригинално питање се прослеђују QA моделу, који издваја одговор из контекста.

```
def answer(self, question: str) -> Dict[str, Any]:
    q = question.strip()
    if len(q) < 3:
        raise ValueError("Question must be at least 3
                           characters.")

    try:
        qa_pipeline, embedder, embeddings = self.
            _load_models()
    except RuntimeError:
        return {
            "answer": "Model initialization failed (offline
                       or missing weights).",
            "context": "",
            "score": 0.0,
        }

    q_emb = embedder.encode(q, convert_to_tensor=True,
                             normalize_embeddings=True)
```

```
scores = util.cos_sim(q_emb, embeddings)[0].cpu().numpy()
idx = int(np.argmax(scores))
context = self.chunks[idx]

result = qa_pipeline(question=q, context=context)
return {
    "answer": result.get("answer", ""),
    "context": context,
    "score": float(scores[idx]),
}
```

Код 4.12: Метода која извршава модел ВИ.

Овај приступ је изабран јер представља репрезентативно рачунарско оптерећење за потребе експеримента: обухвата кодирање текста у векторску репрезентацију, израчунавање косинусне сличности и екстракцију одговора путем језичког модела, чиме се ствара довољно интензивно оптерећење процесорске јединице за анализу динамике скалирања.

Потрошач порука из реда. Класа `RabbitWorker` реализује потрошача порука који ради у позадинској нити и представља примарни режим рада у свим експерименталним сценаријима (датотека `queue_worker.py`). Потрошач се покреће при иницијализацији апликације и остаје активан током целог животног циклуса пода.

Унутрашња метода `_run` (код 4.13) успоставља везу са *RabbitMQ* расподељивачем и конзумира поруке из реда `ai-requests`. При конзумирању се примењује `prefetch_count` параметар, који ограничава број порука које један потрошач може преузети пре него што потврди обраду претходних. Ова поставка је методолошки значајна јер утиче на расподелу `READY` и `UNACKED` порука у реду, а самим тим и на сигнал који `КЕДА` користи за одлучивање о скалирању, као што је описано у потпоглављу 3.4.

```
def _run(self) -> None:
    print(
        f"[QUEUE] Consumer loop started. Connecting to "
        f"{settings.rabbitmq_url}, "
        f"queue={settings.rabbitmq_queue}, prefetch={
```

```
        settings.rabbitmq_prefetch_count}"
    )

    while not self._stop.is_set():
        connection = None
        try:
            params = pika.URLParameters(settings.
                rabbitmq_url)
            connection = pika.BlockingConnection(params
                )
            print("[QUEUE] Connected to RabbitMQ")

            channel = connection.channel()
            channel.queue_declare(queue=settings.
                rabbitmq_queue, durable=True)
            channel.basic_qos(prefetch_count=settings.
                rabbitmq_prefetch_count)

            print(f"[QUEUE] Waiting for messages on
                queue '{settings.rabbitmq_queue}'...")

            for method_frame, properties, body in
                channel.consume(
                    settings.rabbitmq_queue,
                    inactivity_timeout=5,
                ):
                    if self._stop.is_set():
                        break
                    if body is None:
                        continue

                    self._process_message(body)
                    channel.basic_ack(method_frame.
                        delivery_tag)

        except Exception as e:
            print(f"[QUEUE] Consumer error: {e}.
```

```
        "Retrying in 5 seconds...")
    time.sleep(5)
finally:
    try:
        if connection:
            connection.close()
    except Exception:
        pass
```

Код 4.13: Метода за дохватање порука из реда.

Обрада сваке поруке у методи `_process_message` (код 4.14) одвија се у три фазе:

1. **Парсирање** се врши тако што се тело поруке десеријализује из *JSON* формата у *Pydantic* модел `QueueMessage`, који садржи поља `requestId`, `question` и опционо `enqueuedAt`. Уколико десеријализација не успе, порука се одбацује уз евиденцију грешке.
2. **Извршавање модела** се врши методом `QAService.answer`, која се позива са текстом питања.
3. **Повратни позив** чини резултат извршавања модела који се шаље сервису за оркестрацију путем класе `BackendClient`. По успешном повратном позиву порука се потврђује (`basic_ack`), чиме се уклања из реда.

```
def _process_message(self, body: bytes) -> None:
    ai_queue_received_total.inc()
    msg_start = time.perf_counter()
    ai_inflight_messages.inc()

    try:
        raw = json.loads(body)
        msg = QueueMessage(**raw)
    except Exception as e:
        print(f"[QUEUE] Failed to parse message from queue: {e}")
        ai_queue_processed_total.labels(result="parse_error").inc()
```

```
        ai_inflight_messages.dec()
    return

if msg.enqueuedAt is not None:
    try:
        wait_sec = max(0.0, time.time() - float(msg.
            enqueuedAt))
        ai_queue_wait_seconds.observe(wait_sec)
    except Exception:
        pass

# inferencija
inf_start = time.perf_counter()
try:
    answer = self.qa.answer(msg.question)
except ValueError:
    answer = {"answer": "", "context": "", "score":
        0.0}
finally:
    ai_inference_seconds.observe(time.perf_counter() -
        inf_start)

# pouratni poziv ka backend servisu
try:
    self.backend.post_result(msg.requestId, answer)
    ai_queue_processed_total.labels(result="ok").inc()
except Exception:
    ai_queue_processed_total.labels(result="
        callback_error").inc()
finally:
    ai_queue_processing_seconds.observe(time.
        perf_counter() - msg_start)
    ai_inflight_messages.dec()
```

Код 4.14: Метода која процесира поруку и прослеђује је сервису за оркестрацију.

Инструментација метрика. Сервис за извршавање модела излаже оп-

сежан скуп Прометејевих метрика дефинисаних у датотеци `metrics.py`, које се користе за праћење рада на контролној табли у Графани. За разлику од метрика сервиса за оркестрацију, метрике сервиса за извршавање модела имају првенствено оперативну улогу и омогућавају увид у рад система у реалном времену, идентификацију уских грла и верификацију исправности током извршавања експеримената. Кључне метрике укључују бројаче примљених и обрађених порука (`ai_queue_received_total`, `ai_queue_processed_total`), показатељ времена чекања порука у реду (`ai_queue_wait_seconds`) и показатељ тренутно активних порука у обради (`ai_inflight_messages`).

4.4 Контејнеризација и аутоматизација изградње

У поглављу 3.3 описани су принципи контејнеризације и њихов значај за поновљивост и фер поређење у експериментима. У овом одељку представљене су конкретне Докер датотеке за оба сервиса и конфигурација *Skaffold* алата за аутоматизацију целокупног процеса изградње и постављања апликације.

Докер конфигурација сервиса. Сервис за оркестрацију пакује се у Докер слику која садржи извршно окружење за Јаву 21 (код 4.15), док се сервис за извршавање модела пакује у Докер слику која садржи Пајтон верзије 3.12 (код 4.16). Обе Докер датотеке садрже основни скуп команди и подразумеваних вредности који се користи при развоју апликација у тестним окружењима.

```
FROM amazoncorretto:21-alpine
WORKDIR /app

ENV SPRING_PROFILES_ACTIVE=k8s

RUN addgroup -S spring && adduser -S spring -G spring

COPY build/libs/backend-0.0.1-SNAPSHOT.jar /app/app.jar

USER spring:spring
EXPOSE 8080
```

```
HEALTHCHECK --interval=30s --timeout=5s \  
  CMD wget -q -O- http://localhost:8080/actuator/health ||  
    exit 1  
  
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Код 4.15: Докер датотека сервиса за оркестрацију.

```
FROM python:3.12-slim  
  
ENV PYTHONUNBUFFERED=1  
  
WORKDIR /app  
  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY ./app .  
  
EXPOSE 36210  
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port",  
    "36210"]
```

Код 4.16: Докер датотека сервиса за извршавање модела.

Аутоматизација изградње коришћењем *Skaffold* алата. Целокупни процес изградње Докер слика и примене Кубернетесових манифеста аутоматизован је конфигурацијом у датотеци `skaffold.yaml`. *Skaffold* координише три фазе: изградњу артефаката, креирање Докер слика и примену манифеста на кластер.

Конфигурација дефинише три профила за постављање, који одговарају различитим експерименталним сценаријима:

- **Подразумевани профил:** Примењује основне манифесте (*RabbitMQ*, *Deployment* и *Service* за оба сервиса) без механизма аутоскалирања.
- **Профил hpa:** Додатно примењује ХАСП манифест (датотека `ai-model-service-hpa.yaml`).

- **Профил keda:** Додатно примењује КЕДА *ScaledObject* манифест (дато-тека `rabbitmq-keda-scaledobject.yml`).

Прелазак између сценарија се остварује навођењем профила при покре-тању:

```
skaffold run -p hpa      # HASP scenario
skaffold run -p keda     # KEDA scenario
```

Код 4.17: Покретање изградње апликације.

Оваква организација обезбеђује да се при промени сценарија мењају ис-кључиво манифести за скалирање, док апликативне компоненте и њихове Докер слике остају идентичне, што је предуслов за фер поређење описано у поглављу 3.3.

4.5 Кубернетес манифести и распоређивање КОМПОНЕНТИ

У потпоглављу 3.3 описани су типови Кубернетесових ресурса и принципи који обезбеђују фер поређење, а у потпоглављу 3.4 представљене су страте-гије скалирања на концептуалном нивоу. У овом потпоглављу приказана је конкретна реализација која укључује манифесте апликативних компоненти, конфигурацију распоређивача порука, као и организацију именских простора и комуникације.

Распоређивање сервиса за извршавање модела. *Deployment* мани-фест сервиса за извршавање модела (`ai-model-service-deployment.yml`, код 4.18) представља централни експериментални манифест, јер је управо овај манифест централна тачка хоризонталног скалирања у оба сценарија.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-model-service
  namespace: default
spec:
  replicas: 1                                # (1)
```

```
selector:
  matchLabels:
    app: ai-model-service
template:
  metadata:
    labels:
      app: ai-model-service
    annotations:                                # (2)
      prometheus.io/scrape: "true"
      prometheus.io/path: "/metrics"
      prometheus.io/port: "36210"
  spec:
    terminationGracePeriodSeconds: 10          # (3)
    containers:
      - name: ai-model-service
        image: ai-model-service:latest
        ports:
          - containerPort: 36210
        resources:                               # (4)
          requests:
            cpu: "1000m"
          limits:
            cpu: "2000m"
        lifecycle:                               # (5)
          preStop:
            exec:
              command: ["/bin/sh", "-c", "sleep 3"]
        readinessProbe:                           # (6)
          httpGet:
            path: /health
            port: 36210
            initialDelaySeconds: 15
            periodSeconds: 10
        livenessProbe:                           # (7)
          httpGet:
            path: /health
            port: 36210
```

```
initialDelaySeconds: 30
periodSeconds: 30
```

Код 4.18: *Deployment* манифест сервиса за извршавање модела.

Сваки означени део има конкретан утицај на понашање система у експерименту:

- 1. Почетни број реплика.** Почетна вредност броја реплика износи 1 и представља полазно стање пре него што механизам аутошкалирања преузме контролу. Приликом примене манифеста ХАСП-а или КЕДА-а, контролер скалирања преузима управљање бројем реплика, при чему минимални број реплика одговара почетној вредности. Ово обезбеђује да на почетку сваког експеримента систем полази из истог стања.
- 2. Анотације за Прометеј.** Анотације за Прометеј омогућавају приступ метрикама и сакупљање података са сваког пода сервиса за извршавање модела. Коришћењем `kubernetes_sd_configs` (манифест `prometheus-config.yml`) Прометеј аутоматски прикупља метрике са нових подова, без потребе за ручним ажурирањем конфигурације.
- 3. Време завршавања.** Параметар `terminationGracePeriodSeconds` дефинише максимално време које Кубернетес даје поду да заврши обраду пре принудног гашења. Ово је важно при смањивању броја реплика. Током тог периода, потрошач порука може завршити обраду текуће поруке и уредно затворити конекцију са распоређивачем, чиме се спречава губитак поруке.
- 4. Ресурсна ограничења.** Вредност `cpu: requests=1000m` и вредност `cpu: limits=2000m` директно утичу на понашање оба механизма скалирања, али на суштински различите начине. У ХАСП сценарију, `requests` вредност представља именилац у формули за израчунавање процента искоришћености (формула 2.1). При циљној искоришћености од 60%, скалирање се покреће када просечна стварна потрошња по реплици пређе 600m (60% од 1000m). Друга вредност `limits=2000m` омогућава свакој реплици да користи два процесорска језгра. Ови параметри су идентични за обе стратегије скалирања.

5. **Пауза пред заустављање пода.** Инструкција `preStop: sleep 3` уводи паузу од 3 секунде пре гашења контејнера. Ово је практична техника у Кубернетесу која омогућава балансеру оптерећења да заврши ажурирање правила рутирања пре него што контејнер престане да одговара на захтеве. Без ове паузе, поруке би могле бити усмерене на под који је већ у процесу гашења, што би довело до неуспешне обраде поруке.
6. **Провере спремности и живости.** Провера спремности је дефинисана инструкцијом `readinessProbe`, док је провера живости дефинисана инструкцијом `livenessProbe`. Док провера спремности не успе, под се не укључује у сервисни балансер, чиме се спречава слање захтева инстанци која још увек није спремна да обрађује поруке. Провера спремности се врши сваких 10 секунди. Провера живости се врши на сваких 30 секунди и утврђује да ли под формално ради, али је функционално неисправан.

Сервисни ресурс и мрежна апстракција. Поред *Deploymenta*, дефинисан је и сервисни ресурс (код 4.19) типа `ClusterIP`. Селектором `app: ai-model-service` аутоматски обухвата све подове сервиса за извршавање модела, укључујући нове репликае настале скалирањем. У Кубернетесовој архитектури, ово представља апстракцију која раздваја клијенте од подова. У пракси то значи да сервис за оркестрацију комуницира са сервисом за извршавање модела преко адресе `http://ai-model-service`, док Кубернетес балансира саобраћај на тренутно доступне репликае. Повећање броја реплика не захтева промену конфигурација, чиме се остварује хоризонтално аутоскалирање.

```
apiVersion: v1
kind: Service
metadata:
  name: ai-model-service
spec:
  selector:
    app: ai-model-service
  ports:
    - port: 80
      targetPort: 36210
```

```
type: ClusterIP
```

Код 4.19: Сервисни ресурс и мрежна апстракција сервиса за извршавање модела.

Распоређивање сервиса за оркестрацију. *Deployment* манифест сервиса за оркестрацију дефинише једну реплику, јер овај сервис није предмет скалирања у раду. Ресурсна ограничења су постављена тако да обезбеђују довољне ресурсе за Спринг апликацију и JBM. Манифест укључује и анотације за Прометеј.

Распоређивач порука *RabbitMQ*. *RabbitMQ* је у посебном именском простору постављен као *StatefulSet* тип ресурса. Овај тип ресурса обезбеђује стабилан идентитет пода и трајно складиште преко *PersistentVolumeClaim*. Овим се постиже чување података у случају престанка рада пода. Приликом поновног покретања пода, у распоређивач порука се учитавају перзистиране поруке.

За сакупљање метрика је подразумевано коришћен Прометеј, који омогућава излагање `rabbitmq_queue_messages_ready` метрике. Ова метрика је неопходна за КЕДА скалер, јер се користи као сигнал за скалирање. Манифест дефинише и проверу живости и спремности, као и ресурсна ограничења која су идентична у свим експериментима.

Организација именских простора. Систем користи три именска простора:

- `default` - за апликативне компоненте;
- `rabbitmq` - за распоређивач порука;
- `monitoring` - за инфраструктуру праћења и сакупљања метрика.

Поред тога, КЕДА оператор је инсталиран у именском простору `keda`. Оваква организација обезбеђује логичко раздвајање компоненти и олакшава управљање ресурсима, при чему комуникација између именских простора се остварује преко ДНС имена.

4.6 Конфигурација механизма аутоматског скалирања

Као што је описано у потпоглављу 3.4, два механизма скалирања примењују различите сигнале над истим циљним ресурсом. У овом одељку приказана је конкретна конфигурација сваког механизма, са образложењем избора параметара и њиховог утицаја на динамику скалирања.

Конфигурација ХАСП механизма. Манифест `ai-model-service-hpa.yml` (код 4.20) дефинише *HorizontalPodAutoscaler* ресурс над *Deployment* ресурсом сервиса за извршавање модела.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ai-model-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ai-model-service
  minReplicas: 1 # (1)
  maxReplicas: 8
  metrics: # (2)
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  behavior: # (3)
    scaleUp:
      stabilizationWindowSeconds: 0
      selectPolicy: Max
      policies:
      - type: Pods
        value: 4
        periodSeconds: 15
```

```
- type: Percent
  value: 100
  periodSeconds: 15
scaleDown:
  stabilizationWindowSeconds: 30
  selectPolicy: Max
  policies:
    - type: Percent
      value: 50
      periodSeconds: 15
```

Код 4.20: Конфигурација ХАСП механизма.

У манифесту посматрамо три означене ставке:

1. **Опсег реплика.** Минимални (1) и максимални (8) број реплика идентични су у оба сценарија. Максимални број је одређен на основу расположивих ресурса Миникјубовог кластера (10 процесора, 24 GB меморије), при чему 8 реплика са `limits=2000m` захтевају до 16 процесорских језгара, уз остатак за системске компоненте, распоређивач порука и сервис за оркестрацију.
2. **Метрика и циљна вредност.** ХАСП посматра просечну искоришћеност процесорске јединице свих реплика сервиса за извршавање модела и тежи одржавању ове вредности на 60%. Конкретно, на основу формуле 2.1, жељени број реплика се израчунава као:

$$N_{\text{new}} = \left\lceil N_{\text{current}} \cdot \frac{M_{\text{current}}}{M_{\text{target}}} \right\rceil = \left\lceil N_{\text{current}} \cdot \frac{M_{\text{current}}}{0.60} \right\rceil$$

где M_{current} представља тренутну просечну искоришћеност изражену као удео `requests` вредности. На пример, уколико тренутно постоје 2 реплика и просечна потрошња износи 900m (90% од `requests=1000m`), жељени број реплика је $\lceil 2 \cdot 0.90 / 0.60 \rceil = 3$.

3. **Понашање при увећању и смањењу броја реплика.** При увећању (енгл. *scale-up*), параметар `stabilizationWindowSeconds` је постављен на 0, чиме се елиминише прозор стабилизације и омогућава тренутна реакција на пораст оптерећења. Политике `Pods: 4` (додавање до 4 пода)

и `Percent: 100` (удвостручавање тренутног броја) примењују се паралелно уз `selectPolicy: Max`, па се у сваком циклусу (сваких 15 секунди) бира она која даје већи број нових реплика (нпр. са 1 на 5 уместо на 2, или са 6 на 8 у оба случаја, ограничено максимумом). При смањивању (енгл. *scale-down*), `stabilizationWindowSeconds` од 30 секунди уводи кратак период стабилизације током којег се неће смањити број реплика ако је у скорије време тражени број био већи, чиме се спречавају осцилације при флукутирајућем оптерећењу. Додатно, политика `Percent: 50` ограничава смањивање на највише половину тренутног броја реплика по интервалу од 15 секунди, обезбеђујући постепено и контролисано смањење капацитета (нпр. 8→4→2→1) уместо наглог пада.

Конфигурација КЕДА механизма. Манифест који дефинише КЕДА механизам у пројекту је датотека `rabbitmq-keda-scaledobject.yml` (код 4.21). Унутар манифеста су дефинисана три ресурса: (1) *Secret* са конфигурационим параметрима за *RabbitMQ*; (2) *TriggerAuthentication* за аутентикацију; и (3) *ScaledObject*.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: ai-model-keda
spec:
  scaleTargetRef:
    kind: Deployment
    name: ai-model-service
  pollingInterval: 5
  cooldownPeriod: 30
  minReplicaCount: 1
  maxReplicaCount: 8
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleUp:
          stabilizationWindowSeconds: 0
          selectPolicy: Max
          policies:
```

```
        - type: Pods
          value: 4
          periodSeconds: 15
        - type: Percent
          value: 100
          periodSeconds: 15
      scaleDown:
        stabilizationWindowSeconds: 30
        selectPolicy: Max
        policies:
          - type: Percent
            value: 50
            periodSeconds: 15
  triggers:
    - type: rabbitmq
      authenticationRef:
        name: rabbitmq-trigger-auth
      metadata:
        protocol: amqp
        mode: QueueLength
        queueName: ai-requests
        vhostName: /
        value: "100"
```

Код 4.21: Конфигурација КЕДА механизма

Сигнал скалирања. КЕДА периодично дохвата тренутни број спремних порука из реда путем *AMQP* протокола. Режим *QueueLength* значи да жељени број реплика се израчунава на основу укупног броја порука у реду, а не на основу стопе пристизања. Конкретно, жељени број реплика се израчунава као:

$$N_{new} = \left\lceil \frac{Q_{ready}}{100} \right\rceil$$

где Q_{ready} представља тренутни број спремних порука у реду, а 100 је задата циљна вредност по реплици (параметар `value`). На пример, при 250 спремних порука, КЕДА ће захтевати $\lceil 250/100 \rceil = 3$ реплике.

Важно је напоменути да КЕДА реагује искључиво на број спремних порука, а не на укупан број порука у реду. Поруке које су преузете од стране потрошача, али још нису потврђене, не улазе у сигнал скалирања. Из тог разлога је подешена константна вредност `RABBITMQ_PREFETCH_COUNT=5` која ограничава број порука које потрошач може да преузме и обрађује. Већи број би смањивао број спремних порука, односно мањи број би повећавао број спремних порука.

Период хлађења и опсег реплика. Параметар `cooldownPeriod=30` секунди одређује време након којег се, у одсуству порука у реду, број реплика враћа на минималну вредност. Ово спречава преурањено гашење реплика у ситуацијама када краткотрајни период празног реда не значи да је оптерећење заиста престало. Вредност од 30 секунди је изабрана тако да буде идентична као код ХАСП механизма. Што се тиче опсега реплика, он је такође исти као код ХАСП конфигурације: минимално једна реплика, а максимално осам реплика.

Табела 4.1: Упоредни преглед параметара ХАСП и КЕДА конфигурације

Параметар	ХАСП	КЕДА
Циљни ресурс	<code>ai-model-service</code>	<code>ai-model-service</code>
Минимални број реплика	1	1
Максимални број реплика	8	8
Сигнал скалирања	<i>CPU</i> искоришћеност	Дужина реда
Циљна вредност	60% ресурса по реплици	100 порука по реплици
Интервал провере	15s	5s
Стабилизација (<i>scale-down</i>)	30s	30s
Стабилизација (<i>scale-up</i>)	0s	0s
CPU requests	1000m	1000m
CPU limits	2000m	2000m

4.7 Мониторинг, генератор оптерећења и скрипта за анализу

Мониторинг. Мониторинг је постављен у засебном именском простору и састоји се од Прометеја и Графана. Примарне улоге ове инфраструктуре су обезбеђивање увида у понашање система приликом извршавања апликације и складиштење временских серија које се користе за анализу резултата.

Прометеј је конфигурисан са глобалним интервалом прикупљања података од 5 секунди. Прометеј прати сва три кључна елемента инфраструктуре (сервис за оркестрацију, сервис за извршавање модела и распоређивач порука). Имплементација подразумева да Прометеј има приступ Кубернетес АПИ-ју. **Графана** је коришћена за визуелно праћење система у реалном времену током извршавања експеримента. Помоћу Графана утврђивани су проблеми током експеримената и проверавана је исправност прикупљања података.

Генератор оптерећења. Генератор оптерећења производи захтеве према систему и ствара синтетичко продукционо окружење. Реализован је као Пајтон скрипта која чита план оптерећења из *JSON* датотеке и шаље ХТТП захтеве ка сервису за оркестрацију у складу са задатим фазама. Сваки план оптерећења дефинише низ фаза, при чему свака фаза садржи број порука и трајање фазе. Генератор равномерно распоређује захтеве унутар сваке фазе, чиме се постиже симулација продукционог окружења. Контролисан прилив порука омогућава предвидиво понашање система, односно сервиса за извршавање модела, па су и посматрање система и уочавање аномалија поједностављени.

Два плана оптерећења дефинисана у потпоглављу 3.4 чувају се у репозиторијуму и користе се без модификација у свим експерименталним комбинацијама. На тај начин се обезбеђује да идентични профили оптерећења буду примењени и у ХАСП и у КЕДА сценарију, чиме се елиминише једна потенцијална варијабла приликом поређења.

Скрипта за офлајн анализу. Скрипта за офлајн анализу представља централни алат за обраду експерименталних података. Њена улога је да дохвати временске серије из Прометеја, усклади их временски, израчуна агрегатне и изведене метрике и генерише излазне артефакте описане у потпоглављу 3.5. Скрипта комуницира са Прометејевим АПИ-јем путем ХТТП-а и дохвата временске серије. Дохватају се метрике реда порука, метрике скалирања, метрике латенције и пропусности и метрике ресурса.

Прикупљене временске серије потичу из различитих извора и не морају бити узорковане у истим тренуцима. Скрипта их временски усклађује поновним узорковањем на фиксни интервал од 5 секунди, што одговара интервалу прикупљања метрика у Прометеју. На тај начин се обезбеђује да све метрике буду представљене у истим временским тачкама, чиме се омогућава упоре-

ђивање и анализа података. На основу овога скрипта израчуна неколико агрегатних и изведених величина:

- просечне, максималне и минималне вредности метрика реда, латенције и броја реплика,
- интеграл заостатка (формула 3.2), нумеричком интеграцијом временске серије $Q(t)$,
- време прекорачења нивоа услуге (формула 3.3), сумирањем интервала у којима p95 или p99 премашују задати праг (2 секунде),
- композитни индекс ефикасности аутоскалирања AEI (формула 3.4),
- ефикасност скалирања $\eta(N)$, однос пропусности при N реплика и пропусности при једној реплици,
- долазну и сервисну стопу и однос $\rho = \lambda/\mu$.

Глава 5

Резултати и анализа

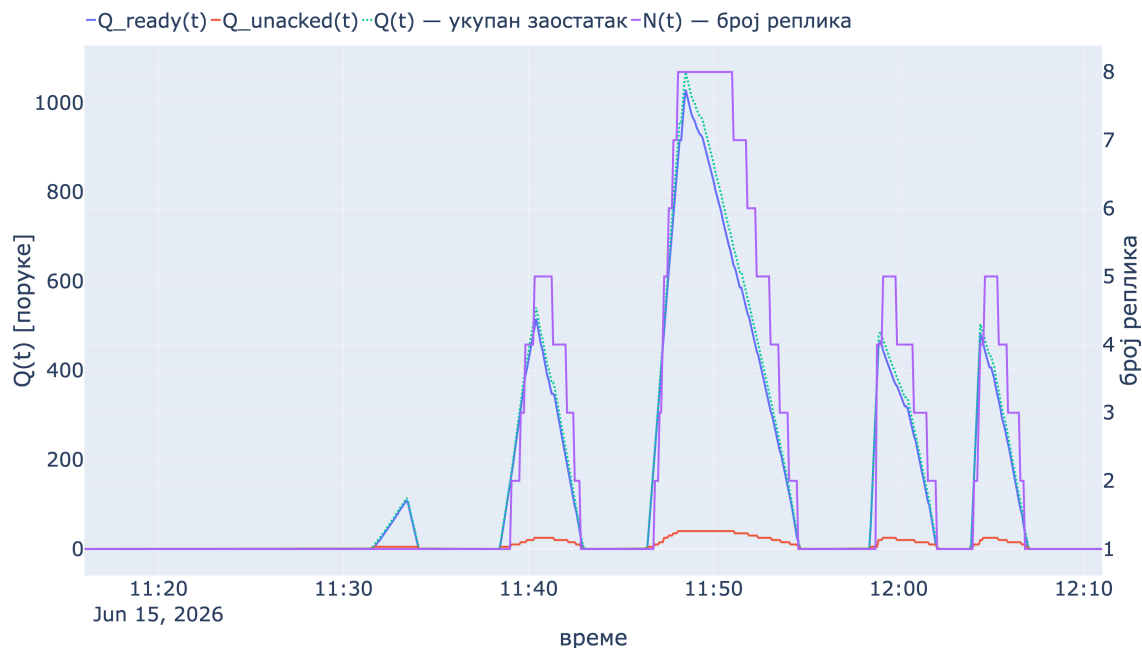
5.1 Резултати за план са наглим скоковима оптерећења

Овај план симулира сценарио у коме оптерећење долази у наглим таласима, што у пракси одговара изненадним порастима саобраћаја, краткотрајним пословима (енгл. *job*) или серијским задацима. Кључно питање које се поставља је: *који механизам брже реагује и колико заостатака капијења остаје у систему током и после таласа?* Свака фаза плана смењује краће периоде интензивног оптерећења са периодима опоравка, при чему оптерећење у појединим фазама прелази тренутни капацитет система. У осталим фазама систем ради испод капацитета и тиме се омогућава посматрање реактивности при повећању оптерећења и понашања система при смањивању броја реплика.

Динамика заостатка у реду и броја реплика

На сликама 5.1 и 5.2 приказане су временске серије укупног заостатка и броја реплика за оба механизма. Код КЕДА механизма (слика 5.1) видљиво је да број реплика расте у корацима непосредно пратећи раст реда, са јасно уочљивом везом између динамике реда и броја реплика. Заостатак достиже максимум од 1049 порука у просеку (табела 5.1), након чега се систем постепено опоравља подизањем до максималних 8 реплика.

Код ХАСП механизма (слика 5.2) уочљиво је другачије понашање: реплике расту у широким степеницама које не прате тренутно стање реда, већ реагују



Слика 5.1: КЕДА - динамика заостатка реда и броја реплика

на посматрану искоришћеност процесорске јединице. Последица је да систем у критичним тренуцима задржава превелики или премали број реплика у односу на стварно накупљен посао. Иако сигнал ХАСП-а касни за стањем реда, већи број реплика које ХАСП превентивно одржава ограничава раст реда, па ХАСП достиже нешто *нижи* максимални заостатак (у просеку 996 порука) него КЕДА. Ова разлика биће детаљно анализирана у наставку, јер открива да заостатак у реду и утрошак реплика чине компромис, а не два показатеља која се истовремено побољшавају.

Метрике реда порука

Табела 5.1 показује да у плану са наглим скоковима ХАСП остварује нижи заостатак у реду по сва три показатеља. Интеграл заостатка AUC_Q виши је код КЕДА за 24,0% (437.085 наспрам 352.414 $msg \cdot s$), а просечни заостатак за исти проценат (132,25 наспрам 106,63 порука). Ова метрика агрегира целокупно кумулативно оптерећење реда током трајања експеримента, па виша вредност код КЕДА значи да је ред у просеку био дужи. Узрок је директан: ХАСП у овом сценарију одржава већи просечан број реплика (видети потпоглавље о метрикама скалирања), чиме повећава укупни капацитет обраде



Слика 5.2: ХАСП - динамика заостатка реда и броја реплика

и тиме скраћује ред. КЕДА, насупротив томе, подиже реплике тек када ред порука пређе циљну вредност по реплици, па допушта да се ред привремено накупи пре него што реагује. Стога нижи заостатак ХАСП-а не треба тумачити као општу предност, већ као последицу већег утрошка ресурса, што се квантификује у наставку.

Табела 5.1: Метрике реда порука - план са наглим скоковима оптерећења. Приказане су средње вредности преко поновљених извршавања са стандардном девијацијом.

Механизам	Q_{max}	$\bar{Q}$	$AUC_Q (msg \cdot s)$
КЕДА	1.049 ± 25	$132,25 \pm 8,23$	437.085 ± 27.195
ХАСП	996 ± 10	$106,63 \pm 1,45$	352.414 ± 4.784

Метрике латенције и прекорачења нивоа услуге

Резултати у табелама 5.2 и 5.3 показују да су просечне латенције два механизма у плану са наглим скоковима блиске, при чему ХАСП има благу предност у просечним вредностима, а КЕДА у укупном времену прекорачења нивоа услуге. Просечна латенција p_{95} виша је код КЕДА за 7,5% (43,45 s наспрам 40,42 s), а просечна p_{99} за 3,9%. Истовремено, укупно време прекорачења дефинисаног нивоа услуге краће је код КЕДА за 4,3% за p_{95} (1.418 s

наспрам 1.482 s) и за $7,1\%$ за $p99$. Треба нагласити да су апсолутне латенције у овом плану реда величине неколико десетина секунди, што показује да је при врхунцу оптерећења систем у оба случаја био у режиму засићења (тражња превазилази капацитет од осам реплика), па ове вредности описују релативну способност апсорпције заостатка под засићењем, а не одржавање ниског времена одзива. Веће стандардне девијације максималних латенција код КЕДА (табела 5.2) указују на то да КЕДА у појединим извршавањима дозвољава краткотрајне, изразитије пикове када се ред нагло накупи пре реакције скалера.

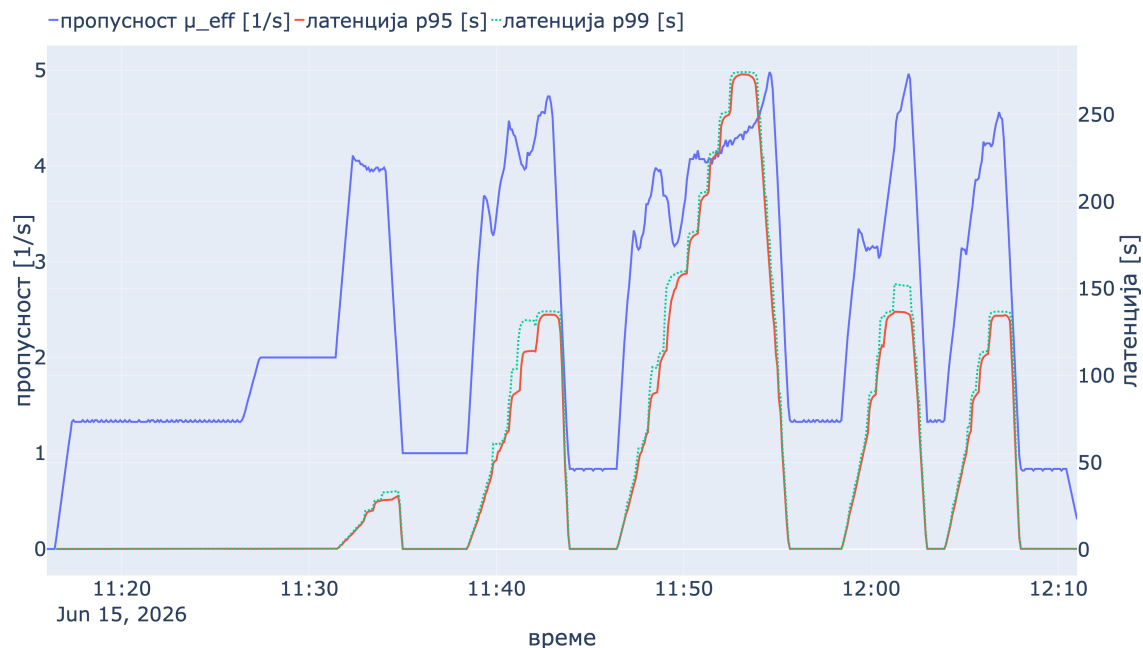
Табела 5.2: Метрике латенције - план са наглим скоковима оптерећења. Средње вредности са стандардном девијацијом.

Механизам	$\overline{p95}\text{ (s)}$	$p95_{max}\text{ (s)}$	$\overline{p99}\text{ (s)}$	$p99_{max}\text{ (s)}$
КЕДА	$43,45 \pm 3,73$	$299,52 \pm 50,26$	$46,10 \pm 3,76$	$305,48 \pm 54,16$
ХАСП	$40,42 \pm 1,81$	$233,67 \pm 8,46$	$44,35 \pm 1,42$	$237,32 \pm 11,72$

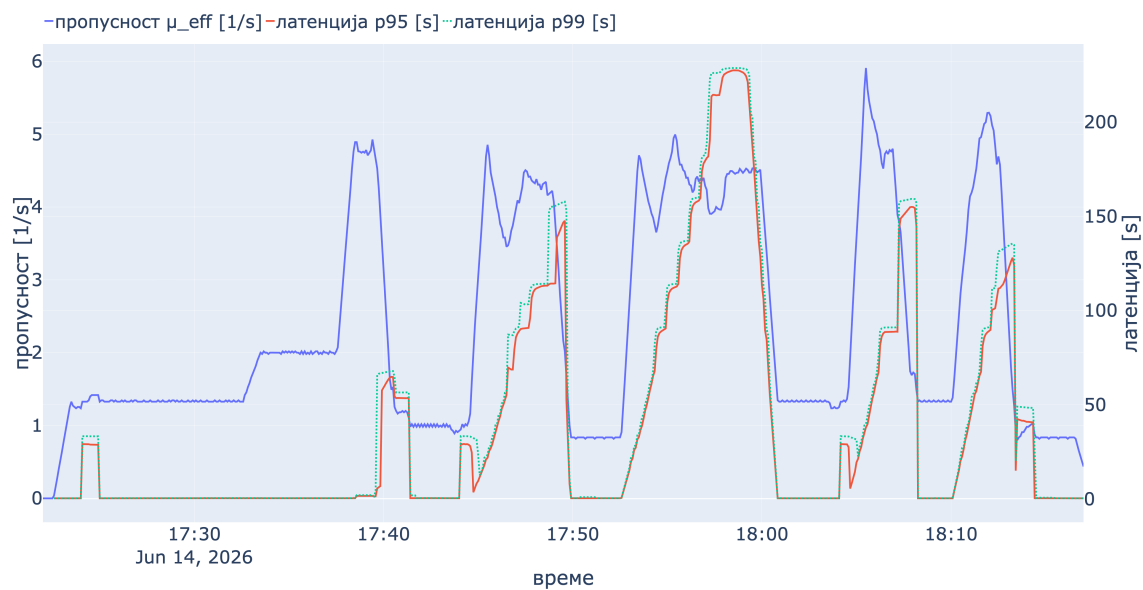
Табела 5.3: Време прекорачења дефинисаног нивоа услуге ($p95 > 2\text{ s}$) - план са наглим скоковима оптерећења. Средње вредности са стандардном девијацијом.

Механизам	$T_{SLA}\text{ p95 (s)}$	$T_{SLA}\text{ p99 (s)}$
КЕДА	1.418 ± 37	1.436 ± 39
ХАСП	1.482 ± 39	1.546 ± 32

На сликама 5.3 и 5.4 је видљиво да код оба механизма латенција расте у тренуцима када пропусност опада, односно да долазна стопа захтева превазилази тренутну сервисну стопу ($\lambda > \mu$, потпоглавље 3.5). Код КЕДА, пикови латенције су нижи и краћег трајања, јер механизам брже повећава сервисну стопу подизањем броја реплика. Код ХАСП механизма ситуација је супротна: јављају се виши и дужи пикови, а у одређеним фазама систем задржава повишену латенцију и у периодима ниске пропусности, јер реплике нису правовремено смањене ни подигнуте у складу са стварним стањем реда.



Слика 5.3: КЕДА - Пропусност и p95/p99 латенција у времену - план са наглим скоковима оптерећења



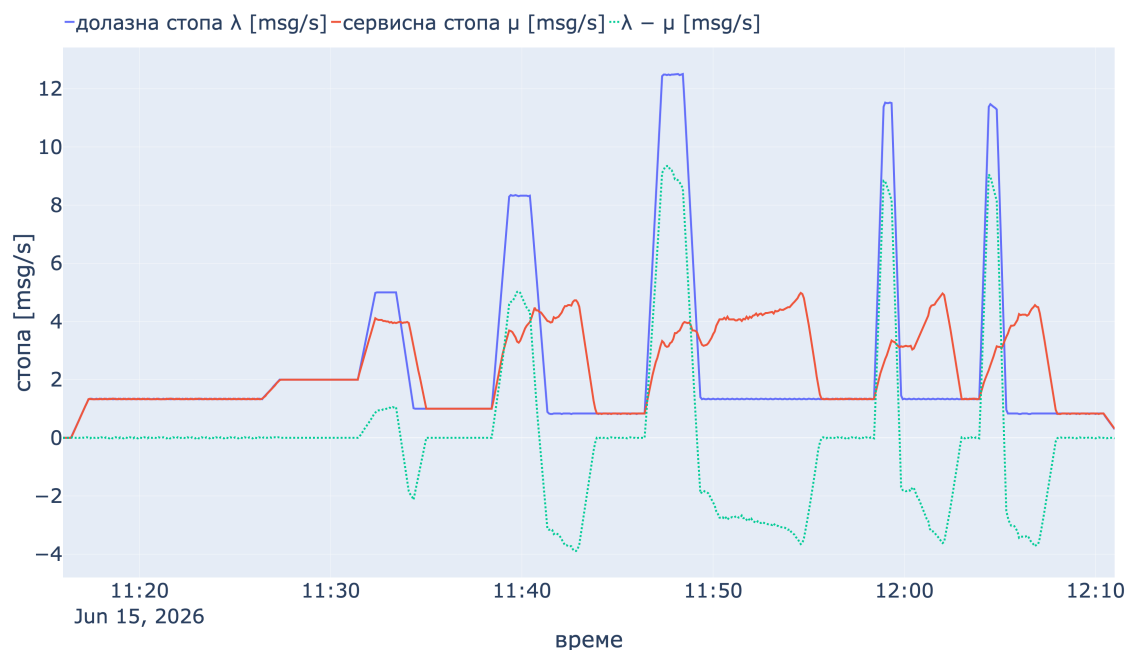
Слика 5.4: ХАСП - Пропусност и p95/p99 латенција у времену - план са наглим скоковима оптерећења

Долазна и сервисна стопа

Слике 5.5 и 5.6 приказују однос долазне стопе λ и укупне сервисне стопе μ_{total} . Разлика $\lambda - \mu_{total}$ (испрекидана линија на сликама) директно описује

тренутак у коме ред почиње да расте ($\lambda > \mu$) или да се празни ($\mu > \lambda$). Механизам који брже подиже број реплика раније враћа систем у режим $\mu \geq \lambda$ и тиме скраћује период накупљања заостатка.

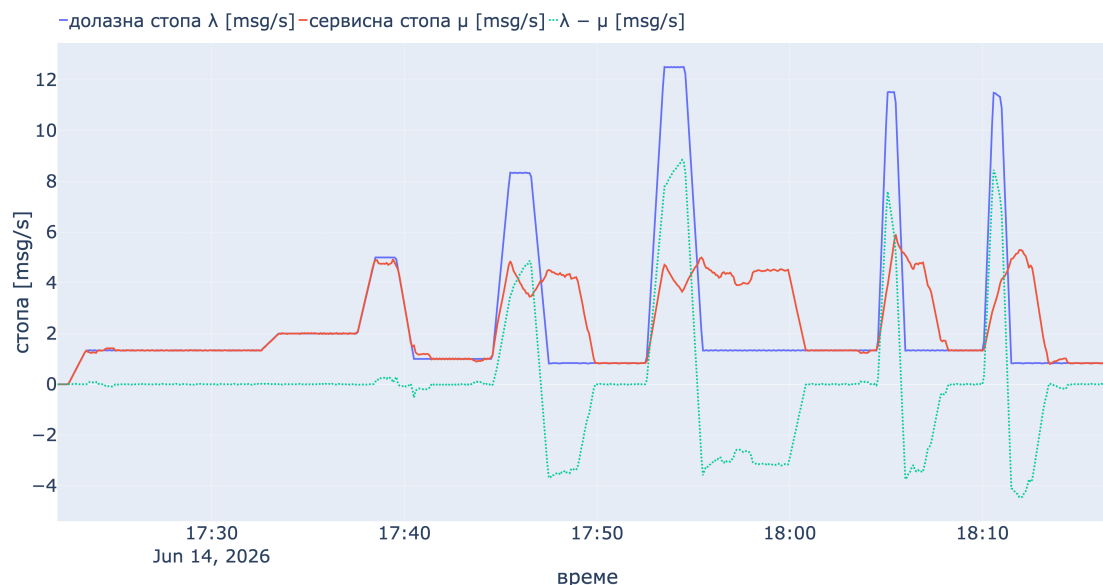
Код КЕДА-е (слика 5.5) систем реагује директно на стање реда: када ред пређе циљну вредност по реплици, капацитет се брзо подиже, што се види по кратком и енергичном опоравку сервисне стопе. Код ХАСП-а (слика 5.6) сервисна стопа је у просеку нешто виша јер ХАСП одржава већи број реплика, али реакција на саме прелазе оптерећења касни, јер сигнал искоришћености процесорске јединице не приказује стање реда на исти начин на који то чини директно мерење дубине реда. Резултат је да ХАСП већим бројем реплика одржава краћи ред, док КЕДА истим послом управља са мање реплика уз привремено дуже задржавање у реду.



Слика 5.5: КЕДА - долазна стопа λ и сервисна стопа μ_{total} - план са наглим скоковима оптерећења.

Метрике скалирања и индекс ефикасности аутоскалирања

Табела 5.4 открива у чему лежи предност КЕДА у овом сценарију. Иако КЕДА допушта виши заостатак у реду, она то постиже са знатно мање ре-



Слика 5.6: ХАСП - долазна стопа λ и сервисна стопа μ_{total} - план са наглим скоковима оптерећења.

плика: просечан број реплика нижи је за 30,4% (2,09 наспрам 3,00), односно ХАСП у просеку користи 1,44 пута више реплика за исту пропусност. Просечна пропусност оба механизма је практично идентична (око 2,36 порука у секунди), што значи да оба обављају исти посао, али КЕДА то ради ресурсно ефикасније. То се директно одражава на пропусност по реплици¹, која је код КЕДА-е виша за 63,0% (1,49 наспрам 0,92), што показује да свака КЕДА реплика носи интензивнији рад, а да систем не одржава велики број слабо оптерећених реплика. Композитни индекс ефикасности аутоскалирања (АЕИ), који кажњава висок број реплика, виши је код КЕДА за 21,7%, чиме се потврђује повољнији компромис између квалитета услуге и утрошка ресурса.

Табела 5.4: Метрике скалирања - план са наглим скоковима оптерећења. Средње вредности са стандардном девијацијом.

Механизам	$\bar{N}$	N_{max}	μ_{max} (1/s)	$\bar{\mu}_N$ (1/s/N)	t_{ttcc} (s)	АЕИ
КЕДА	2,09 ± 0,06	8,0	5,31	1,49	25,6 ± 21,6	7,76 × 10⁻¹⁰
ХАСП	3,00 ± 0,06	6,8	5,86	0,92	65,4 ± 31,6	6,38 × 10 ⁻¹⁰

Поред уштеде ресурса, КЕДА у овом сценарију остварује и знатно бржу

¹Пропусност по реплици израчуната је као количник просечне пропусности и просечног броја реплика током трајања експеримента: $\bar{\mu}_{rep} = \bar{\mu}_{total}/\bar{N}$. Виша вредност указује да свака инстанца у просеку обрађује интензивнији ток захтева.

реакцију. Просечно време до ефективног повећања капацитета (t_{ttec}) краће је код КЕДА за 60,9% (25,6 s наспрам 65,4 s), односно КЕДА у просеку 2,56 пута брже доводи нови капацитет у радно стање. Ово је директна последица тога што КЕДА реагује чим ред почне да расте, док ХАСП мора да сачека да се оптерећење процесорске јединице најпре манифестује и стабилизује пре него што иницира скалирање. Веће стандардне девијације t_{ttec} код оба механизма ($\pm 21\text{--}32\text{ s}$) указују на то да тренутак реакције зависи од фазе у којој скок наступи, али је средња предност КЕДА доследна преко поновљених извршавања.

Реактивност и догађаји скалирања

Слике 5.7 и 5.8 приказују тренутке промене броја реплика у односу на кретање пропусности. Вертикалне испрекидане линије означавају тренутке у којима је аутоskalер донео одлуку о промени броја реплика. Код КЕДА (слика 5.7) уочава се да се скалирање активира у групама, у моментима наглог пораста оптерећења, а број реплика (исцртан степенасто, десна оса) прати облик пропусности са умереном латенцијом. Систем достиже максималних 8 реплика у највишем таласу оптерећења, а потом постепено смањује број инстанци.

Код ХАСП (слика 5.8) уочава се специфичност стратегије засноване на оптерећењу процесорске јединице. У неким фазама реплике остају на вишем нивоу дуже него што оптерећењем захтева (нпр. раздобља равне линије реплика при ниској пропусности), а у другима систем касни са достизањем потребног броја реплика у тренуцима наглог пораста. Максимум је достигнут на 7 реплика, јер оптерећење процесора није захтевало осму реплику.

Закључак потпоглавља

Резултати за план са наглим скоковима оптерећења показују да два механизма заузимају различите тачке у компромису између квалитета реда и утрошка ресурса. ХАСП одржава нижи заостатак у реду (интеграл заостатка нижи за 24,0%) и нешто нижу просечну латенцију (за 7,5% на $p95$), али то постиже већим бројем реплика (1,44 пута више у просеку). КЕДА, са друге стране, истим бројем обрађених захтева управља уз 30,4% мање реплика, остварује 63,0% вишу пропусност по реплици, 21,7% виши композитни индекс

Основни разлог ове разлике лежи у природи сигнала скалирања. КЕДА директно мери дубину реда порука и може иницирати скалирање чим ред почне да расте, што објашњава бржи одзив и ресурсну ефикасност, али уз дозвољено привремено накупљање реда пре реакције. ХАСП зависи од искоришћености процесорске јединице; пошто превентивно одржава већи број реплика, успева да одржи краћи ред и нешто нижу просечну латенцију, али уз већи и нестабилнији утрошак ресурса. Дакле, у плану са наглим скоковима не постоји једнозначан победник: избор механизма зависи од тога да ли се приоритет даје минималном заостатку у реду (ХАСП) или ресурсној ефикасности и брзини одзива (КЕДА).

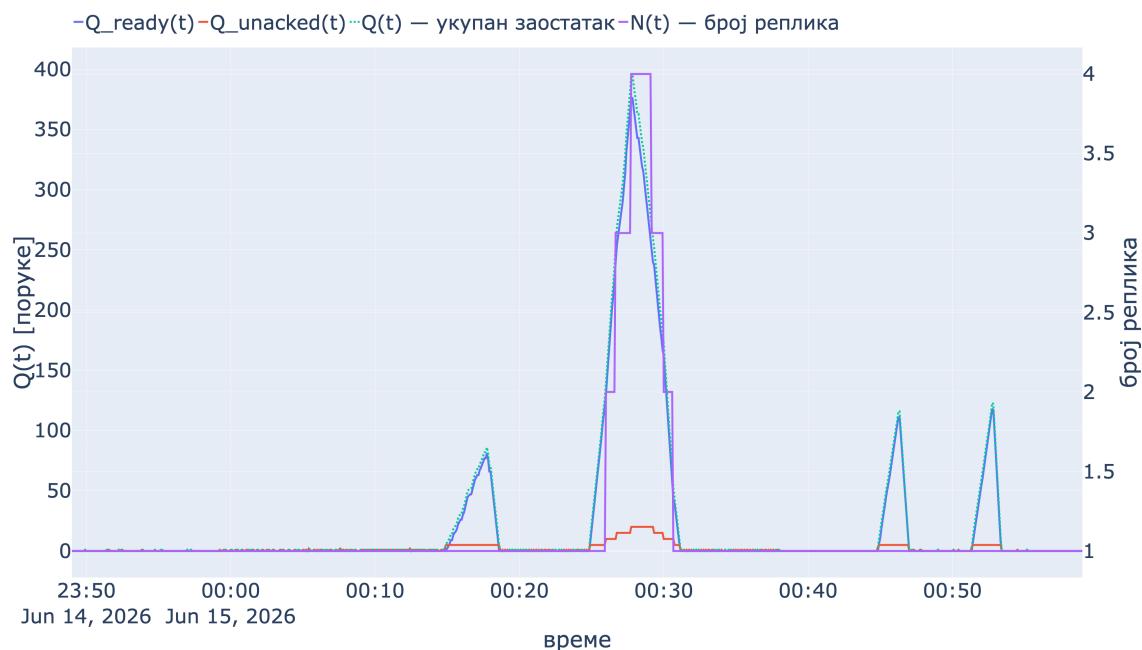
5.2 Резултати за план са постепеним порастом оптерећења

Овај план моделује сценарио у коме оптерећење расте постепено кроз низ фаза различитог интензитета, са раздобљима задржавања на средњим и високим нивоима, те фазама смиривања и малих скокова. Сценарио је методолошки значајан јер ХАСП механизму даје шансу да се процесорска јединица стабилизује, па се очекује да ХАСП буде конкурентнији него у плану са наглим скоковима. Кључно питање је: *да ли сигнал искоришћености процесорске јединице, чак и у стабилнијем профилу оптерећења, верно одражава накупљени посао у реду?*

Динамика заостатка у реду и броја реплика

Слике 5.9 и 5.10 откривају најупечатљивији налаз овог сценарија. Код КЕДА механизма (слика 5.9) систем готово цело трајање експеримента ради са свега једном репликом (просечно 1,18 реплика), а скалирање се активира само у тренуцима када ред порука јасно сигнализира потребу. Максимум од око четири реплике достиже се само у највишој фази оптерећења, а потом се систем потпуно опоравља на једну реплику. Просечан пик заостатка износи око 439 порука, а систем се по завршетку оптерећења у потпуности враћа у стационарно стање.

Код ХАСП механизма понашање је суштински другачије, јер систем одржава две до шест реплика кроз готово цео експеримент, укључујући и фазе

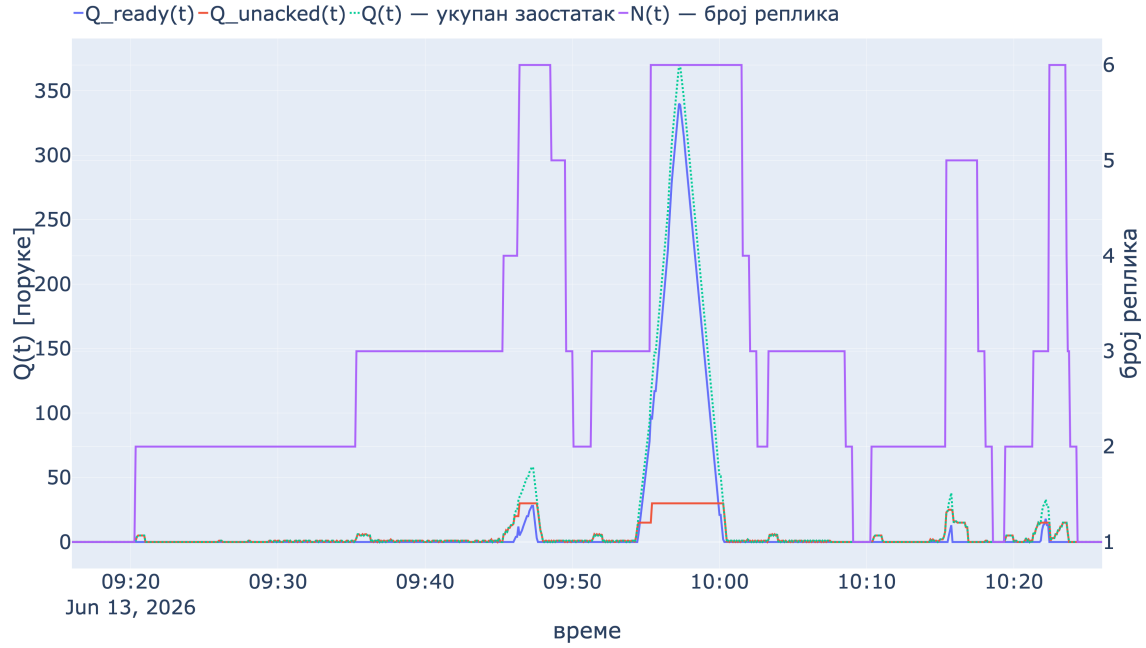


Слика 5.9: КЕДА - динамика заостатка реда и броја реплика

у којима је стварно оптерећење ниско. Сигнал процесорске јединице наводи ХАСП да нагло подигне број реплика, а потом их задржи дуго након смањења оптерећења. ХАСП у просеку одржава 2,49 пута више реплика него КЕДА (2,93 наспрам 1,18). Та обилна резерва капацитета доводи до тога да ХАСП одржава нижи заостатак у реду: просечан пик заостатка код ХАСП-а (370 порука) нижи је за око 16% него код КЕДА, а интеграл заостатка нижи за 44% (видети наставак). Централни налаз овог сценарија стога није да један механизам доминира, већ да ХАСП минималан заостатак купује великим и нестабилним утрошком реплика, док КЕДА истоветну пропусност одржава са свега делићем тог броја реплика, прихватајући нешто дужи ред као компромис.

Метрике реда порука

Табела 5.5 показује да у постепеном профилу ХАСП остварује нижи заостатак у реду по свим показатељима. Интеграл заостатка AUC_Q виши је код КЕДА за 44,5% (118.246 наспрам 81.859 $msg \cdot s$), а просечни заостатак за 45,6% (28,12 наспрам 19,31 порука). Овај резултат је доследан налазу из плана са наглим скоковима (где је разлика AUC_Q износила 24,0%) и потврђује



Слика 5.10: ХАСП - динамика заостатка реда и броја реплика

да механизам заснован на реду по правилу допушта дужи ред него механизам који превентивно одржава вишак реплика. Веће стандардне девијације код КЕДА-е (на пример ± 29.840 за AUC_Q) указују на то да КЕДА у појединим извршавањима задржи ред нешто дуже, у зависности од тренутка када праг по реплици буде пређен. Кључно је, међутим, што се овај виши заостатак не преводи у вишу латенцију. Као што показује наредно потпоглавље, КЕДА у овом плану остварује *нижу* латенцију уз вишеструко мање реплика.

Табела 5.5: Метрике реда порука - план са постепеним порастом оптерећења. Средње вредности са стандардном девијацијом.

Механизам	Q_{max}	$\bar{Q}$	$AUC_Q (msg \cdot s)$
КЕДА	439 ± 51	$28,12 \pm 7,10$	118.246 ± 29.840
ХАСП	370 ± 16	$19,31 \pm 1,65$	81.859 ± 6.171

Метрике латенције и прекорачења нивоа услуге

Разлика у латенцији (табела 5.6) у овом сценарију иде у корист КЕДА, упркос томе што КЕДА одржава дужи ред. Просечна $p95$ латенција нижа је код КЕДА за 8,7% ($9,18s$ наспрам $10,05s$), а просечна $p99$ за чак 28,2%

(10,44 s наспрам 14,54 s). И максималне латенције су ниже код КЕДА (за 7-8%). Укупно време прекорачења нивоа услуге је за $p95$ практично изједначено (1.040 s наспрам 1.010 s, +3,0%), али за $p99$ краће код КЕДА за 13,2% (1.051 s наспрам 1.211 s). Овај налаз је на први поглед противречан: КЕДА одржава дужи ред, али нижу латенцију. Објашњење лежи у томе што КЕДА сваку поруку пушта у обраду на репликама које раде близу свог капацитета, без задржавања у фази распоређивања, док ХАСП, иако одржава краћи ред, дистрибуира посао преко већег броја слабије оптерећених реплика, при чему повремено касни у прелазним фазама, што производи изражене пикове $p99$.

Табела 5.6: Метрике латенције - план са постепеним порастом оптерећења. Средње вредности са стандардном девијацијом.

Механизам	$\overline{p95}$ (s)	$p95_{max}$ (s)	$\overline{p99}$ (s)	$p99_{max}$ (s)
КЕДА	9,18 ± 2,50	108,55 ± 15,47	10,44 ± 2,91	119,45 ± 17,27
ХАСП	10,05 ± 0,47	117,76 ± 36,69	14,54 ± 0,82	129,31 ± 37,66

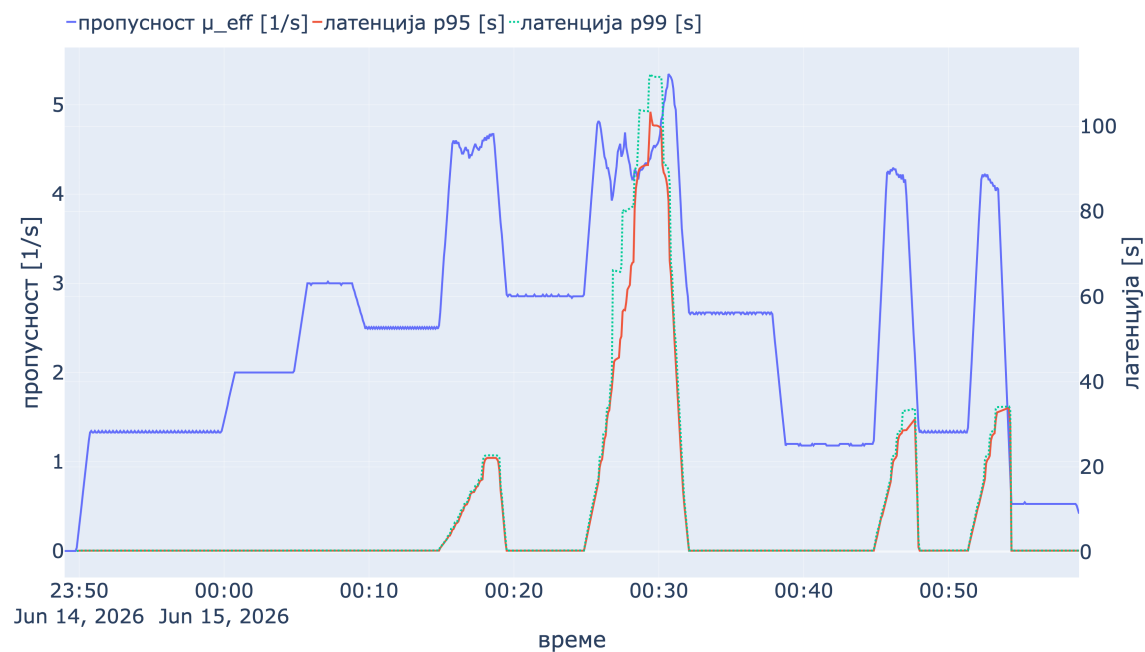
Табела 5.7: Време прекорачења дефинисаног нивоа услуге ($p95 > 2$ s) - план са постепеним порастом оптерећења. Средње вредности са стандардном девијацијом.

Механизам	$T_{SLA} p95$ (s)	$T_{SLA} p99$ (s)
КЕДА	1.040 ± 108	1.051 ± 108
ХАСП	1.010 ± 34	1.211 ± 56

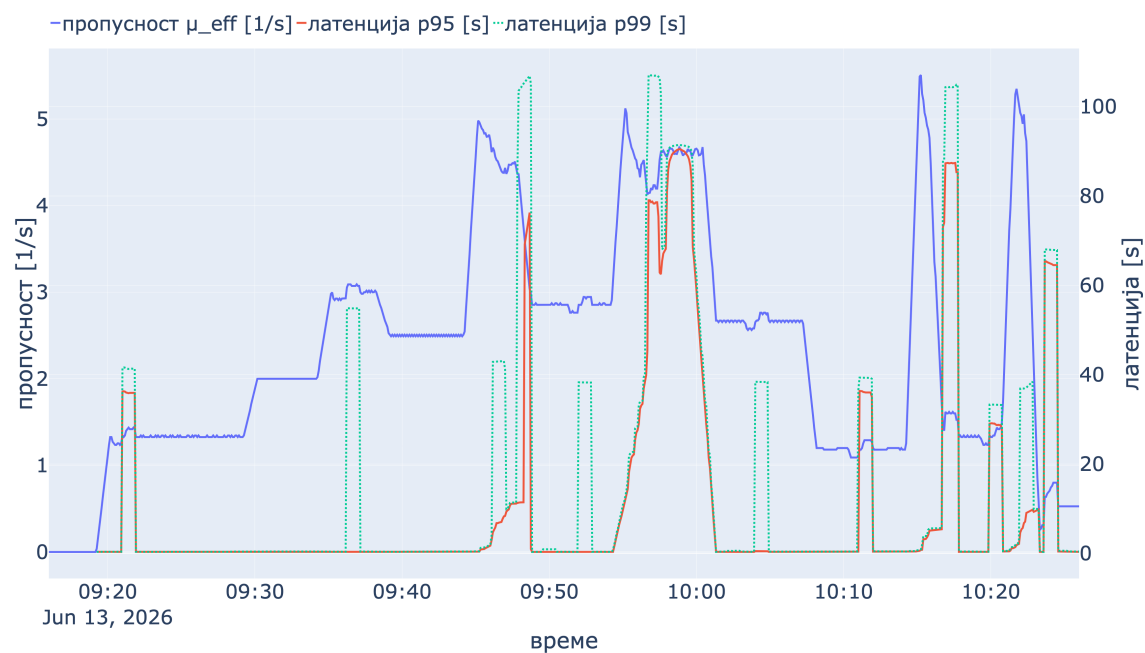
Слике 5.11 и 5.12 илуструју ову разлику у временском домену. Код КЕДА-е (слика 5.11) латенција расте само у кратком периоду максималног оптерећења, а потом се брзо враћа у безбедну зону. Код ХАСП-а (слика 5.12) уочљиви су вишеструки пикови латенције, распоређени кроз цело трајање експеримента, укључујући и фазе у којима пропусност није нарочито висока. Пикови су последица касног реаговања система који не успева да у потпуности испразни ред пре следећег пораста оптерећења. Ово накупљање заостатка кроз фазе посебно је карактеристично за ХАСП у постепеном профилу. Нагли скокови не постоје, али сигнал процесорске јединице не прати ред довољно брзо да спречи нагомилавање кашњења у прелазним фазама.

Долазна и сервисна стопа

Слике 5.13 и 5.14 приказују однос λ и μ_{total} кроз време. Код КЕДА-е (слика 5.13) уочљиво је да сервисна стопа прилично верно прати долазну стопу, а

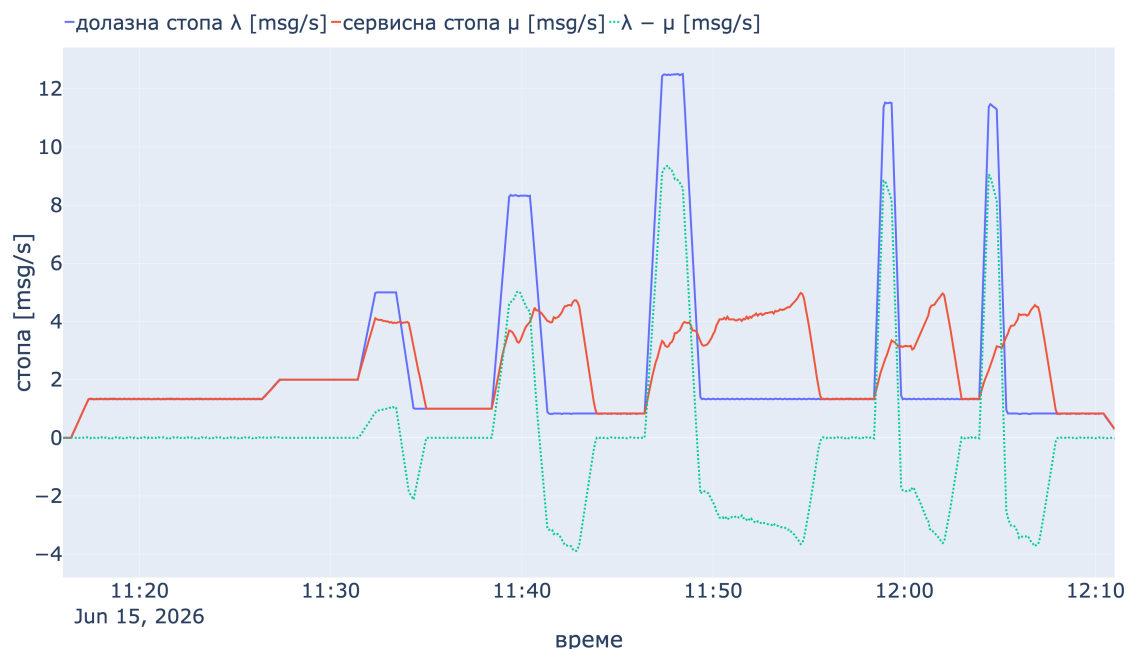


Слика 5.11: КЕДА - пропусност и p95/p99 латенција у времену - план са постепеним порастом оптерећења



Слика 5.12: ХАСП - пропусност и p95/p99 латенција у времену - план са постепеним порастом оптерећења

периоди позитивне разлике $\lambda - \mu > 0$ су краћи и мање изражени. Систем брже успоставља однос $\mu \geq \lambda$ захваљујући директном сигналу реда.

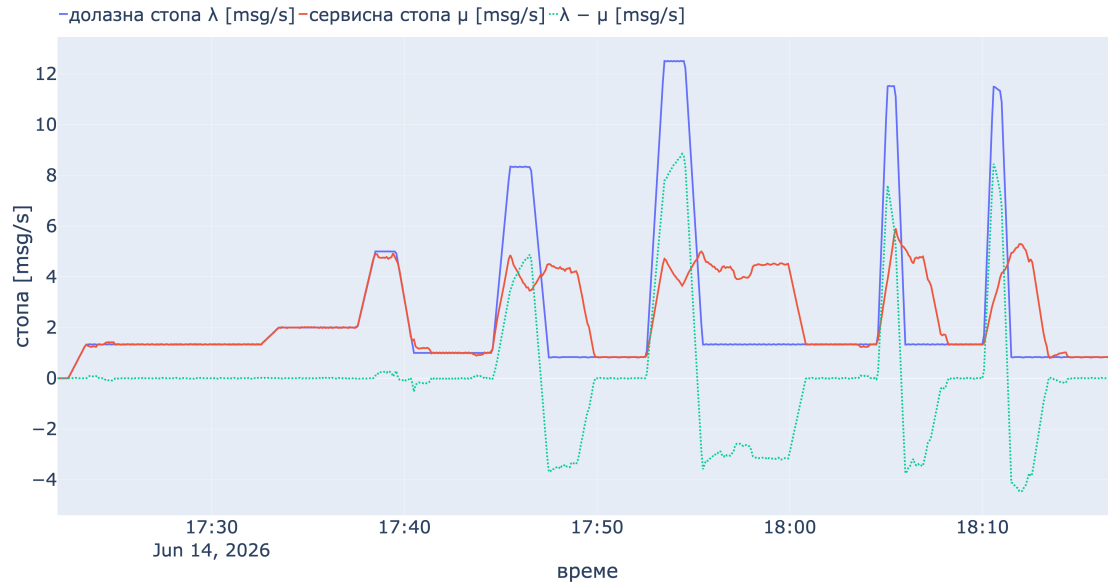


Слика 5.13: КЕДА - долазна стопа λ и сервисна стопа μ_{total} - план са постепеним растом оптерећења.

Код ХАСП-а (слика 5.14) сервисна стопа је такође висока, јер ХАСП одржава велики број реплика, па се ред у просеку држи краћим. Међутим, већи капацитет се не користи ефикасно: реплике су слабо оптерећене, а у прелазним тачкама фаза оптерећења ХАСП повремено касни са прилагођавањем, што производи пикове $p99$ латенције. КЕДА истоветну пропусност одржава са вишеструко мање реплика, прихватајући нешто дужи ред, али без губитка у латенцији.

Метрике скалирања и индекс ефикасности аутоскалирања

Табела 5.8 показује најизразитију разлику у ефикасности ресурса од свих посматраних сценарија. КЕДА у просеку користи 2,49 пута мање реплика него ХАСП (1,18 наспрам 2,93), а истовремено остварује практично идентичну просечну пропусност (2,38 наспрам 2,36 порука у секунди). Последица је да је пропусност по реплици код КЕДА-е виша за 153,6% (2,15 наспрам



Слика 5.14: ХАСП - долазна стопа λ и сервисна стопа μ_{total} - план са постепеним порастом оптерећења.

0,85), односно свака КЕДА реплика обрађује у просеку 2,5 пута интензивнији ток захтева. Поред тога, КЕДА скалира знатно стабилније: стандардна девијација броја реплика (мера осцилације) нижа је за 60,8% (0,63 наспрам 1,61), што значи да КЕДА избегава често подизање и спуштање реплика карактеристично за ХАСП у овом профилу.

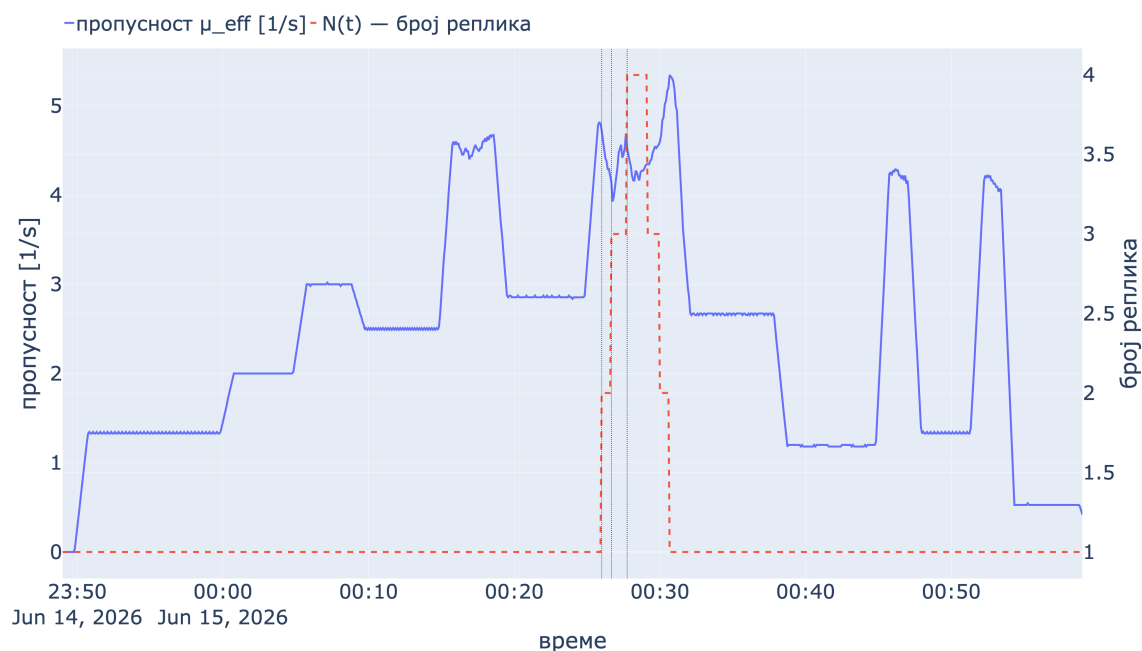
Индекс ефикасности аутоскалирања (АЕИ) код КЕДА-е је виши за 84,0% у поређењу са ХАСП-ом ($7,65 \times 10^{-9}$ наспрам $4,16 \times 10^{-9}$). Иако КЕДА има виши кумулативни заостатак, њена далеко мања потрошња реплика и нижа p_{99} латенција превагну у индексу аутоскалирања, што потврђује да КЕДА остварује повољнији укупни компромис између квалитета услуге и утрошка ресурса.

Табела 5.8: Метрике скалирања - план са постепеним порастом оптерећења. Средње вредности са стандардном девијацијом.

Механизам	$\bar{N}$	N_{max}	μ_{max} (1/s)	$\bar{\mu}_N$ (1/s/N)	АЕИ
КЕДА	$1,18 \pm 0,06$	4,4	5,45	2,15	$7,65 \times 10^{-9}$
ХАСП	$2,93 \pm 0,05$	6,4	5,69	0,85	$4,16 \times 10^{-9}$

Реактивност и догађаји скалирања

Слике 5.15 и 5.16 визуелно показују суштинску разлику у стратегији скалирања. Код КЕДА-е (слика 5.15) евидентирана су свега три догађаја скалирања током целог експеримента, сва три концентрисана у кратком периоду максималног оптерећења. Систем остаје на једној реплици пре и после тог периода, а пропусност прати оптерећење без видљивих осцилација у броју реплика.

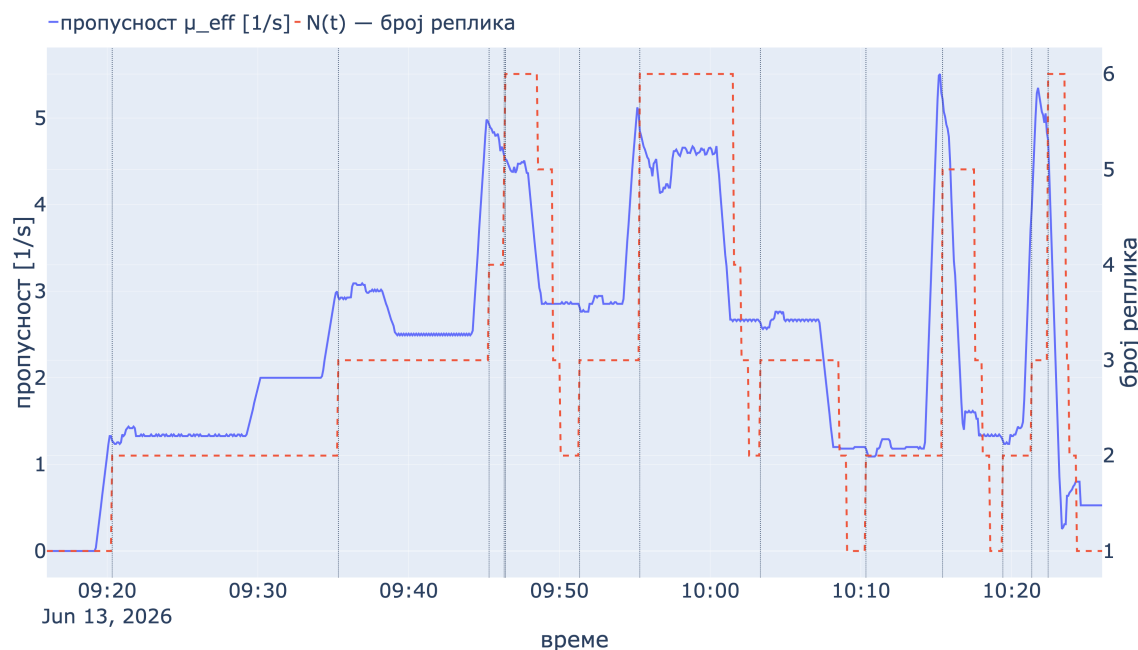


Слика 5.15: КЕДА - догађаји скалирања (испрекидане вертикалне линије) и пропусност у времену - план са постепеним порастом оптерећења.

Код ХАСП-а (слика 5.16) евидентирано је тринаест догађаја скалирања, распоређених кроз цео експеримент. Број реплика осцилира у широком распону (од једне до шест), при чему су видљива дуга раздобља са шест реплика, и то у периодима умереног оптерећења.

Закључак потпоглавља

Резултати за план са постепеним порастом оптерећења показују да КЕДА у овом профилу остварује изразиту предност у ресурсној ефикасности и стабилности, уз истовремено бољу латенцију, али по цену вишег заостатка у реду. КЕДА користи 2,49 пута мање реплика у просеку, скалира 2,55 пута



Слика 5.16: ХАСП - догађаји скалирања (испрекидане вертикалне линије) и пропусност у времену - план са постепеним порастом оптерећења.

стабилније (мања осцилација броја реплика), остварује 8,7% нижу просечну $p95$ и 28,2% нижу просечну $p99$ латенцију и 84,0% виши композитни индекс ефикасности. Истовремено, ХАСП одржава нижи кумулативни заостатак у реду (за 44,5%), али то постиже трајним одржавањем великог броја слабо оптерећених реплика.

Овај налаз делимично потврђује полазну хипотезу да би ХАСП могао бити конкурентнији у стабилнијем профилу: ХАСП заиста овде остварује најнижи заостатак у реду од свих сценарија. Међутим, та предност долази уз знатно већи и нестабилнији утрошак ресурса, без побољшања латенције, при чему ХАСП има вишу просечну $p99$ латенцију. Сигнал искоришћености процесорске јединице наводи ХАСП да превентивно и реактивно одржава велики број реплика и да честим скалирањем осцилује у прелазним фазама, док КЕДА реагује циљано на стварно стање реда и тиме истоветну пропусност и латенцију постиже са делићем ресурса. Стога ни у постепеном профилу не постоји једнозначан победник: ХАСП оптимизује дужину реда, КЕДА оптимизује ефикасност и стабилност уз очување латенције.

5.3 Анализа осетљивости циљних прагова

У претходним потпоглављима установљено је да два механизма заузимају различите тачке у компромису између квалитета услуге и утрошка ресурса, при чему КЕДА троши знатно мање реплика. Природно се намећу два питања осетљивости. Прво: *да ли би виши циљни праг искоришћености процесорске јединице код ХАСП-а (нпр. 70% уместо 60%) приближио ХАСП ресурсној ефикасности КЕДА?* Друго: *колико је понашање КЕДА осетљиво на избор циљне вредности реда по реплици?* Прво питање испитано је у почетној фази експеримената поређењем ХАСП конфигурација са циљем од 60% и 70% над планом са постепеним порастом, а друго систематским варирањем КЕДА прага (60, 80 и 100 порука по реплици) над планом са наглим скоковима, при чему је сваки сценарио поновљен пет пута.

Осетљивост ХАСП прага (60% наспрам 70%)

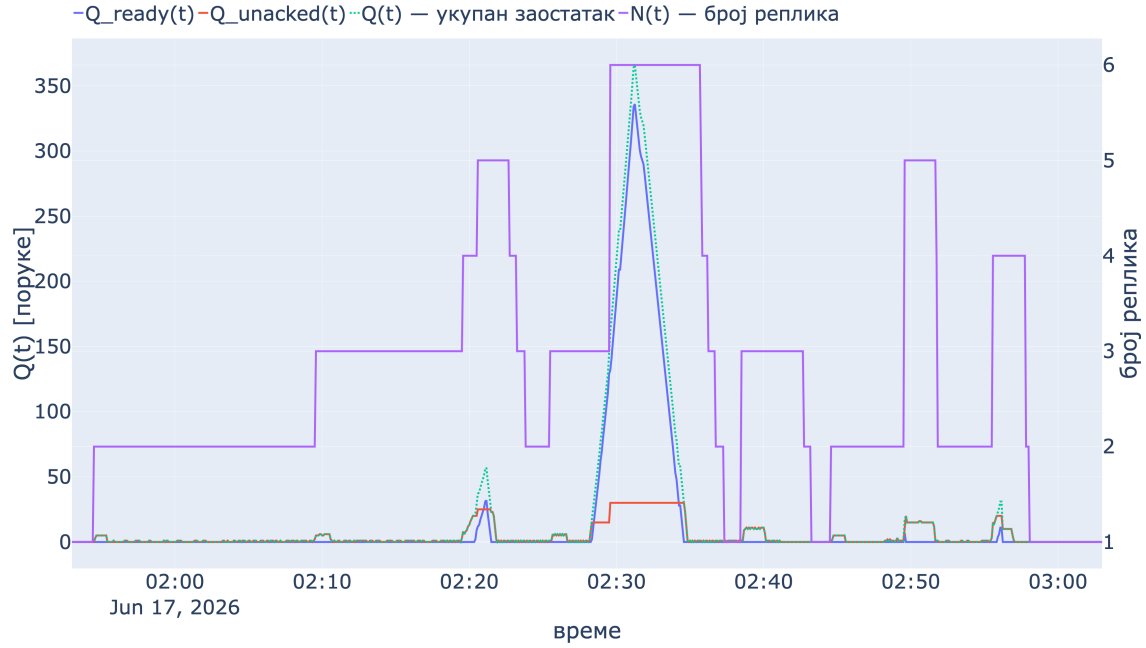
Ради одговора на прво питање, изведен је додатни експеримент са ХАСП конфигурацијом са циљем од 70% над истим планом са постепеним порастом оптерећења, поновљен пет пута. Сви остали параметри задржани су непромењени, а резултати су приказани као средње вредности са стандардном девијацијом.

Динамика заостатка у реду и броја реплика

На слици 5.17 се може видети кретање броја порука у реду и броја реплика. У табели 5.9 види се да подизање прага са 60% на 70% доводи до врло благе уштеде реплика (4,0%, са 2,93 на 2,81 у просеку) и до незнатне промене заостатка у реду, при чему је максимални заостатак готово је идентичан, а просечни заостатак и интеграл заостатка чак благо расту (за око 5%). Промене су по већини показатеља реда у оквиру или близу стандардне девијације, што указује да подизање прага не доноси значајну ресурсну уштеду нити мења понашање реда.

Квантитативно поређење ХАСП 60% и 70%

Табела 5.10 садржи додатне метрике и открива кључни исход подизања прага: ХАСП са прагом од 70% не побољшава квалитет услуге, већ га по-



Слика 5.17: ХАСП 70% - динамика заостатка реда и броја реплика

Табела 5.9: Метрике реда порука и реплика - ХАСП 60% и 70%, план са постепеним порастом оптерећења. Средње вредности са стандардном девијацијом.

Метрика	ХАСП 60%	ХАСП 70%	Промена
Q_{max}	370 ± 16	369 ± 3	-0,5%
$\bar{Q}$	19,31 ± 1,65	20,43 ± 0,62	+5,8%
AUC_Q	81.859 ± 6.171	85.906 ± 2.635	+4,9%
N_{max}	6,4	6,0	-6,3%
$\bar{N}$	2,93 ± 0,05	2,81 ± 0,04	-4,0%

горшава. Просечна $p95$ латенција расте за 22,6% (12,32 s наспрам 10,05 s), просечна $p99$ за 18,2%, а максимална $p95$ за 22,4%. Истовремено, композитни индекс ефикасности практично се не мења (благи пад од 1,8%), јер се ситна уштеда реплика поништава вишом латенцијом и већим заостатком.

Механизам ове зависности је следећи. Виши праг искоришћености доводи до тога да ХАСП касни са подизањем броја реплика, јер систем чека дуже да искоришћеност процесорске јединице достигне 70% пре него што иницира скалирање. У том периоду ред расте, а кад год долазна стопа нагло порасте у прелазним тачкама фаза, систем је у лошијој позицији да апсорбује надолазећи посао, што производи више латенције. Уштеда реплика је минимална (4,0%) јер ХАСП и при вишем прагу мора да одржи сличан капацитет да

Табела 5.10: Додатно поређење ХАСП 60% и ХАСП 70% - план са постепеним порастом оптерећења. Средње вредности са стандардном девијацијом.

Метрика	ХАСП 60%	ХАСП 70%	Промена
$\overline{p95}$	10,05 $\pm$ 0,47	12,32 $\pm$ 1,27	+22,6%
$p95_{max}$	117,76 $\pm$ 36,69	144,14 $\pm$ 2,58	+22,4%
$T_{SLA} p95$	1.010 $\pm$ 34	1.018 $\pm$ 60	+0,8%
$\bar{\mu}_N$	0,847	0,889	+5,0%
AEI	$4,16 \times 10^{-9}$	$4,08 \times 10^{-9}$	-1,8%

би пратио оптерећење. Резултат је да подизање прага не приближава ХАСП ефикасности КЕДА, већ само благо погоршава квалитет услуге уз занемарљиву уштеду ресурса.

Долазна и сервисна стопа

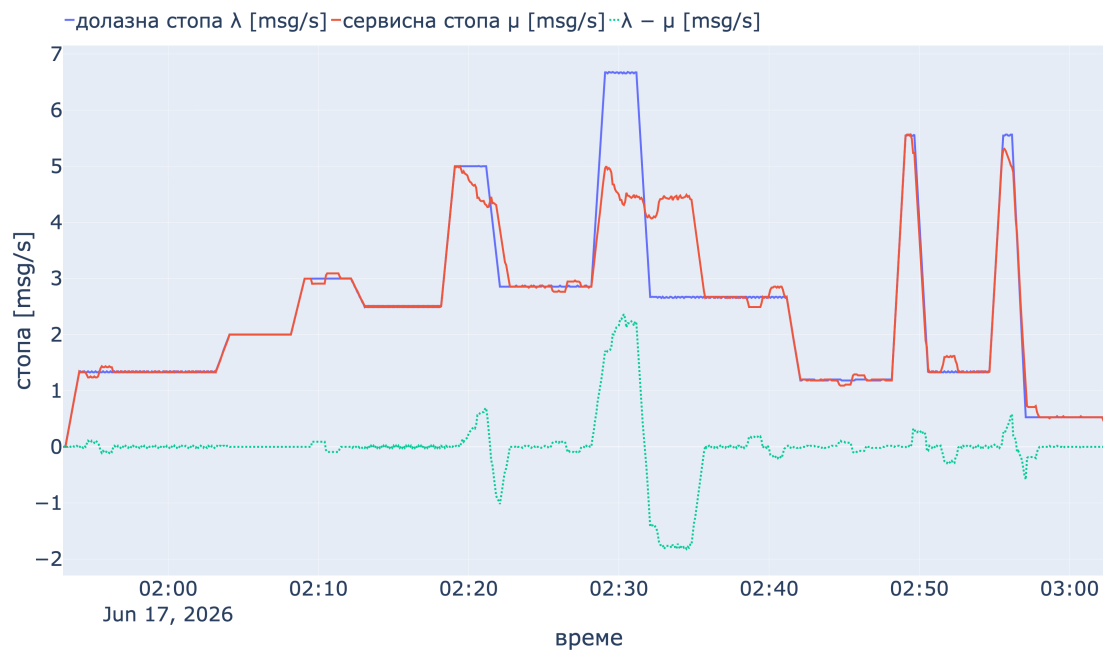
Слика 5.18 и слика 5.14 показују да је облик кривих λ и μ_{total} врло сличан у оба сценарија. У оба случаја уочљиви су периоди у којима важи однос $\lambda > \mu$ у прелазним тачкама фаза оптерећења. Код ХАСП-а са прагом од 70% ти периоди су нешто краћи у неким фазама (мање реплика, бржи раст искоришћености, нешто бржа реакција у неким тренуцима), али у фазама максималног оптерећења разлика $\lambda - \mu$ повремено достиже веће вредности, јер систем почиње са мање реплика и мора да надокнади каскадно. Ово директно доводи до израженијих пикова заостатка и латенције.

Догађаји скалирања

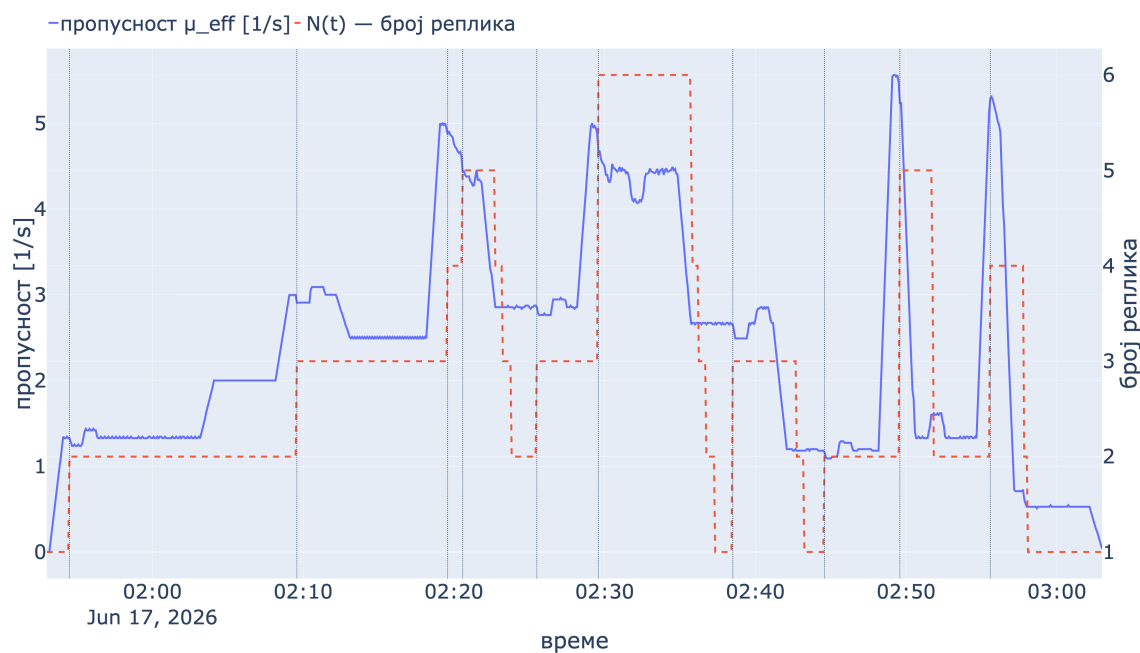
Слика 5.19 илуструје разлику у динамици скалирања. ХАСП 70% генерише десет догађаја скалирања, наспрам тринаест код ХАСП 60% (слика 5.16). ХАСП 70% реагује брже у тренутку покретања скалирања, али бржа реакција не гарантује бољи укупни резултат. Систем са мање реплика у почетном стању улази у фазе оптерећења са мањим капацитетом, па свака латенција у сигналу има пропорционално већи утицај на ред и латенцију.

Поређење са КЕДА

Табела 5.11 обједињује КЕДА (праг 100) са ХАСП конфигурацијама на 60% и 70% над планом са постепеним порастом, све са вишеструким понављањима. Поређење потврђује главни налаз рада: подизање ХАСП прага не



Слика 5.18: ХАСП 70% - долазна стопа λ и сервисна стопа μ_{total} - план са постепеним оптерећења.



Слика 5.19: ХАСП 70% - догађаји скалирања (испрекидане вертикалне линије) и пропусност у времену - план са постепеним порастом оптерећења.

мења природу компромиса између два механизма. КЕДА у односу на ХАСП 60% остварује нижу просечну $p95$ латенцију (за 8,7%), 2,49 пута мање реплика и 84,0% виши композитни индекс, али уз виши кумулативни заостатак у реду (за 44,5%). Подизање ХАСП прага на 70% доноси само минималну уштеду реплика (4,0%), и то уз погоршање латенције, па ХАСП ни приближно не достиже ресурсну ефикасност КЕДА. Другим речима, оба ХАСП прага остају на истој страни компромиса (мањи заостатак, више реплика), док КЕДА заузима супротну (више заостатка, далеко мање реплика и бољи композитни индекс).

Табела 5.11: КЕДА наспрам ХАСП-а 60% и ХАСП-а 70% - план са постепеним порастом оптерећења. Све вредности су средње вредности преко пет до седам понављања.

Метрика	КЕДА	ХАСП 60%	ХАСП 70%	КЕДА наспрам ХАСП-а 60%
AUC_Q (msg s)	118.246	81.859	85.906	+44,5%
$p95$ (s)	9,18	10,05	12,32	-8,7%
$T_{SLA} p95$ (s)	1.040	1.010	1.018	+3,0%
$\bar{N}$	1,18	2,93	2,81	-59,9%
AEI	$7,65 \times 10^{-9}$	$4,16 \times 10^{-9}$	$4,08 \times 10^{-9}$	+84,0%

Закључак о осетљивости ХАСП прага

Анализа осетљивости циљне искоришћености процесорске јединице показује да подизање прага са 60% на 70% не мења природу понашања ХАСП механизма у посматраном систему. Уштеда броја реплика је минимална (4,0%), а праћена је чак благим порастом заостатка и осетним порастом латенције (просечна $p95$ виша за 22,6%), па композитни индекс остаје практично непромењен. То указује да избор конкретне вредности ХАСП прага није кључан фактор: и при 60% и при 70% ХАСП остаје механизам који минималан заостатак постиже превентивним одржавањем већег броја реплика, а његово понашање структурно одређује врста сигнала (искоришћеност процесорске јединице), а не конкретна вредност прага.

Осетљивост КЕДА прага реда по реплици (60/80/100)

Аналогно претходној анализи, испитана је и осетљивост КЕДА механизма на циљну вредност реда по реплици. Над планом са наглим скоковима извршене су три конфигурације: са праговима од 60, 80 и 100 порука по реплици,

при чему је сваки сценарио поновљен пет пута. Резултати су приказани у табели 5.12 као средње вредности са стандардном девијацијом.

Табела 5.12: Осетљивост КЕДА-е на циљну вредност реда по реплици - план са наглим скоковима. Средње вредности са стандардном девијацијом.

Метрика	Праг 60	Праг 80	Праг 100
AUC_Q (msg s)	530.331 $\pm$ 35.895	491.384 $\pm$ 26.491	437.085 $\pm$ 27.195
Q_{max}	1.097 $\pm$ 14	1.072 $\pm$ 6	1.049 $\pm$ 25
$p95$ (s)	58,33 $\pm$ 5,92	53,79 $\pm$ 4,32	43,45 $\pm$ 3,73
T_{SLA} p95 (s)	1.674 $\pm$ 81	1.527 $\pm$ 36	1.418 $\pm$ 37
$\bar{N}$	2,84 $\pm$ 0,09	2,43 $\pm$ 0,07	2,09 $\pm$ 0,06
Осцилација реплика	2,70 $\pm$ 0,05	2,34 $\pm$ 0,05	2,01 $\pm$ 0,07
$\bar{\mu}_N$ (1/s/N)	1,26 $\pm$ 0,02	1,41 $\pm$ 0,02	1,49 $\pm$ 0,01
AEI	$4,01 \times 10^{-10}$	$5,51 \times 10^{-10}$	$7,76 \times 10^{-10}$

Резултати откривају доследан и на први поглед противречан тренд: *виши* праг реда по реплици истовремено побољшава готово све показатеље. Прелазак са прага 60 на праг 100 смањује интеграл заостатка за 17,6%, просечну $p95$ латенцију за 25,5%, просечан број реплика за 26,4% и осцилацију броја реплика за 25,6%, док композитни индекс ефикасности расте за 93,6%. Очекивало би се да нижи праг (агресивније скалирање) даје боље перформансе реда, али се дешава супротно.

Механизам ове зависности лежи у интеракцији прага са динамиком реда и временом покретања реплике. При ниском прагу (60) КЕДА реагује већ на мали пораст реда и често подиже и спушта реплике, што се види по највишој осцилацији броја реплика (2,70). Свако подизање реплике носи трошак иницијализације (време док нова реплика не постане спремна), током кога ред наставља да расте, па честа реакција на ситне промене заправо погоршава укупно понашање. При вишем прагу (100) КЕДА реагује тек када се накупи смисленији заостатак, ради са стабилнијим бројем реплика и сваку реплику користи интензивније (виши пропусност по реплици), што у крајњем збиру даје и нижи заостатак и нижу латенцију и мање ресурса. Овај налаз је практично значајан: он показује да ефикасност КЕДА не зависи од агресивног, ниског прага, већ да умеренији праг даје бољи укупни компромис и потврђује да структурне предности сигнала заснованог на реду нису артефакт једне специфичне вредности прага.

5.4 Преглед свих резултата

У претходним потпоглављима анализирани су резултати четири основне експерименталне комбинације: КЕДА и ХАСП над планом са наглим скоковима и планом са постепеним порастом оптерећења. Поред ових основних комбинација, испитана је осетљивост прага ХАСП-а (60% наспрам 70%) и осетљивост прага реда по реплици код КЕДА-е (60/80/100). У овом одељку резултати се обједињују у јединствени упоредни преглед, са циљем да се идентификују конзистентни обрасци понашања и формулише општи закључак о испитаним стратегијама аутоматског скалирања.

Упоредна табела кључних показатеља

Табела 5.13 показује доследан образац у оба плана: ХАСП остварује нижи интеграл заостатка у реду, док КЕДА остварује знатно мањи број реплика и виши композитни индекс ефикасности, уз упоредиву или нижу латенцију. Образац, дакле, представља компромис, а не једнострану доминацију: оно што ХАСП добија на дужини реда, плаћа већим бројем реплика.

Табела 5.13: Укупан преглед кључних метрика по плановима оптерећења и механизмима скалирања (средње вредности).

План	Механизам	AUC_Q	$\overline{p95}$	$T_{SLA} p95$	$\overline{N}$	AEI
Нагли скокови	КЕДА	437.085	43,45	1.418	2,09	$7,76 \times 10^{-10}$
	ХАСП	352.414	40,42	1.482	3,00	$6,38 \times 10^{-10}$
Постепени пораст	КЕДА	118.246	9,18	1.040	1,18	$7,65 \times 10^{-9}$
	ХАСП 60%	81.859	10,05	1.010	2,93	$4,16 \times 10^{-9}$
	ХАСП 70%	85.906	12,32	1.018	2,81	$4,08 \times 10^{-9}$

Анализа понашања по типу оптерећења

Нагли скокови оптерећења. У овом сценарију ХАСП остварује нижи заостатак у реду (интеграл заостатка нижи за 24,0%) и нешто нижу просечну латенцију (за 7,5% на $\overline{p95}$), али то постиже са 44% више реплика у просеку. КЕДА за исту пропусност користи 30,4% мање реплика, реагује 2,56 пута брже (краће време до ефективног повећања капацитета) и остварује краће укупно време прекорачења нивоа услуге. Дакле, у наглom профилу ХАСП оптимизује дужину реда, а КЕДА брзину одзива и ресурсну ефикасност.

Постепени пораст оптерећења. Овде је предност КЕДА у ресурсној ефикасности најизразитија: КЕДА користи 2,49 пута мање реплика уз 60,8% мању осцилацију броја реплика, остварује нижу просечну $p95$ (за 8,7%) и $p99$ латенцију (за 28,2%) и 84,0% виши композитни индекс. Истовремено, ХАСП у овом плану остварује најнижи заостатак у реду од свих сценарија (интеграл заостатка нижи за 44,5% од КЕДА), али трајним одржавањем великог броја слабо оптерећених реплика. Полазна хипотеза да ХАСП постаје конкурентнији у стабилнијем профилу делимично је потврђена, али само у погледу заостатка у реду, не и латенције или ефикасности.

Осетљивост ХАСП прага. Подизање прага са 60% на 70% доноси минималну уштеду реплика (4,0%), уз благи пораст заостатка и осетно вишу латенцију (просечна $p95$ виша за 22,6%), што показује да понашање ХАСП-а структурно одређује врста сигнала, а не конкретна вредност прага.

Осетљивост КЕДА прага. Виши праг реда по реплици (100 наспрам 60) доследно побољшава готово све показатеље (нижи заостатак, нижа латенција, мањи број реплика и стабилније реплика), јер избегава пречесто скалирање на ситне промене реда. Ово показује да ефикасност КЕДА не зависи од агресивно ниског прага.

Ефикасност коришћења ресурса

Табела 5.14: Показатељи ефикасности коришћења ресурса по сценаријима (средње вредности).

План	Механизам	$\bar{N}$	$\bar{\mu}_{\text{rep}}$	$\overline{\text{CPU}}_{\text{rep}}^2$
Нагли скокови	КЕДА	2,09	1,494	0,733
	ХАСП	3,00	0,917	0,523
Постепени пораст	КЕДА	1,18	2,147	1,139
	ХАСП 60%	2,93	0,847	0,509
	ХАСП 70%	2,81	0,889	0,554

Табела 5.14 открива разлику у начину коришћења ресурса између две стратегије. Код КЕДА-е просечна искоришћеност по реплици у постепеном сценарију износи 1,139, односно реплика у просеку ради изнад захтеваног

² $\overline{\text{CPU}}_{\text{rep}} = \overline{\text{CPU}}_{\text{total}} / N$ - просечна искоришћеност процесорске јединице по реплици, изражена апсолутним бројем процесорских језгара (не у процентима). Сервис је конфигурисан са `cpu:requests = 1000m` (1 језгро) и `cpu:limit = 2000m` (2 језгра), па вредност преко 1,0 значи да реплика прелази захтевани ресурс, али остаје у оквиру лимита.

ресурса (али у оквиру лимита). То је могуће јер КЕДА скалира на основу реда, не на основу искоришћености процесора, па свака активна реплика носи пун терет посла без превентивног подизања капацитета.

Код ХАСП-а са прагом од 60%, просечна искоришћеност по реплици износи свега 0,509, дакле репликае у просеку раде испод половине свог захтева-ног ресурса, јер ХАСП превентивно одржава вишак реплика. Иако је циљна искоришћеност 60%, стварна просечна вредност је нижа, што је директна последица прозора стабилизације и конзервативног алгоритма скалирања ХАСП механизма.

Ова разлика у искоришћености по реплици (1,139 наспрам 0,509) није последица лоше конфигурације, већ *структурна* карактеристика две стратегије: КЕДА допушта интензиван рад по реплици јер реагује директно на ред, а ХАСП то спречава скалирањем на основу сигнала процесорске јединице. Резултат је да КЕДА у постепеном плану остварује 2,54 пута виша пропусност по реплици (2,147 наспрам 0,847 порука у секунди по реплици) уз 2,49 пута мање реплика. Ова ресурсна ефикасност управо је извор предности КЕДА у композитном индексу; цена тог приступа је дужи ред порука, што чини суштину компромиса између два механизма.

Резиме налаза

На основу анализе свих сценарија, могу се формулисати следећи закључци:

- **Предност ниједног механизма није једнозначна.** Два механизма заузимају супротне тачке у компромису између дужине реда и утрошка ресурса: ХАСП у оба плана одржава нижи кумулативни заостатак у реду, али трошећи знатно више реплика.
- **КЕДА остварује исту пропусност уз вишеструко мањи број реплика и стабилније реплика.** У постепеном профилу КЕДА користи 2,49 пута мање реплика уз 60,8% мању осцилацију, а у наглом 1,44 пута мање реплика и реагује 2,56 пута брже. Просечна пропусност је у свим случајевима практично једнака.
- **КЕДА не плаћа ресурсну ефикасност латенцијом.** У постепеном плану КЕДА има чак нижу просечну $p95$ (за 8,7%) и $p99$ латенцију (за 28,2%), а у наглом плану упоредиву латенцију уз краће укупно време

прекорачења нивоа услуге. Композитни индекс ефикасности доследно је виши код КЕДА (за 21,7% у наглом и 84,0% у постепеном плану).

- **Цена приступа заснованог на реду јесте дужи ред.** КЕДА допушта да се ред привремено накупи пре реакције, што доводи до вишег интеграла заостатка (за 24-44%) у односу на ХАСП.
- **Понашање оба механизма структурно одређује врста сигнала, не конкретна вредност прага.** Подизање ХАСП прага не мења страну компромиса на којој се ХАСП налази, док код КЕДА умеренији (виши) праг даје бољи укупни компромис.

5.5 Претње валидности и ограничења резултата

Иако је експериментална поставка осмишљена тако да обезбеди контролисано и поновљиво поређење, а сваки сценарио је поновљен више пута (пет до седам извршавања) ради процене варијабилности, потребно је имати у виду неколико ограничења која утичу на домет закључака.

Варијабилност појединих метрика. Иако су резултати усредњени преко више понављања, поједине метрике показују осетну варијабилност између извршавања, нарочито код КЕДА-е у постепеном плану (нпр. стандардна девијација интеграла заостатка износи око 25% средње вредности). То значи да апсолутне вредности тих метрика треба тумачити уз припадајући распон, а не као тачне константе. Закључци рада засновани су на робусним и доследним разликама (нпр. вишеструко мањи број реплика код КЕДА), а не на малим разликама које би могле бити у оквиру варијабилности.

Синтетички планови оптерећења. Оба плана оптерећења конструисана су синтетички, са унапред дефинисаним фазама и интензитетима. Иако покривају два квалитативно различита профила (нагли скокови и постепени пораст), они не могу у потпуности да обухвате сву сложеност оптерећења у реалним продукционим окружењима. Добијене резултате стога треба тумачити као резултате у контролисаним условима, а не као универзалне вредности применљиве у свим сценаријима.

Локално Кубернетесово окружење. Сви експерименти изведени су у Миникјуб кластеру на јединственом физичком рачунару. У продукционим

окружењима са вишеструким чворовима, дистрибуираним мрежним саобраћајем, динамика скалирања се може резликовати, нарочито у погледу апсолутних вредности времена до ефективног повећања капацитета и латенције. Квалитативни обрасци понашања (нпр. предност сигнала реда над сигналом искоришћености процесорске јединице) очекују се да остану важећи, али апсолутне бројчане вредности захтевају верификацију у продукционом окружењу.

Вредност ХАСП прага и лимита броја захтева. ХАСП механизам у овом раду анализиран је са праговима 60% и 70% и фиксним параметром броја захтева који износи 1000m. Понашање ХАСП-а је изразито осетљиво на вредност броја захтева, јер различите вредности овог параметра могу значајно променити тачку активације скалирања. Иако је у раду коришћена иста вредност за оба механизма (чиме се обезбеђује фер поређење), неопходно је накнадно истраживање ширег распона вредности и параметра.

Осетљивост КЕДА сигнала на параметар `prefetch_count`. КЕДА у овом раду реагује на број порука у стању спремности за испоруку (*READY*), а не на поруке које су већ преузете и налазе се у обради (*UNACKED*). Због тога је параметар `prefetch_count` фиксиран на вредност 5: уз највише осам реплика то значи да је у сваком тренутку највише око 40 порука у стању *UNACKED* које су тиме невидљиве за сигнал скалирања. Ова вредност повољна је за сигнал заснован на стању *READY*, јер задржава готово цео заостатак видљивим у дубини реда. Међутим, знатно већи `prefetch_count` (нпр. 50) деформисао би КЕДА сигнал, јер би велик део заостатка био преузет од стране реплика и не би се рачунао у дубину реда коју КЕДА посматра. Осетљивост на овај параметар није експериментално варирана у овом раду и представља правац будућег истраживања.

Композитни индекс аутоскалирања. Индекс је уведен као синтетички показатељ за потребе овог рада и не представља стандардизовану метрику из литературе. Интерпретација индекса вредности ваљана је унутар истог експерименталног оквира, али поређење са резултатима из других радова захтева опрез.

Специфичност архитектуре и сервиса. Резултати се односе на конкретну апликативну архитектуру - асинхрони систем са *RabbitMQ* редом и *FastAPI* сервисом за извршавање модела и конкретну радну јединицу (одговарање на питања над скупом података). У системима са другачијим карак-

теристикама обраде (нпр. синхрони *REST* позиви, краћа или дужа времена извршавања модела, различити модели конкурентности), репликативна предност КЕДА-е над ХАСП-ом може бити другачија. Преношење закључака на друге архитектуре захтева додатну експерименталну потврду.

Глава 6

Закључак

У овом раду представљена је имплементација и емпиријско поређење два механизма хоризонталног аутоматског скалирања контејнеризованог сервиса за извршавање модела вештачке интелигенције у Кубернетесовом окружењу: хоризонталног аутоskalера подова, ХАСП, заснованог на искоришћености процесорске јединице, и КЕДА оквира, заснованог на дубини реда порука у распоређивачу порука *RabbitMQ*. Експериментална поставка пројектована је тако да омогући фер и репродуцибилно поређење. Оба механизма су скалирала исти циљни ресурс, са идентичним параметрима захтева и ограничења, истим опсегом реплика и истим апликативним кодом. Једина разлика међу сценаријима је сигнал на основу кога се доноси одлука о скалирању.

Резултати (табела 6.1) показују да предност ниједног механизма није једнозначна, већ да два механизма заузимају супротне тачке у компромису између дужине реда порука и утрошка ресурса. ХАСП у оба плана одржава нижи кумулативни заостатак у реду, док КЕДА истоветну пропусност постиже са вишеструко мањим бројем реплика и стабилнијим репликама, уз упоредиву или нижу латенцију и виши композитни индекс ефикасности.

Главни налаз рада је да у асинхроним системима за извршавање модела вештачке интелигенције избор сигнала скалирања представља компромис, а не једнострану предност. КЕДА омогућава ресурсно ефикасније и стабилније скалирање са бржим одзивом и без губитка у латенцији, али уз дужи ред порука. ХАСП одржава краћи ред, али трошећи знатно више и нестабилније реплика. Предност КЕДА у ресурсној ефикасности израженија је у стабилнијем профилу оптерећења, где би се интуитивно очекивало да искоришћеност процесора буде довољан сигнал за праћење.

Табела 6.1: Сумарно поређење механизма КЕДА и ХАСП по кључним метрикама (средње вредности; разлика је промена КЕДА-е у односу на ХАСП).

	AUC_Q	$\overline{p95}$	$T_{SLA} (p95)$	$\overline{N}$	АЕИ
<i>План са најлим скоковима</i>					
КЕДА	437.085	43,45	1.418	2,09	$7,76 \cdot 10^{-10}$
ХАСП	352.414	40,42	1.482	3,00	$6,38 \cdot 10^{-10}$
Разлика	+24,0%	+7,5%	-4,3%	-30,4%	+21,7%
<i>План са постојећим порастом</i>					
КЕДА	118.246	9,18	1.040	1,18	$7,65 \cdot 10^{-9}$
ХАСП	81.859	10,05	1.010	2,93	$4,16 \cdot 10^{-9}$
Разлика	+44,5%	-8,7%	+3,0%	-59,9%	+84,0%

Основни механизам уочене разлике лежи у природи сигнала скалирања. У асинхронном систему са променљивим временом извршавања модела, искоришћеност процесорске јединице не одражава директно количину накупљеног посла, већ расте тек када се обрада интензивира. ХАСП због тога превентивно одржава већи број реплика, чиме одржава краћи ред, али уз слабо оптерећене реплике. КЕДА, насупрот томе, реагује директно на дубину реда и подиже капацитет тек када се посао стварно накупи, чиме сваку реплику користи интензивније и постиже исту пропусност са делићем ресурса, прихватајући нешто дужи ред као компромис.

Анализе осетљивости потврдиле су да понашање оба механизма структурно одређује врста сигнала, а не конкретна вредност прага. Подизање циљне искоришћености ХАСП-а са 60% на 70% не мења страну компромиса на којој се ХАСП налази, а варирање КЕДА прага реда по реплици (60, 80, 100) показало је да умеренији (виши) праг доследно даје бољи укупни компромис, јер избегава пречесто скалирање на ситне промене реда.

Додатни допринос рада представља увођење композитног индекса ефикасности аутоскалирања, који интегрише кумулативни заостатак, прекорачење нивоа услуге и просечан број реплика у јединствени показатељ. Индекс се показао као користан инструмент за упоредну анализу, јер омогућава рангирање стратегија скалирања према балансу између квалитета услуге и утрошених ресурса. Ипак, треба имати у виду да индекс није стандардизована метрика из литературе и да је његова интерпретација ваљана унутар истог експерименталног оквира.

Резултате овог рада треба тумачити у контексту идентификованих ограничења. Експерименти су изведени у локалном Минијуб кластеру на једин-

ственом физичком рачунару, са синтетичким плановима оптерећења и конкретном апликативном архитектуром (асинхрони систем са *RabbitMQ* редом и *FastAPI* сервисом за извршавање модела). Квалитативни обрасци, односно предност сигнала реда над сигналом који прати искоришћеност процесорске јединице у асинхроним системима, очекују се да остану важећи и у ширим контекстима, али апсолутне бројчане вредности захтевају верификацију у продукционим окружењима.

На основу спроведеног истраживања и добијених резултата може се идентификовати неколико праваца будућег рада. Прво, верификација резултата у продукционим Кубернетес кластерима са вишеструким чворовима и реалистичним мрежним условима омогућила би увид у то колико се апсолутне вредности метрика мењају у односу на контролисано окружење. Друго, испитивање хибридних стратегија скалирања које би комбиновањем сигнала реда порука са ресурсним метрикама или предиктивним моделима и могле додатно побољшати реактивност и ефикасност аутоskalера. Треће, проширење анализе на различите типове радних оптерећења (синхрони *REST* системи, системи за извршавање модела помоћу графичких процесора, оптерећења са хетерогеним временом обраде) допринело би генерализацији закључака. Четврто, систематско испитивање ширег простора параметра ХАСП механизма, укључујући различите вредности броја захтева код сигнала који прати искоришћеност ресурса процесорске јединице (`cpu:requests`), прозоре стабилизације и комбинације метрика, могло би прецизније одредити границе применљивости ресурсног скалирања у асинхроним системима.

Добијени резултати пружају емпиријски засноване смернице за избор стратегије скалирања: у асинхроним системима са редом порука и променљивим временом извршавања модела, КЕДА представља ресурсно ефикаснију и стабилнију алтернативу скалирању заснованом на искоришћености процесорске јединице, без губитка у латенцији и стога је пожељан избор када су приоритет ефикасност и стабилност. Уколико је, међутим, кључни критеријум минимална дужина реда порука (нпр. због строгих ограничења на број порука на чекању), ХАСП може бити оправдано, уз свест о вишем и нестабилнијем утрошку ресурса који тиме настаје.

Библиографија

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve, 2024.
- [2] AMQP Working Group. Advanced message queuing protocol (amqp) version 0-9-1 specification. Technical report, AMQP Working Group, 2008.
- [3] Apdex Alliance. Apdex technical specification. Technical Specification Version 1.1, Apdex Alliance, Inc., 2007. Приступљено: јун 2026.
- [4] Dariusz R. Augustyn, Łukasz Wyciślik, and Mateusz Sojka. Tuning a Kubernetes horizontal pod autoscaler for meeting performance and load demands in cloud deployments. *Applied Sciences*, 14(2):646, 2024.
- [5] The Kubernetes Authors. Official Kubernetes Documentation, 2025. on-line at: <https://kubernetes.io/docs/home/>.
- [6] B. Burns, J. Beda, K. Hightower, and L. Evenson. *Kubernetes: up and running: dive into the future of infrastructure.* „ O’Reilly Media, Inc.”, 2022.
- [7] Andrzej Chydzinski and Błażej Adamczyk. Response time of queueing mechanisms. *Symmetry*, 16(3):271, 2024.
- [8] Russ Cox, Robert Griesemer, Rob Pike, Ian Lance Taylor, and Ken Thompson. The Go programming language and environment. *Communications of the ACM*, 65(5):70–78, may 2022.
- [9] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627. USENIX Association, 2017.

- [10] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.
- [11] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56:74–80, 2013.
- [12] Inc. Docker. Official Docker Documentation - dockerdocs, 2025. on-line at: <https://docs.docker.com/>.
- [13] Mohamed Hedi Fourati, Semia Marzouk, and Mohamed Jmaiel. Towards microservices-aware autoscaling: A review. In *2023 IEEE Symposium on Computers and Communications (ISCC)*, pages 1080–1083. IEEE, 2023.
- [14] Mohamed Imdoukh, Imtiaz Ahmad, and Mohammad G. Alfaiakawi. Machine learning-based auto-scaling for containerized applications. *Neural Computing and Applications*, 32:9745–9760, 2020.
- [15] Josephine Eskaline Joyce and Shoney Sebastian. An experimental evaluation of horizontal pod autoscaler for diverse machine learning workloads in kubernetes, April 2024. Preprint.
- [16] KEDA Authors. Keda official documentation. <https://keda.sh/>, 2026. Приступљено: 2026-02-27.
- [17] M. Luksa. *Kubernetes in action*. Simon and Schuster, 2017.
- [18] G. Mirsky, J. Halpern, X. Min, A. Clemm, J. Strassner, and J. Francois. Rfc 9544: Precision availability metrics (pams) for services governed by service level objectives (slos), March 2024. RFC 9544.
- [19] Siddharth Mitra. Comparative study between gpu utilization and inflight requests based autoscaling systems. Master’s thesis, University of Groningen, 2021. Computing Science.
- [20] Thanh-Tung Nguyen, Young-Jin Yeom, Taehong Kim, Dae-Heon Park, and Sungwoo Kim. Horizontal pod autoscaling in Kubernetes for elastic container orchestration. *Sensors*, 20(16):4621, 2020.
- [21] J. Nickoloff and S. Kuenzli. *Docker in Action, Second Edition*. Manning, 2019.

- [22] Oracle. Java se documentation, 2026. on-line at: <https://docs.oracle.com/en/java/>.
- [23] István Pintye, József Kovács, and Róbert Lovas. Enhancing machine learning-based autoscaling for cloud resource orchestration. *Journal of Grid Computing*, 22:68, 2024.
- [24] N. Poulton. *The Kubernetes Book*. NIGEL POULTON LTD, 2023.
- [25] N. Poulton. *Docker Deep Dive: Zero to Docker in a single book*. Packt Publishing Ltd, 2025.
- [26] Python Software Foundation. Python 3 documentation, 2026. on-line at: <https://docs.python.org/3/>.
- [27] Chenhao Qu, Rodrigo N. Calheiros, and Rajkumar Buyya. Auto-scaling web applications in clouds: A taxonomy and survey. *ACM Computing Surveys*, 51(4):1–33, 2018.
- [28] RabbitMQ Team. Rabbitmq official documentation. <https://www.rabbitmq.com/documentation.html>, 2025. Приступљено: 2026-02-27.
- [29] Sebastián Ramirez. Fastapi documentation, 2026. on-line at: <https://fastapi.tiangolo.com/>.
- [30] Christian Schroeder, Rene Boehm, and Alexander Lampe. Comparison of autoscaling frameworks for containerised machine-learning-applications in a local and cloud environment, 2024.
- [31] László Toka, Géza Dobreff, Balázs Fodor, and Balázs Sonkoly. Adaptive AI-based auto-scaling for Kubernetes. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, pages 599–608. IEEE, 2020.
- [32] Alvaro Videla and Jason J. W. Williams. *RabbitMQ in Action: Distributed Messaging for Everyone*. Manning Publications, Shelter Island, NY, 2012.
- [33] VMware, Inc. and Spring Team. Spring boot documentation, 2026. on-line at: <https://docs.spring.io/spring-boot/index.html>.
- [34] Haibin Yuan and Shengchen Liao. A time series-based approach to elastic kubernetes scaling. *Electronics*, 13(2), 2024.

Биографија аутора

Александар Н. Јаковљевић (*Краљево, 18. јануар 1996.*) је завршио основне академске студије на Математичком факултету Универзитета у Београду, на студијском програму Информатика. Мастер академске студије уписао је на истом факултету, такође на студијском програму Информатика.

Од 2019. године запослен је као софтверски инжењер у компанији *Ezderm*, где се бави развојем, дизајном и архитектуром софтверских система.